A Survey on Inference Engines for Large Language Models: Perspectives on Optimization and Efficiency

SIHYEONG PARK, Korea Electronics Technology Institute, South Korea
SUNGRYEOL JEON, Mobilint, South Korea
CHAELYN LEE, Korea Electronics Technology Institute, South Korea
SEOKHUN JEON, Korea Electronics Technology Institute, South Korea
BYUNG-SOO KIM, Korea Electronics Technology Institute, South Korea
JEMIN LEE*, Jeonbuk National University, South Korea

Large language models (LLMs) are widely applied in chatbots, code generators, and search engines. Workload such as chain-of-thought, complex reasoning, and agent services significantly increase inference cost by invoking the model repeatedly. Optimization methods such as parallelism, compression, and caching have been adopted to reduce costs, but the diverse service requirements make it hard to select the right method. Recently, specialized LLM inference engines have emerged as a key component for integrating the optimization methods into service-oriented infrastructures. However, a systematic study on inference engines is still lacking. This paper provides a comprehensive evaluation of 25 open-source and commercial inference engines. We analyze each inference engine in terms of ease-of-use, ease-of-deployment, general-purpose support, scalability, and suitability for throughput- and latency-aware computation. Furthermore, we explore the design goals of each inference engine by investigating the optimization techniques it supports. In addition, we assess the ecosystem maturity of open-source inference engines and handle the performance and cost policy of commercial solutions.We outline future research directions that include support for complex LLM-based services, support of various hardware, and enhanced security, offering practical guidance to researchers and developers in selecting and designing optimized LLM inference engines. We also provide a public repository to continually track developments in this fast-evolving field: https://github.com/sihyeong/Awesome-LLM-Inference-Engine.

CCS Concepts: • **General and reference** → *Surveys and overviews*; • **Software and its engineering** → **Development frameworks and environments**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Large Language Model, Transformer, Inference Engine, Framework, Optimization

1 Introduction

Large Language Models (LLMs) are being utilized in a wide range of services, such as chatbots, code generation, and search engines, with remarkable examples including OpenAI's ChatGPT [5], GitHub Copilot [103], and

*Corresponding Author

Authors' Contact Information: Sihyeong Park, sihyeong@keti.re.kr, Korea Electronics Technology Institute, Seongnam-si, Gyeonggi-do, South Korea; Sungryeol Jeon, Mobilint, Seoul, South Korea, sungryeol@mobilint.com; Chaelyn Lee, Korea Electronics Technology Institute, Seongnam-si, Gyeonggi-do, South Korea, mylynchae@keti.re.kr; Seokhun Jeon, Korea Electronics Technology Institute, Seongnam-si, Gyeonggi-do, South Korea, seokhun.jeon@keti.re.kr; Byung-Soo Kim, Korea Electronics Technology Institute, Seongnam-si, Gyeonggi-do, South Korea, bskim4k@keti.re.kr; Jemin Lee, Jeonbuk National University, Jeonju-si, South Korea, jemin.lee@jbnu.ac.kr.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2157-6912/2026/3-ART
https://doi.org/10.1145/3803798

Just Accepted

Google Gemini [107]. Building on these successes, numerous new models and services have rapidly emerged; however, this expansion introduces new challenges in deploying and serving LLMs on a scale.

Recent trends like reasoning-centric test-time scaling [165, 301] and LLM-based AI agents [115, 179] have significantly increased both computational demands and the number of inference calls in LLM-based applications. Reasoning-centric test-time scaling replaces single-pass answer generation with multi-step reasoning or iterative self-verification to improve output quality. Also known as chain-of-thought (CoT) [346], self-consistency [53], and test-time reasoning [123], these methods increase accuracy by invoking the model multiple times per query, thereby raising latency and computing costs. Meanwhile, LLM-based AI agents such as AutoGPT [30] and LangChain [167] autonomously plan a sequence of tasks to fulfill a single user request, repeatedly calling the model within a single session. Consequently, inference efficiency has become essential for deploying both reasoning-oriented LLMs and AI agents in practice.

To manage the growing inference costs of LLMs, various optimization techniques—such as quantization [76], lightweight architectures [358], and knowledge distillation (KD) [364]—have been adopted. In large-scale services, however, the diversity of prompt lengths, query types, and output formats often means that a single optimization method cannot cover every scenario. As a result, LLM inference engines, which offer multiple optimization strategies and handle the inference process, have become crucial infrastructure components that directly affect both service quality and cost.

Although general-purpose deep learning frameworks like PyTorch [263] and TensorFlow [1]—originally designed to support a wide range of models, from convolutional neural networks (CNNs) to recurrent neural networks (RNNs)—are widely used for LLM inference, they prioritize broad hardware and architecture compatibility. Consequently, they do not include various specialized optimizations for LLMs or for sequential decoding. Running large-scale models on these frameworks can lead to slower performance and higher resource usage, underscoring the need for dedicated inference solutions.

Reflecting this need, a growing number of specialized LLM inference engines have emerged. They provide capabilities such as batching, streaming, and attention optimizations that are not typically found in general-purpose frameworks. However, Each engine targets different hardware such as graphics processing units (GPUs) and LLM accelerators, optimization scopes ranging from model compression to memory offloading, and intended use cases varying from real-time conversational systems to large-scale text generation. As a result, the ecosystem has become both rapidly evolving and fragmented, making it difficult to determine which optimization methods are supported by each engine and how effectively they perform under various conditions. Consequently, there is a pressing need for a comprehensive review and comparison of LLM inference engines and the optimization techniques they offer.

Most existing surveys on LLM optimization (Table 1) have focused on specific methods, such as model compression or hardware acceleration, and therefore have not fully explored which optimization techniques are supported by individual inference engines. In addition, many of these surveys omit recently released commercial engines. For instance, Chitty-Venkata et al. [56] and Yuan et al. [382] focus on transformer-based model compression, while Park et al. [262] and Zhu et al. [407] examine compression methods in detail. Similarly, works such as Xu et al. [358], Xu et al. [357] and Wang et al. [340] discuss optimization strategies for LLM inference or serving systems in cloud or edge environments, but they lack a detailed examination of the design and implementation of each engine. Consequently, there remains a gap in the literature for a survey that not only presents the current landscape of LLM inference engines, but also systematically links their specialized features to the optimization techniques they implement.

To fill this gap, this paper adopts a framework-centric perspective, thoroughly examining a range of LLM inference engines and categorizing the optimization techniques each one implements. In particular, it maps how these engines handle methods like quantization, caching, and parallelization, enabling readers to quickly identify engines that align with specific requirements. This paper also includes recently released commercial engines

Just Accepted

Table 1. Comparison of Representative Surveys on Efficient LLM Inference

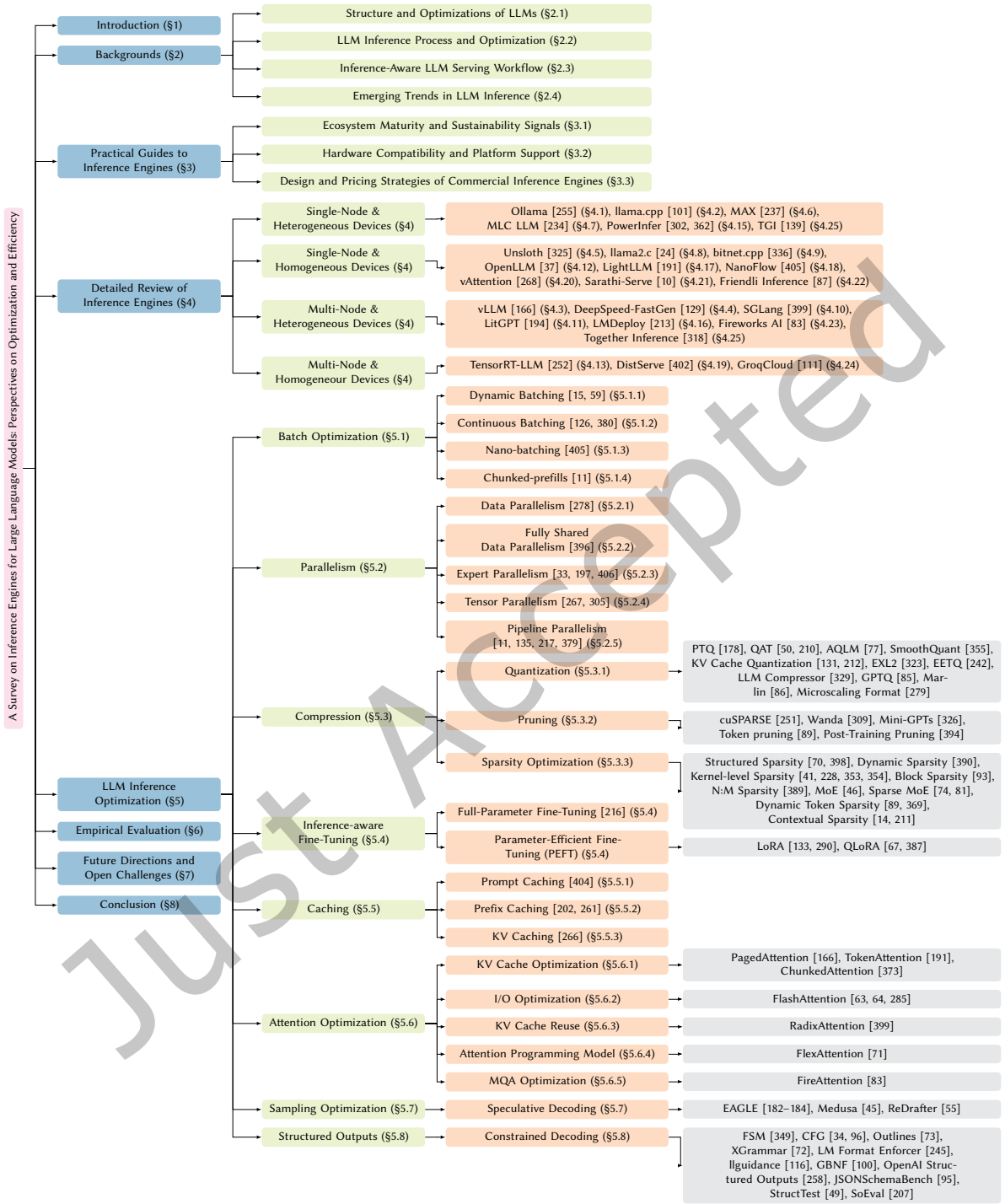
Survey	Scope	# of Reviewed Inference Engine	Limitation
Chitty-Venkata et al., JSA (2023) [56]	**Efficient inference** - architecture design, knowledge distillation, pruning, quantization	✘	Covers only optimization techniques for efficient inference
Miao et al., ArXiv (2023) [230]	**Efficient model serving** - decoding algorithms, architecture design, model compression, quantization, parallel computation, memory management, request scheduling, kernel optimizations	10	Covers only parallel computation, iteration scheduling, attention kernel support, and brief main features of the inference engine
Bai et al., ArXiv (2024) [32]	**Resource-efficient model** - architecture design, pre-training, fine-tuning, inference optimization, system design	✘	Covers only model-side optimization techniques for efficient inference
Xu et al., ArXiv (2024) [358]	**Resource-efficient foundation models** - foundation model, architecture design, resource-efficient algorithms, resource-efficient systems	23	Provides information on training and inference support in cloud and edge environments, as well as inference optimization techniques, but lacks detailed description of the inference engine.
Park et al., ArXiv (2024) [262]	**Model compression** - pruning, quantization, knowledge distillation, low-rank approximation, parameter sharing, architecture design	✘	Covers only model compression techniques
Yuan et al., ArXiv (2024) [382]	**Efficient inference** - model compression, fast decoding algorithm, compiler/system optimization, hardware optimization	✘	Covers only optimization techniques for efficient inference
Zhu et al., TACL (2024) [407]	**Model compression** - quantization, pruning, knowledge distillation, low-rank factorization	✘	Covers only model compression techniques
Wang et al., ArXiv (2024) [340]	**Model compression and efficient inference** - quantization, pruning, knowledge distillation, architecture design, framework	6	Provides explanations focused on optimization features rather than the inference engine itself, and includes outdated inference engines
Zhou et al., ArXiv (2024) [403]	**Efficient inference** - data-level optimization, architecture design, model compression, inference engine, serving system	18	Explores optimization techniques from the perspectives of inference optimization and serving optimization, but describes only a limited set of techniques
Wan et al., TMLR (2024) [331]	**Efficient models** - model optimization schemes, data selection/engineering, framework	18	Describes training, fine-tuning, and inference support of the inference engine along with key features, but takes a more comprehensive view rather than focusing specifically on inference
Li et al., ArXiv (2024) [176]	**Hardware perspective inference optimization** - hardware architecture (CPU, GPU, FPGA, ASIC, PIM/NDP), quantization, sparsity, speculative decoding, homogeneous/heterogeneous cooperation	✘	Covers only hardware-aware optimization techniques for efficient inference
Xu et al., CSUR (2025) [357]	**Resource-efficient algorithms** - attention optimization, architecture design, pre-training, fine-tuning, inference algorithm, model compression, distributed training, serving	18	Covers training and inference support, but lacks sufficient description of the inference engine
Zheng et al., CSUR (2025) [401]	**Efficient models** - model compression, runtime optimization, on-device applications	8	Covers mobile and desktop support and related optimization techniques only briefly
Ours	**Inference engine and efficient inference** - open-source and commercial inference engine, inference optimization of inference engine	**25 (New: 13)**	Covers only covers inference engines and their optimizations

that are not covered in previous surveys, comparing their architectural goals, hardware targets, and significant features.

This study provides a comprehensive analysis not only of previously examined inference engines such as vLLM [166], llama.cpp [101], MLC-LLM [234], and Sarathi-Serve [10], but also of recently released frameworks including MAX [237], LitGPT [194], and vAttention [268]. In contrast to previous work, which has mainly focused on presenting optimization techniques offered by each engine, we also address practical indicators such as ecosystem maturity of open-source projects and the ease of installation and deployment. Furthermore, we conduct a comparative analysis of each inference engine from the perspectives of throughput-aware, latency-aware, and scalability design, thereby presenting selection criteria suitable for real-world service environments.

The goal is to offer practical insights for researchers and engineers who need to build or operate high-performance, cost-efficient LLM services.

Just Accepted



Just Accepted

Fig. 1. Taxonomy of LLM Inference Engines and Optimizations

As shown in Fig. 1, this paper systematically organizes the major LLM inference engines and their respective optimization methods. Section 2 outlines the core aspects of decoder-based transformer architectures, attention mechanisms, and the standard LLM inference process. Section 3 presents a comprehensive review of the leading LLM inference engines, including ecosystem, hardware and operating system (OS) support. In particular, commercial offerings are discussed to help readers find suitable solutions for their own service environments and deployment objectives. To this end, we analyzed various aspects of inference engines, including their ecosystem, usability, as well as their support for hardware and platforms across both edge and server environments. Section 4 offers a detailed discussion of the architectures of various LLM inference engines and the inference-specific optimization features offered by each engine. Section 5 classifies fundamental inference optimization techniques found in current inference engines—covering batch optimization (§ 5.1), parallelization (§ 5.2), model compression (§ 5.3), fine-tuning (§ 5.4), caching (§ 5.5), attention optimization (§ 5.6), sampling optimization (§ 5.7) and structured outputs (§ 5.8)—while also examining emerging trends. By synthesizing these techniques, the chapter helps readers choose the inference engine that best matches their service requirements. Based on these discussions, Section 7 explores future directions and major challenges in the development of LLM inference engines. Specifically, we examine the ongoing evolution of LLMs and how inference engines accommodate these changes, with particular attention to security and compatibility across diverse hardware platforms. We present perspectives on multiple aspects, including inference engine optimization strategies, security for inference and support for diverse hardware platforms and architectures. Finally, section 8 concludes the paper.

2 Backgrounds

To enhance the efficiency of LLM inference, it is crucial not only to select a model suited to the domain but also to choose and optimize an appropriate inference engine while taking a diverse approach to overall development. This section examines LLMs from the perspective of inference, demonstrating how tasks such as model compression and deployment strategies can be seamlessly integrated with inference engines to achieve fast and cost-effective services.

First, we review the decoder-only transformer architecture, along with various attention mechanisms and considerations related to efficient inference. Second, we explain the inference process, focusing on the prefill and decode phases, and highlight corresponding optimization techniques from the inference engine perspective. Finally, we combine these elements to provide a comprehensive overview of the entire pipeline for inference and service deployment.

2.1 Structure and Optimizations of LLMs

LLM Architecture Types. LLMs can be broadly categorized into three types based on the Transformer architecture [327]: decoder-only, encoder-decoder, and encoder-only models. The encoder-decoder model first encodes the entire input and then uses the decoder with cross-attention at each step, which leads to higher memory usage and more complex procedures during inference. Encoder-only models are suitable for tasks like classification or retrieval, but since they are optimized for one-time inference, they are not ideal for token-by-token generation.

In contrast, decoder-only models have a simpler structure and are widely adopted in recent LLMs due to their strong zero-shot performance through autoregressive training [339, 345]. Therefore, this paper mainly focuses on the decoder-only architecture.

Standard Decoder-Only Architecture. Fig. 2 shows the architecture of a decoder-only transformer. When a text input is received, it is first tokenized and then converted to high-dimensional vectors by an embedding layer. At this stage, positional encoding is added to incorporate token order. The resulting embeddings pass through several transformer blocks, each comprising Multi-Head Attention (MHA), a Feed-Forward Network (FFN), and residual connections. The MHA layer splits the input into Query (**Q**), Key (**K**), and Value (**V**) vectors

Just Accepted

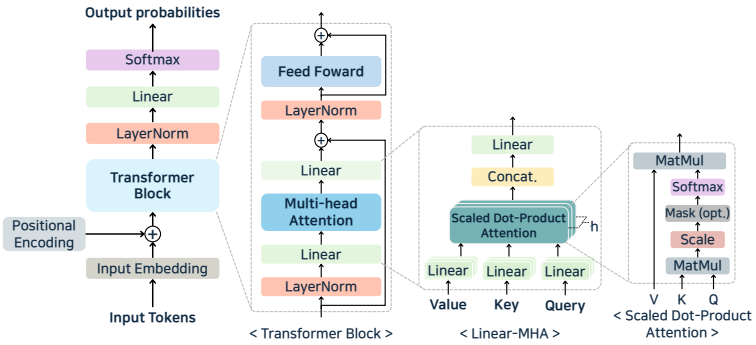


Fig. 2. Overview of Decoder-only Transformer Architecture

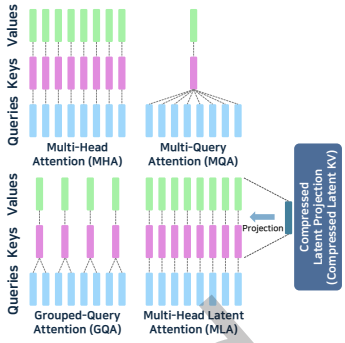


Fig. 3. Attention Mechanism

and performs scaled dot-product attention in parallel across multiple heads. In each head, $\mathbf{Q} - \mathbf{K}$ similarity scores are computed and applied to $\mathbf{V}$, aggregating the results. Causal masking ensures that only previously generated tokens are attended to, enabling autoregressive context learning. Next, the FFN layer refines the attention output by applying a linear transformation, expanding it to a higher-dimensional space, and using an activation function (e.g., ReLU [6], GELU [127], or SiLU [78]) before reducing it back to the original dimension. This sequence of operations increases the model's representational capacity. Both the MHA and FFN layers employ residual connections and layer normalization. Residual connections mitigate the vanishing gradients in deep networks [383], and layer normalization keeps the output distributions stable, facilitating smoother training.

After the transformer block operations are finished, each input token produces a hidden state which is normalized and used to predict the next token in text generation. The hidden state is then passed through a linear layer, resulting in a logit vector over the vocabulary. Applying the softmax function converts these logits into a probability distribution, and the token with the highest probability is chosen as the next token. This procedure is repeated iteratively to generate the final text.

Attention Structure Variants. In a standard transformer, MHA is employed, but recent modifications—like the one shown in Fig. 3—have been introduced to improve inference efficiency. In MHA, each of the N_h heads uses its own $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ matrices, enabling the model to learn distinct subspace representations. However, increasing the number of heads also expands the size of the Key-Value (**KV**) cache during inference, because all $\mathbf{K}$ and $\mathbf{V}$ values must be stored. To address this, Multi-Query Attention (MQA) [288] was proposed. MQA retains multiple query heads while sharing a single set of $\mathbf{K}$ and $\mathbf{V}$ across all heads, thereby reducing the **KV** cache to roughly $1/N_h$ of what MHA requires. Although this approach may slightly reduce expressiveness, it significantly decreases memory usage. Grouped-Query Attention (GQA) [13] takes a middle ground by sharing K and V among head groups rather than across all heads. By tuning the number of groups (N_g), developers can strike a balance between memory efficiency and model performance. More recently, DeepSeek-v2 [196] introduced Multi-Head Latent Attention (MLA), which compresses $\mathbf{K}$ and $\mathbf{V}$ from multiple heads into a shared latent vector. This design further minimizes cache size while preserving accuracy. Because these alternative attention mechanisms alter the size and structure of the **KV** cache, inference engines must adapt accordingly. For instance, MQA and GQA require cache management that reflects shared $\mathbf{K}$ and $\mathbf{V}$, whereas MLA involves reconstructing compressed $\mathbf{K}$ and $\mathbf{V}$. As a result, the compatibility of an inference engine can vary depending on the attention structure of the model.

Variants in Positional Embedding, Tokenization and Normalization. In LLMs, key architectural variants include the type of positional embedding, tokenizer choice, and the placement of normalization layers. Even well-known LLMs adopt different configurations: BLOOM [350] uses Attention with Linear Biases (ALiBi) [269], while Llama [109, 320, 321] and Mistral [148] employ Rotary Position Embedding (RoPE) [307]. The selection of

Just Accepted

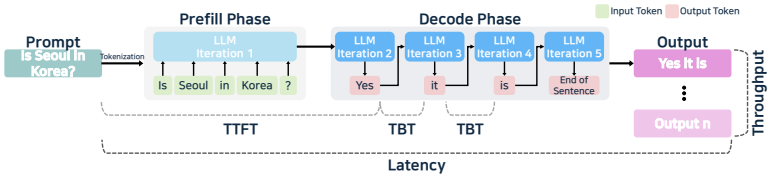


Fig. 4. LLM Inference Process

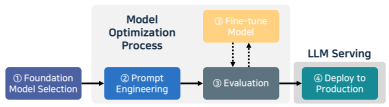


Fig. 5. Inference and Serving Process of LLM

tokenizer also varies across models-GPT variants typically use Byte-Pair Encoding (BPE) tokenizers [408], whereas Llama [109, 320, 321] and T5 [275] rely on SentencePiece-based unigram tokenizers [164]. Other architectural differences include the placement of normalization layers [276].

2.2 LLM Inference Process and Optimization

LLM inference proceeds by tokenizing the user's input text and generating subsequent tokens until a stopping criterion (e.g., token limit or end-of-sequence (EOS) command) is met. As shown in Fig. 4, this process comprises two main phases: the prefill phase, which generates the first token based on the input, and the decode phase, which sequentially produces the remaining tokens.

Prefill phase. This phase processes the input text to compute the hidden state of the last token for each sample, thereby capturing the contextual and semantic meaning of the input. In decoder-only transformer models, this phase involves tokenization, embedding, and transformer block computations. Attention and FFN operations are performed on all input tokens. In this step, attention scales approximately with the square of the sequence length n ($\mathcal{O}(n^2)$), and the complexity of FFN increases with the size of the intermediate layer, resulting in large-scale array-to-array computations. **Q**, **K**, and **V**—used to capture relationships among all input tokens—are generated immediately, loading substantial data into memory for intensive computation.

For example, as shown in Fig. 4, if the user input is `Is Seoul in Korea?`, it is tokenized into [`Is`, `Seoul`, `in`, `Korea`, `?`], mapped to a unique token value such as [101, 4523, 1102, 2342, 63]. Position embeddings are then applied to these token IDs, converting them into high-dimensional vectors (e.g., [[0.1, 0.2, ...], [0.3, 0.5, ...], ...]). These vectors undergo attention and FFN computations, allowing the model to learn contextual relationships among tokens and refine their representations. Finally, the hidden state of the last token (?) is stored for use in the decode phase, where it guides the generation of subsequent tokens.

Decode phase. This phase iteratively generates new tokens based on the hidden state computed during the prefill phase, following an autoregressive process in which only one token is produced at a time. In this phase, the final hidden state of the transformer block is passed through a linear transformation and a softmax function, which yields a probability distribution over the vocabulary. The token with the highest probability is selected and appended to the input sequence. Throughout this process, **K** and **V**, as well as the input and output tokens, are stored on the GPU, system memory, or cache. Because **K** and **V** must be accessed and updated repeatedly, the decode phase often becomes a limitation of the memory bandwidth. Although the attention computation resembles that of the prefill phase, frequent reference to previously generated tokens increases latency, and the data to be accessed grows linearly with the sequence length.

Specifically, during the decode phase, the hidden state of the last token (?) saved in the prefill phase is used to predict the next token. For example, when the attention mechanism processes the newly generated token, the transformer block produces a final hidden state that is then linearly projected into a logit vector over the vocabulary. After softmax is applied, the most probable word—say, `Yes`—is chosen and appended to the existing sequence. Repeating this procedure can generate a full response, such as `Yes it is`, as shown in Fig. 4.

Just Accepted

Table 2. Key metrics of LLM performance

Metric	Definition	User Perspective	Optimization Technique
Time-to-first-token (TTFT)	Time taken for the model to generate the first token	Most directly impacts the user's perception of response speed	Batching (§5), Kernel fusion, Prompt caching (§5.5.1), Speculative decoding (§5.7)
Time-between-tokens (TBT)	Time interval between each token	Reflects the speed at which subsequent tokens are generated	KV caching (§5.5.3), Kernel fusion, Attention optimization (§5.6), Quantization (§5.3.1)
End-to-end latency	Total time from client request to complete response	Reflects overall response time and user experience	Batching (§5), KV caching (§5.5.3), Pruning (§5.3.2), Speculative decoding (§5.7), FlashAttention (§5.6.2)
Throughput	Number of tokens processed per unit time	Represents the system's processing capacity	Batching (§5), Prefill optimization (§5.1.4), Parallelism (TP/PP) (§5.2), Quantization (§5.3.1)

System terms. The performance terms of an LLM system are illustrated in Fig. 4 and the accompanying Table 2. Time-To-First-Token (TTFT) measures the time it takes to receive a user request to generate the first token. It is especially important for how fast the system feels to the user. Time-Between-Tokens (TBT) (or Inter-token Latencies (ITL)) refers to the time it takes to generate each following token. It is often described as Time Per Output Token (TPOT), which is the average token generation speed during decoding. In addition, end-to-end latency represents the total response time for a user query and can be calculated as: Latency = $TTFT + (TBT \times number\ of\ tokens)$) While latency gives an overall measure of responsiveness, throughput shows how many user requests the system can handle at the same time.

From a phase-wise perspective, the prefill phase affects TTFT and the decode phase impacts TBT. The latency of the prefill phase increases with input length, but can be reduced using parallel computation. On the other hand, latency in the decode phase grows with the number of generated tokens and has a more direct impact on the user experience.

Optimization. Taking these performance metrics into account, LLM inference engines employ various customized optimization techniques for the prefill and decode phase. Most engines use **KV** caching to avoid redundant computation during decoding by reusing cached context and computing new operations only for the latest token. Recently, techniques such as continuous batching [380] and hybrid batching [154] have been introduced to further improve decode phase efficiency, group prefill, and decode operations from multiple requests to better use GPU resources.

In addition, many inference engines reduce per-token overhead during decoding through kernel fusion [82, 310, 395] and hardware-specific computation kernels. Kernel fusion consolidates operations—such as LayerNorm, matrix multiplication, and activation functions—into a single GPU kernel, which decreases memory access and kernel launch overhead.

Quantization [76] is another key optimization. By representing model parameters in 8-bit or 4-bit integers instead of 16-bit or 32-bit floating-point formats, memory usage and bandwidth demands drop, especially during decoding. Quantized models can cache more tokens and handle more concurrent requests on the same hardware, often boosting the computation speed.

In general, caching, batching, kernel optimization, and quantization are fundamental to optimizing token throughput and minimizing latency in LLM inference services. Providing robust support for these techniques within an inference engine is crucial for delivering high-quality, scalable LLM solutions.

2.3 Inference-Aware LLM Serving Workflow

LLM development typically involves gathering training data, pretraining on a large corpus, and then aligning and evaluating the resulting model. For production, inference often relies on a pretrained foundation model [358]. This complete pipeline is commonly referred to as LLM Operations (LLMOps) and, as shown in Fig. 5, consists of four.

Just Accepted

❶ **Model selection.** Selecting a model and an inference engine that match service-level requirements, performance needs, and available hardware is crucial for a successful LLM deployment. A model might be well suited to the target domain but incompatible with a specific inference engine—so both factors must be considered together. When choosing an inference engine, it is equally important to assess the expected user concurrency and service-level objectives (SLO), then select a solution capable of meeting the necessary latency and throughput goals. Ultimately, the design principles and implementation of the inference engine dictate achievable performance, ease of integration, and general ease of use.

❷ **Prompt engineering.** This step involves optimizing how the model is prompted and deployed. Prompt design can significantly influence model performance, as developers carefully craft system messages and user prompts to ensure consistent, high-quality outputs. This practice is known as prompt engineering [374], which directs the model to produce desired responses without requiring additional computation. For example, a well-structured system prompt can adjust the model tone or decrease inappropriate responses, reducing trial-and-error during inference and contributing to more stable operation. During development, prompt templates undergo iterative testing and revision so that, in production, the model achieves the intended output with minimal further tuning.

❸ **Evaluation and fine-tuning.** When prompt design is completed, the model must be evaluated to verify if it achieves the required level of performance. If not, fine-tuning can be applied to enhance accuracy or domain-specific capabilities. For example, instruction tuning [385] can train the model with instruction response datasets to increase accuracy or domain-related responses. Other techniques include prompt tuning [172], which adds task-optimized vectors to input embeddings, and prefix tuning [330], which modifies the model by inserting trainable parameters into hidden states at all layers. If the model size exceeds the available hardware, quantization can be used to compress activations or weights. Post-training quantization [85, 178, 355] is based on a calibration dataset to calculate scaling parameters, converting weights or activations to lower precision. Alternatively, quantization-aware training [50, 210] simulates quantized conditions during training, ensuring that the model retains accuracy despite low-precision weights.

❹ **Deployment.** Once an LLM achieves the desired performance level after fine-tuning, it should be prepared for production deployment. A key decision at this point involves choosing between a cloud application programming interface (API) or on-premise hosting. Cloud APIs (i.e., external LLM services) offer quick setup and the flexibility to scale with changing workloads, but they depend on external infrastructure and may raise data privacy issues. Because each query traverses a network, latency increases, making cloud APIs potentially unsuitable for latency-critical use cases. However, hosting LLMs on-premise avoids these concerns and markedly reduces latency. Eliminating network overhead accelerates response times, and keeping data inside internal systems improves privacy. Additionally, on-premise hosting allows for fine-grained control over model parameters and hardware configurations. Although it can require substantial infrastructure investment, this approach may prove more economical for large-scale services. Given these factors, it is advantageous to consider the deployment methods early in development. Aligning the model to the intended inference environment (for example, by quantization [76] or KD [364]) and selecting an appropriate inference engine from the outset can streamline the process.

2.4 Emerging Trends in LLM Inference

Traditional CNN and Deep Neural Network (DNN) workloads assume fixed input sizes and regular computation graphs. Larger batch sizes and kernel tuning therefore deliver almost linear throughput gains, while techniques such as layer partitioning, offloading, and pipelining remain reliable ways to trim latency or boost throughput. LLM inference faces different constraints because it generates token-by-token text inference.

- **Step-by-step execution.** Prefill and decode run in separate phases, and each has its own bottlenecks.

Just Accepted

Table 3. Comparison between CNN/DNN and LLM Inference Workloads

Category	CNN/DNN Workload	LLM Inference Workload
Input/Computation Graph	Fixed-size inputs, regular convolution graphs; favorable to large-batch scaling; partitioning and pipelining work reliably	Token streaming-based autoregressive; heterogeneity between prefill-decode phases causes interference and bottleneck shifting
Memory Characteristics	Feature map reuse and locality-centric; memory hierarchy design highly predictable	Dominated by KV-cache capacity/bandwidth bottlenecks
Latency/Bottlenecks	Compute-bound tendency; FLOPs utilization is a key indicator	Memory-bound with TBT; heterogeneous bottlenecks coexist across stages
Batching/Scheduling	Large batches yield near-linear throughput scaling; partitioning and scheduling alone are effective	Interference between prefill-decode in mixed traffic; preventing decode hotspots is critical
Compression/Quantization	INT8/FP16 maximizes kernel efficiency; serving benefits are relatively intuitive	Ultra-low precision (e.g. MXFP4, 1-bit) suffers from dequantization/kernel path overhead, offsetting gains
Offloading/Communication	Predictable feature-map transfers; static pipelining effective	GPU-CPU-Memory transfers fluctuate dynamically; static offloading risks idle time
Energy/Operations	Cluster operations driven by throughput and latency metrics	Simultaneous optimization of Perf/$, Perf/W, and SLO; Dynamic Voltage and Frequency Scaling (DVFS) effective when leveraging repetitive structures
Edge/Distributed	Partitioning-scheduling co-optimization effective in collaborative inference	On-device speculative inference, collaborative sharding, migration require dynamic decision-making

- **KV cache.** Long contexts require large memory capacity and high bandwidth to store and fetch key-value pairs.
- **Real-time requirements.** Long CoT prompts [346], structured outputs [201], and many concurrent requests often overlap, therefore, systems must trade off latency against stability.

Recently, LLMs have moved beyond short input and output pairs to tasks such as CoT [346], reasoning [165], long context processing [313], structured output [201], high concurrency and operation under strict energy and cost limits. This shift increases the need for inference engines and system-level optimizations that target the distinct bottlenecks of the prefill and decode phases. Workloads with long input contexts often reach memory and bandwidth limits during the prefill phase, while tasks such as mathematical problem solving or code generation, which produce long outputs, hit latency limits during the decode phase.

Therefore, unlike CNNs, LLM inference cannot rely on high-performance kernels and large batch sizes. It needs engine-level methods such as phase separated batching and scheduling, hardware and software co-design, quantization, and adaptive offloading. A holistic approach that integrates web services, inference engines, and system infrastructure is also essential to efficiently handle many concurrent user requests. Table 3 compares the main differences between CNN and LLM inference workloads.

Recent trends in LLM inference include the following:

- **Spread of CoT and inference intensive workloads.** CoT [346] improves the accuracy of complex problems by explicitly generating intermediate reasoning steps, which increases the fraction of workloads that are inference-heavy at decode time. As the explanation-implementation-verification-correction loop deepens, the number of output tokens grows substantially and makes the decode stage the dominant source of latency [45, 91].
- **Long-context inference.** Many workloads now need tens of thousands or even millions of tokens, as in legal review or large codebase analysis. Prefill attention grows quadratically with sequence length, while the key value cache demands more bandwidth and memory. Both factors sharply increase TTFT [313].
- **Application-specific decoding.** Because applications differ in their priorities among accuracy, latency, and cost, fixed decoding strategies are insufficient to consistently ensure quality across domains, motivating quality of service (QoS)-guided decoding policies that adapt search and verification budgets to SLOs and cost [153].

Just Accepted

- **Increased concurrency and mixed workload** In real-world service environments, a single model instance typically handles multiple sessions simultaneously with mixed workload (conversation, summarization, math, code), and the autoregressive nature produces dissimilar resource profiles for prefill versus decode that interfere with one another under naive batching [132].
- **Collaboration across heterogeneous devices.** The execution environment for inference has expanded beyond a single cloud GPU to encompass multiple edge nodes and heterogeneous accelerators, requiring the co-optimization of batching, partitioning, and scheduling strategies under joint communication, computation, memory, and power constraints [62]. Recently, a disaggregated inference [132] approach has been introduced to improve efficiency by separating the prefill and decode phase based on device-specific computational and memory characteristics. For example, the prefill phase, which involves intensive large-scale matrix multiplications, is allocated to high-bandwidth GPUs, while the decode phase, which requires frequent token-wise cache access, is assigned to low-latency CPUs or devices with larger memory capacity. This configuration reduces overall latency and improves resource utilization.
- **Expansion to the MoE Model.** As parameter size in large-scale LLMs rise from tens to hundreds of billions, dense architectures that turn on every parameter for each token drastically increase inference floating-point operations per second (FLOPs), memory, and communication cost. Mixture of Experts (MoE) models ease this burden by activating only the top-k experts per token, keeping large capacity while trimming computation. This sparse approach shifts memory and communication overhead, increases arithmetic intensity, and reduces inference latency, cost, and energy use [60, 227].
- **Extension toward Multi-Agent Environments.** With the increasing sophistication of LLM applications, the traditional paradigm in which a single model handles all requests is rapidly evolving into a multi-agent environment [179], where multiple models (agents) collaborate to solve complex tasks. As multiple agents operate simultaneously, the overall memory requirements increase sharply, creating new challenges in efficiently sharing and coordinating input/output data and **KV** caches among agents.

Modern LLM inference engines should expose integrated, composable optimization techniques spanning algorithms, runtime, and batch management, guided by a latency/energy model and key performance indicator (KPI) targets including Perf/$, Perf/W, Joule/request, and SLO miss rate, allowing principled policy selection under mixed workloads and heterogeneous resources [153, 283].

3 Practical Guides to Inference Engines

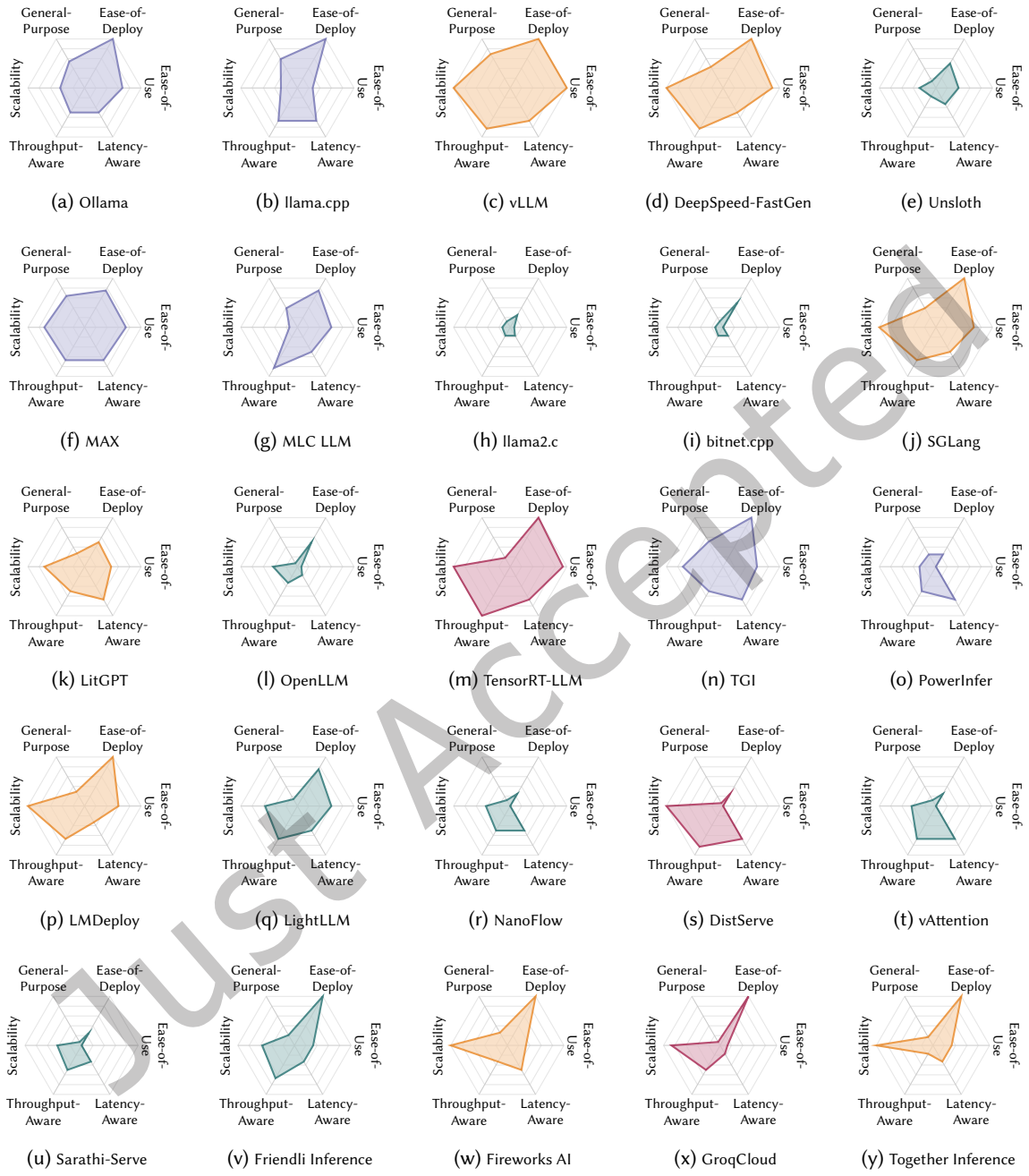
This section offers practical guidance on choosing an LLM inference engine by examining several key aspects. First, we look at ecosystem maturity and sustainability signals, such as how the engine is developed, licensed, and supported by its community. Next, we discuss hardware compatibility and platform support, focusing on whether the engine targets edge devices or server environments. We then explore the design and pricing strategies of commercial inference engines, including cost considerations and memory usage. Finally, we present a hardware-aware categorization of LLM inference engines, comparing engines based on their target use (edge or server), device types, and performance goals.

Table 4 provides a summary of the LLM inference engines examined in this paper, and Fig. 6 offers a visual representation of the characteristics of each engine. General-Purpose is a composite metric derived from the number of supported models in Table 4 and the range of hardware platforms in Table 5. A higher score indicates broader compatibility with diverse models and hardware.

Ease-of-Deploy measures how easily an engine can be installed via the Python package installer (pip), Debian Advanced Package Tool (APT), Homebrew [225], customized through source builds, Docker [69] or Conda [22] environments or prebuilt binaries. A higher rating suggests simpler, faster installation and deployment.

Ease-of-Use evaluates both documentation quality and user community activity level (as shown in Table 4).

Just Accepted



Just Accepted

Fig. 6. Representative characteristics comparison of LLM inference engines across six dimensions: model generality, ease of deployment and use, latency and throughput optimization, and scalability

Table 4. Comparison of LLM Inference Engines

Frameworks	Organization	Release Date	Open-Source Support†	GitHub (Sep. 2025)			Supported Models‡	Docs*	User Forum**		
				# Stars (Rate)	Star	Commit			S	F	M
Ollama [255]	Community (Ollama)	Jun. 2023	✔	153.0K (187.2)				✔	✔	✘	✔
llama.cpp [101]	Community (gml.ai)	Mar. 2023	✔	86.6K (101.2)				✔	✘	✘	✘
vLLM [166]	Academic (vLLM Team)	Feb. 2023	✔	58.3K (61.2)				✔	✔	✔	✔
DeepSpeed-FastGen [129]	Big Tech (Microsoft)	Nov. 2023	✔	40.1K (50.0)				✔	✘	✘	✔
Unsloth [325]	Startup (unsloth AI)	Nov. 2023	▲	45.6K (69.2)				✔	✔	✔	✘
MAX [237]	Startup (Modular Inc.)	Apr. 2023	▲	24.8K (28.4)				✔	✔	✔	✔
MLC LLM [234]	Community (MLC-AI)	Apr. 2023	✔	21.4K (24.5)				✔	✔	✘	✘
llama2.c [24]	Community (Andrej Karpathy)	Jul. 2023	✔	18.8K (23.8)				✘	✔	✘	✘
bitnet.cpp [336]	Big Tech (Microsoft)	Oct. 2024	✔	22.0K (53.9)				✘	✘	✘	✘
SGLang [399]	Academic (SGLang Team)	Jan. 2024	✔	18.0K (29.1)				✔	✔	✘	✔
LitGPT [194]	Startup (Lightning AI)	Jun. 2024	✔	12.8K (14.7)				✔	✔	✘	✔
OpenLLM [37]	Startup (BentoML)	Apr. 2023	▲	11.8K (13.3)				✘	✔	✘	✘
TensorRT-LLM [252]	Big Tech (NVIDIA)	Aug. 2023	▲	11.6K (15.2)				✔	✘	✔	✔
TGI [139]	Startup (Hugging Face)	Oct. 2022	✔	10.5K (9.8)				✔	✘	✔	✘
PowerInfer [302]	Academic (SJTU-IPADS)	Dec. 2023	✔	8.3K (13.0)				✘	✘	✘	✘
LMDeploy [213]	Startup (MMRazor/MMDeploy)	Jun. 2023	✔	7.1K (8.6)				✔	✔	✘	✘
LightLLM [191]	Academic (Lightllm Team)	Jul. 2023	✔	3.6K (4.6)				✔	✔	✘	✘
NanoFlow [405]	Academic (UW Efeslab)	Aug. 2024	✔	0.8K (2.3)				✘	✘	✘	✘
DistServe [402]	Academic (PKU)	Jan. 2024	✔	0.6K (1.1)				✘	✘	✘	✘
vAttention [268]	Big Tech (Microsoft)	May. 2024	✔	0.4K (0.8)				✘	✘	✘	✘
Sarathi-Serve [11]	Big Tech (Microsoft)	Nov. 2023	✔	0.4K (0.6)				✘	✘	✘	✘
Friendli Inference [87]	Startup (FriendliAI Inc.)	Nov. 2023	✘	–	–	–		✔	✘	✘	✔
Fireworks AI [83]	Startup (Fireworks AI, Inc.)	Jul. 2023	✘	–	–	–		✔	✔	✘	✘
GroqCloud [111]	Startup (Groq Inc.)	Feb. 2024	✘	–	–	–		✘	✔	✘	✔
Together Inference [318]	Startup (together.ai)	Nov. 2023	✘	–	–	–		✔	✔	✘	✘

† ▲indicates partial open-source support, ‡ Each square represents 50 models (Sep. 2025)
* Indicates the level of detail of the document (✔: Simple, ✔: Moderate, ✔: Detail),
**S refers for social networking services (Discord/Slack), F refers for discussion forums (private forums/reddit), and M refers for meetups

Latency-Aware and Throughput-Aware represent the each engine's support for latency- and throughput-specific optimization techniques, respectively, based on the metrics in Table 2 (§2) and the optimization features in Table 8 (§5). Higher values imply more robust capabilities to optimize in those areas.

Lastly, Scalability indicates how effectively an engine accommodates edge, server, and multi-node environments. Higher scores indicate suitability for large-scale LLM workloads.

For commercial inference engines, some metric scores may be lower because they rely on publicly available information.

By referring to Fig. 6, users can determine which LLM inference engine best matches their service needs and deployment settings.

3.1 Ecosystem Maturity and Sustainability Signals

This section discusses non-technical indicators related to the current status of LLM inference engines. As shown in Table 4, LLM inference engines can be categorized into open-source and closed-source commercial tools. For open-source tools, we analyze sustainability based on the types of development and maintenance organization, open software licenses, and the maturity of user support.

Development and Maintenance Organizations. Open-source inference engines are mainly developed and maintained by big tech companies, startups, communities or academic institutions. Among the **21** inference engines surveyed, the number of inference engines by organization type is: `Academic` (**6**), `Startup` (**6**), `Big Tech` (**5**), and `Community` (**4**). While the difference is not large, this shows that LLM inference engines are being developed

Just Accepted

Table 5. Hardware Features of LLM Inference Engines

Engines	Supported Platform				CPU		GPU			AI Accelerators					Mobile/ Edge	ETC.
	Linux	Windows	macOS	Web/ API	x86-64	ARM/ Apple Silicon (Vulkan, Metal)	NVIDIA (CUDA)	AMD (ROCm, HIP)	Intel (SYCL)	Google TPU	AMD Instinct	Intel Gaudi	Huawei Ascend	AWS Inferentia		
Ollama [255]	✔	✔	✔	✘	✔	✔	✔	✔	✔	✘	✔	✘	✘	✘	✔ (NVIDIA Jetson)	–
llama.cpp [101]	✔	✔	✔	✘	✔	✔	✔	✔	✔	✘	✔	✘	✔	✘	✔ (Qualcomm Adreno)	Moore Thread MTT
vLLM [166]	✔	✘	✘	✘	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔ (NVIDIA Jetson)	–
DeepSpeed-FastGen [129]	✔	✔	✘	✘	✔	✘	✔	✘	✔	✘	✔	✔	✔	✘	✘	Tecorigin SDAA
Unsloth [325]	✔	✔	✘	✘	✔	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–
MAX [237]	✔	✔	✔	✘	✔	✔	✔	✔	✘	✘	✘	✘	✘	✘	✘	–
MLC LLM [234]	✔	✔	✔	✘	✔	✔	✔	✔	✔	✘	✘	✘	✘	✘	✔ (Qualcomm Adreno, ARM Mali)	–
llama2.c [24]	✔	✔	✔	✘	✔	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘	–
bitnet.cpp [336]	✔	✔	✔	✘	✔	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘	–
SGLang [399]	✔	✘	✘	✘	✔	✘	✔	✘	✔	✘	✔	✔	✘	✘	✔ (NVIDIA Jetson)	–
LitGPT [194]	✔	✘	✔	✘	✔	✘	✔	✘	✘	✔	✔	✘	✘	✘	✘	–
OpenLLM [37]	✔	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–
TensorRT-LLM [252]	✔	✔	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✔ (NVIDIA Jetson)	–
TGI [139]	✔	✘	✘	✘	✔	✔	✔	✘	✔	✔	✔	✔	✘	✔	✘	–
PowerInfer [302, 362]	✔	✔	✔	✘	✔	✔	✔	✔	✘	✘	✘	✘	✘	✘	✔ (Qualcomm Snapdragon 8)	–
LMDeploy [213]	✔	✔	✘	✘	✔	✘	✔	✘	✘	✘	✘	✘	✔	✘	✔ (NVIDIA Jetson)	–
LightLLM [191]	✔	✘	✘	✘	✔	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–
NanoFlow [405]	✔	✘	✘	✘	✔	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–
DistServe [402]	✔	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–
vAttention [268]	✔	✘	✘	✘	✔	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–
Sarathi-Serve [11]	✔	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–
Friendli Inference [87]	✘	✘	✘	✔	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–
Fireworks AI [83]	✘	✘	✘	✔	✘	✘	✔	✘	✘	✘	✔	✘	✘	✘	✘	–
GroqCloud [111]	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘	✘	✘	Groq LPU
Together Inference [318]	✘	✘	✘	✔	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	–

and maintained by a variety of organizations. Regardless of the organization, most open-source projects use permissive licenses such as MIT or Apache 2.0, making them easy to adopt and use.

Projects maintained by community groups may face challenges in long-term maintenance, which could limit the integration of new technologies. Some projects like Unsloth [325], MAX [237], OpenLLM [37], and TensorRT-LLM [252], which are led by big tech or startups, only release parts of their source code.

While open-source engines are developed by diverse groups such as big tech, startups, communities, and academia, most commercial LLM inference engines are developed and run by startups. This is because startups can move quickly and develop specialized technologies to enter the market faster with differentiated, high-performance services.

User Preference. We measured user preference for open-source LLM inference engines using GitHub statistics such as total stars, daily average growth rate, and star growth trends over time. In this study, we considered a project to be highly popular if it gained more than 25 stars per day on average. Projects like Ollama [255] (**187.2**), llama.cpp [101] (**101.2**), Unsloth [325] (**69.2**), vLLM [166] (**61.2**), bitnet.cpp [336] (**53.9**) and DeepSpeed-FastGen [129] (**50.0**) meet this criterion and have tens of thousands of total stars, indicating high interest and rapid adoption by the community.

On the other hand, some projects have a large number of total stars but show slower recent growth. For example, TGI [139], TensorRT-LLM [252], and OpenLLM [37] each have more than 10K stars, but their daily growth is below 25, and their growth curves are flat after an initial spike. This may suggest that they received attention early, but are now facing difficulties in maintaining community interest. Possible reasons include limited usability or closed ecosystems.

This kind of analysis helps estimate the future growth potential of projects, providing a long-term perspective when choosing engines for practical or research use.

Just Accepted

Ease of Use. We evaluate the user-friendliness of LLM inference engines based on the quality of documentation and the availability of user forums. Our analysis shows that top projects like vLLM [166], and DeepSpeed-FastGen [129], TensorRT-LLM [252] provide well-written documentation, and vLLM [166] and MAX [237], LitGPT [194] have active community channels (e.g., Discord, forums), making onboarding and troubleshooting easier. This is closely related to their high star counts and rapid user adoption.

In contrast, projects such as bitnet.cpp [336], OpenLLM [37], PowerInfer [362], NanoFlow [405], DistServe [402], etc. have limited documentation or lack community channels. This is also reflected in their slow star growth, indicating a higher entry barrier for users. Projects with poor documentation and no forums tend to have lower popularity and slower growth.

These results suggest that beyond technical performance, user support systems are important factors in engine selection and community growth.

Development Activity. We evaluated the development activity of LLM inference engines based on GitHub commit trends and the number of supported models. By considering both indicators together, we achieved more reliable results than simply counting commits alone. Projects such as llama.cpp [101], vLLM [166], and DeepSpeed-FastGen [129] show consistent and frequent updates in their commit histories, while also supporting a wide range of LLM models. On the other hand, engines like TGI [139] and TensorRT-LLM [252], which gained many stars early on, show relatively stagnant commit activity and limited model support. This may indicate lower flexibility for future feature extensions. In particular, projects such as OpenLLM [37] and PowerInfer [362], which have a narrow range of supported models or only short-term commit activity, show signs of limited technical adaptability, which can be a constraint for real-world applications.

Overall, the number of GitHub stars and commit activity show similar patterns, suggesting that user interest and active development often go hand-in-hand. Inference engines that are frequently updated and support diverse models are more likely to be well-maintained over the long term.

3.2 Hardware Compatibility and Platform Support

Hardware and OS Support. As shown in Table 5, each inference engine is designed with different goals and target systems. Some inference engines support various hardware types, while others are optimized for a single platform. These hardware compatibility differences affect performance-related features such as quantization data formats, kernel fusion, and support for multi-node or multi-GPU configurations. Therefore, for optimal service performance, inference engines should be selected based on their compatibility with the intended hardware setup. In addition, Table 5 summarizes the OS and hardware support status for each inference engine.

Most inference engines operate in Linux environments, with some additionally supporting Windows or macOS. Commercial engines often provide web-based inference services, but they also enable on-premise deployments. These platform differences can affect both development complexity and inference performance, depending on the range of software capabilities.

CPU-Based Inference. While many engines include CPU-based inference, non-edge-focused solutions typically employ the CPU for specific tasks—such as offloading operations or handling model weights—rather than as the primary compute resource.

Edge and Server Environments. On edge devices (e.g., mobiles and Internet of Things (IoTs) systems), limited compute and memory resources require inference engines to focus on lightweight design. These engines reduce model size and apply techniques like quantization to minimize memory usage and enable execution on low-power hardware. Mobile and edge-oriented engines may need to run entirely on CPUs or leverage AI accelerators embedded in system-on-chip (SoC) platforms, such as Neural Processing Units (NPUs) or Digital Signal Processors (DSPs). For example, Apple Core ML [27] and Google AI Edge SDK [108] allow deployment of transformer operations to dedicated hardware on consumer devices. Edge inference engines include Ollama [255],

Just Accepted

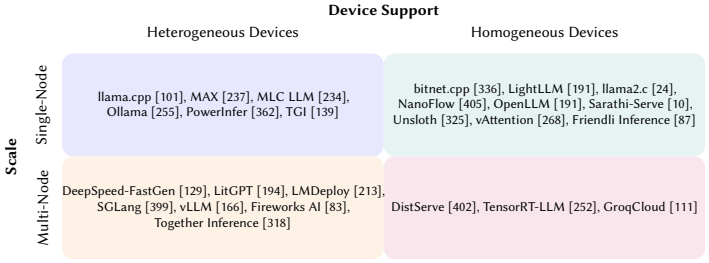


Fig. 7. A taxonomy of LLM inference engines categorized by scalability and hardware support

llama.cpp [101], and MLC LLM [234], and in particular, MLC LLM provides compiler technology for various edge hardware.

Conversely, server-side inference engines are optimized for multi-GPU environments to handle high volumes of requests. They rely on distributed computing techniques such as model and pipeline parallelism to spread large models across devices, and they use large batch sizes and dynamic scheduling to maximize hardware utilization.As AI accelerators such as Intel Max [144], Google TPU [151], AMD Instinct [300], and Intel Gaudi [155] are adopted as replacements for NVIDIA GPUs in inference servers, more and more engines are offering heterogeneous hardware backends. Server inference engines include TensorRT-LLM [252], vLLM [166], DeepSpeed-FastGen [129], etc., and provide optimization techniques for throughput or latency.

Scalability and Device Types. Fig. 7 groups the inference engines from Table 4 according to their hardware characteristics. The X-axis distinguishes between support for a single device type versus multiple types, while the Y-axis shows whether each engine supports single-node or multi-node configurations. A single node generally includes one to eight GPUs, whereas multi-node systems connect multiple such nodes.

Single-node inference engines emphasize intra-node optimization for CPUs, consumer-level GPUs, or edge/IoT devices. Ollama [255] and llama.cpp [101] focus on consumer-level hardware (e.g., laptops and PCs), and MLC LLM [234] targets efficient inference on various edge platforms. By contrast, multi-node inference engines handle both inter-node and intra-node computations, optimizing scalability and performance for multi-user workloads. Representative inference engines belonging to this category include vLLM [166], TensorRT-LLM [252], and SGLang [399].

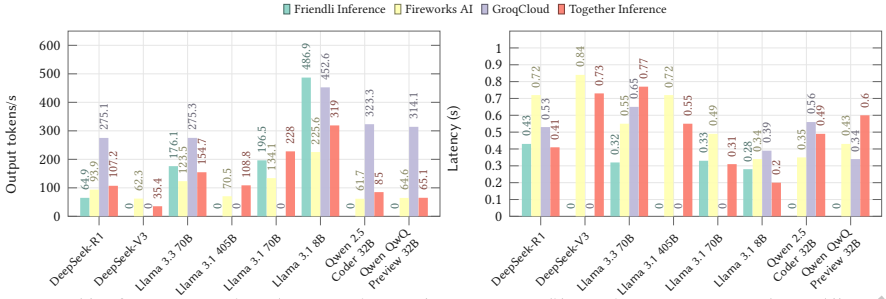
Inference engines supporting heterogeneous devices can operate with multiple hardware types beyond GPUs, allowing developers to choose hardware based on application requirements. In contrast, engines that support only homogeneous devices—such as those specialized for NVIDIA GPUs or Groq LPU [4]—can deliver high performance through custom kernels and low-level optimizations, though their narrower hardware support may limit portability.

3.3 Design and Pricing Strategies of Commercial Inference Engines

Cloud Services and Model Coverage. Commercial inference engines offer cloud-based services that simplify the setup of LLM applications and underlying hardware, compared to many open-source solutions. In particular, Friendli Inference [87], Fireworks AI [83], and Together Inference [318] support a broader model range than most open-source inference engines, covering not only LLMs but also image, audio, and multimodal models, and they facilitate rapid adoption of newly released models.

A key advantage of commercial inference engines is that they can provide various model and hardware support customized to the scale of the service, reducing the cost and complexity of server deployment and maintenance. Unlike some open-source engines—whose maintenance may be inconsistent or whose licensing might shift to paid

Just Accepted



(a) Inference Throughput (Output Tokens/sec) (b) Initial Response Latency (TTFT (s))

Fig. 8. Comparison of Inference Performance across commercial LLM inference engines

Models	Friendli AI†		Fireworks AI		GroqCloud		Together AI‡	
	Input	Output	Input	Output	Input	Output	Input	Output
DeepSeek-R1	3.00	7.00	3.00	8.00	0.75*	0.99*	3.00	7.00
DeepSeek-V3	–	–	0.90	0.90	–	–	1.25	1.25
Llama 3.3 70B	0.60	0.60	–	–	0.59	0.79	0.88	0.88
Llama 3.1 405B	–	–	3.00	3.00	–	–	3.50	3.50
Llama 3.1 70B	0.60	0.60	–	–	–	–	0.88	0.88
Llama 3.1 8B	0.10	0.10	–	–	0.05	0.08	0.18	0.18
Llama 4 Maveric	–	–	0.22	0.88	0.20	0.60	0.27	0.85
Qwen 2.5 Coder 32B	–	–	–	–	0.79	0.79	0.80	0.80
Qwen QwQ Preview 32B	–	–	–	–	0.29	0.39	1.20	1.20
OpenAI gpt OSS 120b	–	–	0.15	0.60	0.15	0.75	0.15	0.60

†Llama is Instruct model, ‡Turbo mode price
*DeepSeek-R1 Distill Llama 70B

Table 6. Pricing by Model in Commercial LLM Engines ($/1M tokens)

Hardwares	Friendli AI	Fireworks AI	GroqCloud†	Together AI
NVIDIA A100 80GB	2.9	2.9	–	1.30
NVIDIA H100 80GB	3.9	5.8	–	2.29
NVIDIA H200 141GB	4.5	6.99	–	3.79
NVIDIA B200 180GB	8.9	11.99	–	5.50
AMD MI300X	–	4.99	–	–
Groq LPU	–	–	–	–

†Charging prices based on tokens and requests per model rather than per device

Table 7. Pricing by Hardware Type in Commercial LLM Engines ($/hour for 1 device)

Just Accepted

models if resources become constrained—commercial services generally guarantee updates and enhancements over a specified duration, ensuring reliable long-term operation.

Hardware Variety and Specialization. Among these services, Friendli Inference [87] and Together Inference [318] focus on optimizing inference for NVIDIA GPUs, whereas GroqCloud [111] leverages the proprietary Groq LPU AI accelerator [4]. Fireworks AI supports a broader range of hardware, including AMD Instinct MI300X [300], and meets privacy and reliability standards through relevant certifications.

Performance and Cost Trade-offs. When selecting a commercial engine, it is also necessary to consider hardware support and cost. Commercial inference engines typically aim for low latency and high throughput by implementing batch optimization, request pipelining, and other techniques that offer faster and more streamlined deployment compared to open-source alternatives. Fig. 8 and Table 6 show the inference performance and costs for various models (e.g., reasoning (DeepSeek-R1 [120]), MoE (DeepSeek-V1 [38]), large-scale (Llama 3 [109]), code generation (Qwen 2.5 Coder [141]), multimodal (Qwen QWQ [315])) using different commercial engines [28]. Additionally, Table 7 summarizes the hardware costs provided by each commercial engine. Even when using the same hardware, the cost may vary depending on the degree of kernel and compute optimization in each engine.

4 Detailed Review of Inference Engines

This section provides a detailed literature review of the 25 inference engines listed in Table 4. For each engine, we describe its architecture, key features, and distinctive traits. We also explain, engine by engine, the representative characteristics shown in Fig. 6's six-axis radar plots.

4.1 Ollama

Ollama [255] is a Go programming language [105]-based inference engine designed to run LLMs in local environments, enabling users without technical background to easily test and deploy models. Consequently, it primarily targets single-GPU setups rather than multi-GPU systems, relying on llama.cpp as its core backend.

Ollama is composed of two primary components: a `client` and a `server`. The `client` sends requests to the `server` via a command-line interface (CLI), while the `server` includes an HTTP server and a llama.cpp [101] backend. The HTTP server manages `client-server` communication, and the llama.cpp backend loads the model and processes inference requests.

The inference engine supports a variety of models—such as Llama [109], Falcon [16], Mistral [148], and DeepSeek-R1 [120]—and is important to quickly adapt to newly released models. It uses both GGUF [99] and Safetensors [137] formats for model inference and provides model customization via a *Modelfile*. In addition, Ollama offers a REST API that allows users to manage and execute models through HTTP requests, making it suitable for chat, text generation, and other applications. Integration options include Open WebUI [256], SwiftChat [19], Google Cloud, and oterm [378], extending its deployment capabilities in mobile, cloud, and local environments.

However, Ollama prioritizes user accessibility over advanced inference optimizations, meaning it lacks features such as memory optimization, multi-GPU functionality, and multi-node support. In return, it delivers broad compatibility by supporting not only NVIDIA GPUs but also AMD GPUs and ARM platforms.

Representative Characteristics Summary
- **General-Purpose [Medium]**: Supports popular community models and both NVIDIA and AMD GPUs, but lacks multi-GPU or edge specialization.
- **Ease-of-Deploy [High]**: One-line installation via Homebrew, pip, or Docker makes setup extremely simple.
- **Ease-of-Use [Medium]**: A concise CLI and REST API, plus GUI integrations such as Open WebUI, lower the entry barrier for non-experts.
- **Latency-Aware [Medium]**: The engine provides no Flash- or KV-cache optimizations, so single-token latency remains higher.
- **Throughput-Aware [Medium]**: Single-GPU operation without batching strategies limits sustained throughput.
- **Scalability [Medium]**: Designed for local single-GPU use and cannot extend to multi-node deployments.

4.2 llama.cpp

llama.cpp [101] is a C++ library for LLM inference that runs models on CPUs without a GPU. Consequently, it depends on minimal external software and operates efficiently on diverse hardware architectures. It supports quantization for multiple data types (e.g., 1.5-bit, 4-bit, 8-bit), reducing memory usage, and boosting efficiency.

llama.cpp also introduces the Georgi Gerganov Unified Format (GGUF) [99] for streamlined LLM storage and deployment. GGUF consolidates model parameters, structure, and metadata into a single file, improving the Georgi Gerganov Machine Learning (GGML) [98] format by providing better flexibility and compatibility. This approach standardizes model storage and simplifies deployment.

llama.cpp supports a range of hardware platforms—including x86, ARM, and NVIDIA GPUs—and uses the GGML context to configure these backends. It provides hardware-specific kernel and graph optimizations that facilitate efficient inference. Additionally, llama.cpp extends usability with subprojects such as `llama-cli` for command-line execution, llama-server for OpenAI API compatible HTTP-serving, and lightweight runners like `llama-run` and `llama-simple`.

Representative Characteristics Summary
- **General-Purpose [Medium]**: Runs on x86, ARM CPUs and NVIDIA GPUs with several quantization formats for broad hardware reach.

Just Accepted

- **Ease-of-Deploy [High]**: A single static binary or minimal CMake build keeps external dependencies near zero.
- **Ease-of-Use [Low]**: CLI helpers and an OpenAI-style server exist, but documentation is concise and community-driven.
- **Latency-Aware [Medium]**: Optional FlashAttention kernels and GPU offload reduce token delay on capable devices.
- **Throughput-Aware [Medium]**: Multithreading and continuous batching boost CPU throughput, although distributed support is minimal.
- **Scalability [Low]**: Optimized for single-node execution and lacks native cluster features.

4.3 vLLM

vLLM [166] is a high-performance LLMs serving library, focusing on fast token generation and low latency. Its `PagedAttention` mechanism enhances memory efficiency by storing **KV** cache in non-contiguous memory blocks, preventing the fragmentation issues associated with contiguous storage.

vLLM is built around `AsyncLLM` for asynchronous request handling, an OpenAI-compatible API server, and an `EngineCore` that conducts inference. A ZeroMQ [316]-based multiprocessing API server overlaps operations between `AsyncLLM` and the API layer. `EngineCore` features modules for scheduling and model execution, enabling concurrent handling of CPU-heavy tasks (e.g., tokenization, multimodal input management, and token detokenization) alongside the main execution loop for improved throughput. Its symmetric architecture reduces inter-process overhead and supports optimized tensor parallelism.

Additionally, vLLM supports FlashAttention-3 [285] to further reduce inference latency. It employs a distributed system architecture for multi-GPU workload distribution, leveraging Megatron-LM's tensor parallelism [297]. Beyond CPU and GPU support, vLLM is compatible with AWS Inferentia [17] and Google TPU [151], extending its capabilities to multimodal inference.

vLLM provides a batch decoding technique, called cascade inference [376], which utilizes shared prefixes to efficiently manage memory bandwidth. During LLM inference, when multiple requests contain identical prefixes, recomputing the prefix segment for each request incurs substantial memory and time overhead. This issue becomes more severe as the prefix length increases and the number of concurrent requests grows.

Cascade inference separates the common prefix from each request's individual suffix and stores the **KV** cache for the prefix in the GPU's shared memory, allowing multiple requests to reference it simultaneously. As a result, redundant prefix computations are eliminated, significantly reducing both latency and memory consumption. In vLLM, cascade inference can be toggled on or off through a dedicated flag, but is typically designed to activate automatically based on detected input patterns.

Representative Characteristics Summary

- **General-Purpose [High]**: Serves a wide range of LLMs across GPUs, TPUs, and AWS Inferentia accelerators.
- **Ease-of-Deploy [High]**: Docker images and a pip package simplify setup, but distributed configuration still requires manual steps.
- **Ease-of-Use [High]**: OpenAI-compatible endpoints and an active community streamline application integration.
- **Latency-Aware [Medium]**: FlashAttention-3 and PagedAttention aggressively cut attention-time latency.
- **Throughput-Aware [High]**: AsyncLLM scheduling and ZeroMQ multiprocessing maintain high token-per-second rates.
- **Scalability [High]**: Built-in tensor parallelism enables multi-GPU and multi-node clustering.

4.4 DeepSpeed-FastGen

DeepSpeed-FastGen [129] is an LLM inference engine integrating Microsoft DeepSpeed Inference [21] and DeepSpeed Model Implementations for Inference (MII) [233]. It optimizes memory usage to enable efficient model inference.

DeepSpeed-FastGen deploys DeepSpeed MII for its frontend and backend, handling requests through features like dedicated query/response APIs, continuous batching, and a model pipeline. Internally, it leverages DeepSpeed

Just Accepted

Inference to support hardware-optimized kernels (e.g., NVIDIA CUDA), as well as Blocked **KV**-Cache and tensor parallelism.

A major feature is the `Dynamic SplitFuse` technique, which splits long prompts into smaller segments and processes them in multiple forward passes, improving throughput and reducing latency. By maintaining a consistent forward pass size, system processing efficiency increases. DeepSpeed-FastGen also offers replica-level load balancing, distributing inference workloads across multiple nodes. Compared to single-node inference, multi-node deployments can deliver significant speedups in query processing.

Representative Characteristics Summary
- **General-Purpose [Medium]**: DeepSpeed-MII front end supports numerous HuggingFace checkpoints and custom models.
- **Ease-of-Deploy [High]**: A containerized launcher is available, though model conversion and registry are still required.
- **Ease-of-Use [High]**: MII-style APIs are clear, but some DeepSpeed configuration know-how is assumed.
- **Latency-Aware [Medium]**: Dynamic SplitFuse splits long prompts to cap worst-case latency.
- **Throughput-Aware [High]**: Continuous batching, blocked KV-cache, and tensor parallelism keep GPUs saturated.
- **Scalability [High]**: Replica-level load balancing supports efficient multi-node service.

4.5 Unsloth

Unsloth [325] is a engine focused on efficient fine-tuning and inference for LLMs. It achieves rapid fine-tuning and reduced memory usage through techniques such as Low-rank adaptation (LoRA) [133] and Quantized-LoRA (QLoRA) [67] while preserving model accuracy. All kernels are implemented in OpenAI Triton [317], further enhancing the execution speed of LLM. Although Unsloth integrates modules such as xFormers [228] to accelerate transformer operations. This approach allows for flexible customization of attention blocks and other modules, providing greater adaptability for diverse use cases.

For compatibility, Unsloth supports both GGUF [99] and vLLM [166] formats and offers a straightforward API for creating inference services. However, it currently runs only on NVIDIA GPUs, and advanced optimization features such as multi-GPU and multi-node support are exclusive to the paid version. The open-source release is restricted to single-GPU setups and supports a limited number of models.

Representative Characteristics Summary
- **General-Purpose [Low]**: Provides GGUF and vLLM model formats but currently restricts execution to NVIDIA GPUs.
- **Ease-of-Deploy [Medium]**: A single pip install delivers both fine-tuning and inference capabilities.
- **Ease-of-Use [Medium]**: High-level Python APIs are simple, though advanced documentation is still limited.
- **Latency-Aware [Medium]**: Triton-fused kernels shorten attention steps, trimming token latency moderately.
- **Throughput-Aware [Low]**: xFormers integration helps single-GPU throughput; distributed execution is paywalled.
- **Scalability [Low]**: The open-source edition runs on a single GPU and omits multi-node features.

4.6 MAX

Modular Accelerated Xecution (MAX) [237] is an integrated platform aimed at simplifying the creation and deployment of high-performance AI endpoints, while maintaining flexibility across diverse hardware setups. It offers a graph compiler and runtime capable of accelerating generative AI models through hardware-agnostic libraries. By compiling models into optimized computation graphs, MAX enhances execution efficiency and reduces latency for better performance.

MAX is built on the Mojo programming language [236]. Mojo extends Python with system programming features from C, C++, and CUDA via Multi-Level Intermediate Representation (MLIR) [169], enabling high performance on CPUs, GPUs, and specialized AI accelerators.

Just Accepted

MAX comprises two main components: the `MAX Engine` (an inference library and runtime) and `MAX Serve`, a serving utility for model deployment. `MAX Serve` hosts LLMs and provides OpenAI API compatible REST endpoints in both local and cloud environments. It applies continuous heterogeneous batching and multi-step scheduling to maximize GPU utilization and ensure stable performance, particularly for large-scale workloads. Internally, `MAX Serve` integrates the `MAX Engine`, which utilizes its graph compiler and runtime to accelerate models on CPUs and GPUs.

Currently, MAX supports inference workloads across both local and cloud environments and operates on CPUs and NVIDIA GPUs.

Representative Characteristics Summary
- **General-Purpose [Medium]**: Mojo's MLIR compiler targets CPUs, GPUs, and future accelerators from one model graph.
- **Ease-of-Deploy [High]**: Docker images and a CLI exist, but users still package models into MAX Serve.
- **Ease-of-Use [High]**: REST endpoints are easy to consume, yet Mojo tooling is early-stage for newcomers.
- **Latency-Aware [Medium]**: Ahead-of-time graph compilation fuses kernels and shortens critical paths.
- **Throughput-Aware [Medium]**: Continuous heterogeneous batching and multi-step scheduling keep devices busy.
- **Scalability [High]**: Operates on local and cloud machines with experimental multi-GPU support.

4.7 MLC LLM

MLC LLM [234] is a compiler and high-performance deployment engine for LLMs, designed to enable model development, optimization, and deployment across multiple platforms. It supports inference not only on NVIDIA and AMD GPUs but also on mobile and edge devices such as iOS and Android, unifying server and edge environments into a single LLM engine. The provided engine, `MLCEngine`, delivers high throughput and low latency in server environments and also supports lightweight local deployment.

Achieving platform-wide LLM acceleration requires extensive GPU programming and runtime compatibility. To address this, MLC LLM builds on Apache TVM [51], generating GPU libraries automatically for each hardware and platform. It integrates LLM-specific optimizations such as continuous batching [380] and speculative decoding [173, 352], and employs FlashInfer [375] to accelerate NVIDIA GPUs. MLC LLM either converts and quantizes foundation model weights or loads pre-converted weights, using the `model-weights-mlc` module for operator fusion, memory allocation, and hardware-specific optimizations; the `model-lib` component then constructs platform-native runtimes for each device. MLC LLM offers a range of deployment modes—Python APIs, OpenAI-compatible APIs, REST servers, and WebLLM [280]—ensuring broad portability across cloud and local platforms.

Representative Characteristics Summary
- **General-Purpose [Medium]**: A single engine serves desktop, mobile, and WebLLM runtimes across NVIDIA and AMD GPUs.
- **Ease-of-Deploy [High]**: The installer script compiles TVM kernels for each target automatically.
- **Ease-of-Use [Medium]**: Python and REST APIs plus a web demo provide moderate integration effort.
- **Latency-Aware [Medium]**: FlashInfer kernels and continuous batching enable low-latency generation.
- **Throughput-Aware [High]**: Speculative decoding and operator fusion lift tokens-per-second on GPUs.
- **Scalability [Medium]**: Generates native runtimes for edge devices through to cloud servers.

4.8 llama2.c

llama2.c [24] is an inference engine designed to run small Llama2 [321]-based models in a single C file. It comprises approximately 700 lines of C code and can load models trained with PyTorch [263] for inference.

The inference engine focuses on small-scale domains and is intended for educational use and features a simple structure. Rather than implementing advanced optimization techniques, it only includes the essential code needed

Just Accepted

for LLM inference. Parallel processing is limited to OpenMP-based multithreading and runs exclusively on CPUs, without support for GPU execution or distributed environments.

Representative Characteristics Summary
- **General-Purpose [Low]**: Runs only small Llama-2 checkpoints on CPUs for education and demonstration.
- **Ease-of-Deploy [Low]**: Compiles in seconds with no external libraries for rapid experimentation.
- **Ease-of-Use [Low]**: Approximately 700 lines of readable C code make learning and modification easy.
- **Latency-Aware [Low]**: Only basic OpenMP threading is present, leaving high per-token latency.
- **Throughput-Aware [Low]**: No batching, GPU support, or cache management reduces sustained throughput.
- **Scalability [Low]**: Designed for a single CPU host with no distributed or GPU pathway.

4.9 bitnet.cpp

bitnet.cpp [336] is a CPU-only inference engine developed in the context of one-bit LLM research. Built based on llama.cpp [101], it focuses on fast, lossless inference of ternary models (BitNet b1.58 [218]) while minimizing power consumption. The project offers three kernel types—`I2_S`, `TL1`, and `TL2`—optimized for both x86 and ARM processors.

The `I2_S` kernel converts full-precision weights to a two-bit format offline, then restores the original values during inference to accelerate general matrix-vector multiply (GEMV) operations. This approach reduces memory and bandwidth and also improves performance in multithreading systems. The `TL1` kernel compresses every two weights into a four-bit index and employs a lookup table (LUT) with nine precomputed activation values based on the T-MAC [344] method, allowing large models to run efficiently even with limited thread environments. `TL2` compresses every three weights into a five-bit index, shrinking the model size to one-sixth of the `TL1` footprint and making it suitable for environments with tight memory or bandwidth constraints.

bitnet.cpp supports only local CPU execution and relies on multithreading rather than distributed parallelism for acceleration. In addition to BitNet b1.58 [218], it can run the Llama 3 8B [109] and Falcon 3 [79] family models, but it does not yet support broader hardware platforms or large-scale distributed deployments.

Representative Characteristics Summary
- **General-Purpose [Low]**: Runs only on local CPUs and supports a narrow model set (BitNet b1.58 plus a few Llama 3 and Falcon variants), so overall hardware and model diversity is limited.
- **Ease-of-Deploy [Medium]**: Ships as a self-contained C++ binary that builds with minimal dependencies and requires no GPU drivers, enabling rapid installation on almost any x86 or ARM host.
- **Ease-of-Use [Low]**: While the CLI closely mirrors llama.cpp, documentation and community examples are still sparse, which raises the learning curve for first-time users.
- **Latency-Aware [Low]**: The engine focuses on memory-bandwidth reduction rather than dedicated latency techniques; single-token delay remains governed by CPU core speed.
- **Throughput-Aware [Low]**: Multithreaded I2_S, TL1, and TL2 kernels use 2- to 5-bit weight compression to boost GEMV throughput compared with full-precision CPU baselines.
- **Scalability [Low]**: All acceleration is confined to one multicore server; there is no support for multi-socket or distributed execution across nodes.

4.10 SGLang

Structured Generation Language for LLMs (SGLang) [399] is a system designed to execute LLMs efficiently by overcoming limitations found in existing inference engines, including multimodal input handling, parallel processing, and **KV** cache reuse. To achieve this, SGLang uses multi-call structures and introduces Language Model Programs (`LM Programs`), which support various model types (vision, embedding, reward models) as well as multi-node operation.

Just Accepted

The inference engine comprises a frontend and a backend (runtime) and provides an OpenAI-compatible API. SGLang's frontend, written in Python, enables flexible authoring of `LM Programs` using conventional control flow and libraries, enhancing developer ease. Meanwhile, the backend applies execution optimizations that include RadixAttention-based **KV** cache management and structured decoding with compressed finite state machines, enabling rapid inference. These methods allow SGLang to outperform existing inference engines in throughput and excel in tasks such as agent control and logical reasoning.

SGLang provides both an interpreter and a compiler. The interpreter manages prompt states as streams and asynchronously handles fundamental operations to improve synchronization and parallelism. It also tracks program execution paths, enabling further compiler optimizations. After compiling these programs into computation graphs, the SGLang graph executor rewrites the graph or establishes static execution plans.

For further optimization, SGLang employs a `Zero-Overhead Batch Scheduler`, similar to NanoFlow's Nano-batching strategy [405], to increase parallelism in model inference. It also features a cache-aware load balancer that improves prefix cache hit rates, thus boosting overall throughput.

Representative Characteristics Summary
- **General-Purpose [Medium]**: Language-Model Programs manage multimodal models and support multi-node execution.
- **Ease-of-Deploy [High]**: Requires source compilation and CUDA toolchain configuration before use.
- **Ease-of-Use [Medium]**: Python DSL is flexible but introduces a learning curve with stream-based semantics.
- **Latency-Aware [Medium]**: RadixAttention and compressed finite-state decoding reduce tail latency.
- **Throughput-Aware [Medium]**: The Zero-Overhead Batch Scheduler maximizes overlap, achieving extreme throughput.
- **Scalability [High]**: Cache-aware load balancing enables cluster execution, though tooling is still maturing.

4.11 LitGPT

LitGPT [194] is an end-to-end framework that covers fine-tuning, inference, testing and deployment. Built on nanoGPT [23], Lit-LLaMA [193], and Lightning Fabric [192], it supports pretrained models for rapid prototyping.

LitGPT scales from a single GPU to multi-GPU and multi-node environments, offering distributed parallelism through Fully Sharded Data Parallelism (FSDP) [396] and faster computation with FlashAttention-2 [63]. This framework also includes memory and speed optimizations via quantization [76] and LoRA [133], and it can run LLMs on Google TPUs through the PyTorch/XLA compiler [271].

Representative Characteristics Summary
- **General-Purpose [Low]**: Supports NVIDIA GPUs, AMD Instinct, and Google TPU, but is primarily optimized for NVIDIA GPUs.
- **Ease-of-Deploy [Medium]**: Offers easy installation via pip and provides prebuilt packages.
- **Ease-of-Use [Medium]**: Provides brief manuals and maintains community through forums and meet-ups.
- **Latency-Aware [Medium]**: Reduces response time with FlashAttention-2, speculative decoding, and KV caching.
- **Throughput-Aware [Medium]**: Increases overall throughput with FSDP and batching optimizations.
- **Scalability [High]**: Extends from a single-GPU setup to multi-GPU and multi-node deployments.

4.12 OpenLLM

OpenLLM [37] is a platform for the straightforward execution and deployment of open-source LLMs and custom models through simple commands. Designed as a cloud-based solution that overcomes the scalability and high-load issues of existing platforms like Ollama [255], OpenLLM targets multi-user support, high throughput, and low latency. This makes it well suited for deploying LLMs on cloud or on-premise servers and for building LLM-based applications. A key advantage is data security, achieved via a Bring Your Own Cloud (BYOC) model.

OpenLLM provides an OpenAI-compatible API server that simplifies LLM execution and employs vLLM [166] and BentoML [36] as backends to maintain high throughput in large-scale environments. It uses `Bento`, a custom

Just Accepted

file format developed by BentoML, which packages source code, models, data files, and dependencies into a single entity. These `Bento` objects can be transformed into container images for convenient deployment.

Representative Characteristics Summary
- **General-Purpose [Low]**: Combines vLLM and BentoML back ends to run varied open-source models in the cloud.
- **Ease-of-Deploy [Medium]**: One command converts a model into a Bento image deployable in any BYOC environment.
- **Ease-of-Use [Low]**: CLI, web UI, and OpenAI-style endpoints cut application integration time sharply.
- **Latency-Aware [Low]**: FlashAttention from vLLM lowers core latency; additional cloud overhead may remain.
- **Throughput-Aware [Medium]**: Bento containers batch requests continuously and scale horizontally.
- **Scalability [Medium]**: Multi-tenant support is built-in, while multi-node GPU pods require custom orchestration.

4.13 TensorRT-LLM

TensorRT-LLM [252] is a inference engine to optimize inference on NVIDIA GPUs and is part of NVIDIA's NeMO [163] end-to-end generative AI development ecosystem. It includes compilation and optimization libraries to boost model inference performance. During compilation, the TensorRT [250] compiler analyzes the computation graph to select optimal kernels, fusing them to minimize memory overhead. This allows maximal exploitation of CUDA kernels and Tensor Cores, and supports various low-precision operations for faster inference.

Models for inference can be trained using NVIDIA NeMo or PyTorch [263], or sourced from pretrained weights on platforms like Hugging Face, and must be converted to a TensorRT-compatible format using the Model Definition API. Although TensorRT-LLM primarily uses TensorRT as its backend, it also includes Python and C++ backends for NVIDIA Triton Inference Server [248], providing an end-to-end solution for online LLM deployment. A PyTorch backend is available experimentally. With support from NVIDIA Collective Communication Library (NCCL) [247], TensorRT-LLM offers distributed inference via tensor parallelism and pipeline parallelism in multi-GPU environments. For optimized serving, in-flight batching groups incoming requests dynamically.

To overcome performance constraints of ring-based All-Reduce topologies in multi-node environments, TensorRT-LLM introduces a multishot approach that harnesses NVSwitch's multicast capabilities, reducing latency by up to 3×. However, TensorRT-LLM is limited to NVIDIA GPUs, restricting hardware scalability.

Representative Characteristics Summary
- **General-Purpose [Low]**: Targets NVIDIA GPUs exclusively, limiting hardware diversity.
- **Ease-of-Deploy [High]**: Model conversion and Triton back-end registration add setup steps despite helper scripts.
- **Ease-of-Use [High]**: Sample Python and C++ code exist, but NeMo and Triton familiarity helps.
- **Latency-Aware [Medium]**: Kernel fusion on Tensor Cores delivers very low single-token latency.
- **Throughput-Aware [High]**: In-flight batching and pipeline parallelism maintain high throughput on large models.
- **Scalability [High]**: NVSwitch multicast and NCCL enable efficient multi-GPU and multi-node deployment.

4.14 Hugging Face TGI

Hugging Face Text Generation Inference (TGI) [139] is a toolkit for deploying and serving LLMs, supporting diverse inference workloads and integrating with backends like vLLM [166] and TensorRT-LLM [252]. It accommodates various hardware platforms, including NVIDIA GPUs, AWS Inferentia [17], and Intel Gaudi [155] letting users choose suitable backends for their hardware. Built in Rust, TGI's backend supports streaming and concurrency, efficiently handling high LLM traffic.

TGI comprises three key components: a `router`, `launcher` and `model server`. The `router` is an HTTP server that manages client requests (supporting Hugging Face's custom APIs and the OpenAI Message API), batching incoming requests with a queue, scheduler and a memory block allocator. The `launcher` spins up one or more

Just Accepted

`model server` and shards models based on parameters from the `router`. The `model server`—implemented in Python—receives Google Remote Procedure Call (gRPC) [112]-based requests for model loading and inference.

To optimize inference, TGI employs quantization, RoPE scaling [205], Safetensors [137], and `Zero Config` for automatic configuration depending on hardware and model. It also leverages Flashinfer [375] and Flashdecoding [130] to deliver fast performance on long prompts. For observability, it connects with tools like Prometheus [324] and Grafana [47]. When running models on multiple devices, TGI synchronizes using NVIDIA NCCL [247] Although it supports tensor parallelism for multi-device inference, only certain LLM models are currently compatible.

Representative Characteristics Summary
- **General-Purpose [Medium]**: Swappable vLLM or TensorRT-LLM back ends cover NVIDIA, Inferentia, and Gaudi hardware.
- **Ease-of-Deploy [High]**: A single launcher auto-configures hardware and downloads model weights.
- **Ease-of-Use [Medium]**: Supports custom HF APIs and OpenAI messages with built-in monitoring hooks.
- **Latency-Aware [Medium]**: FlashInfer and Flashdecoding accelerate long-sequence generation.
- **Throughput-Aware [Medium]**: Router and scheduler batch inputs continuously for high request volume.
- **Scalability [High]**: Model sharding and NCCL permit multi-GPU serving across nodes.

4.15 PowerInfer

PowerInfer [302] is an LLM inference system built by extending llama.cpp [101], designed to run LLMs on a single consumer-grade GPU. Running LLMs without model compression techniques often leads to accuracy loss and memory limitations. CPU-GPU offloading methods suffer from high PCIe latency, which slows down the inference. Additionally, speculative decoding becomes inefficient with small batch sizes and can degrade model performance.

To address these limitations, PowerInfer leverages the observation that neuron activations in LLMs follow a power-law distribution. It separates frequently activated neurons (`hot neurons`) from less active ones (`cold neurons`). `Hot neurons` are loaded onto the GPU for fast computation, while `cold neurons` are handled on the CPU. This design reduces GPU memory usage and minimizes CPU-GPU data transfer. PowerInfer uses an offline profiling step to identify hot and `cold neurons` based on their activation frequency, and an online predictor to determine which neurons are active for each input.

PowerInfer uses a hybrid approach for inference, comprising offline and online components. In the offline phase, it analyzes neuron activation patterns (`Insight-1`) and classifies neurons into `hot` and `cold` categories using the activation data. It then performs neuron assignment optimization through Integer Linear Programming (ILP) to maximize memory utilization. In the online component, neurons are assigned to the GPU or CPU based on predefined policies, and distributed computations are performed via GPU and CPU executors.

PowerInfer also introduces neuron-aware sparse operators to overcome the limitations of existing sparse computation libraries. These operators can directly handle irregular tensors at the neuron level without format conversion, and are optimized for both GPU and CPU execution.

As a result, PowerInfer enables efficient LLM inference without fully loading the model into GPU memory, making it a practical solution for memory-constrained local environments.

Recently, PowerInfer-2 [362] has been proposed to further extend this approach to mobile devices such as smartphones. PowerInfer-2 extends PowerInfer's capabilities to scenarios involving memory-constrained mobile devices. Relying on the same `hot-cold neuron` algorithm, it partitions matrix operations by neuron clusters and allocates them efficiently between the CPU and NPU, implementing I/O pipeline optimizations for faster inference. During the offline phase, PowerInfer-2 generates an execution plan adapted to neuron activation patterns, hardware constraints, and batch sizes. In the online inference phase, it uses neuron caching along with an NPU-based prefill stage and a CPU-NPU hybrid decoding phase, thus boosting overall performance.

Just Accepted

Representative Characteristics Summary
- **General-Purpose [Low]**: Extends llama.cpp for single consumer GPUs and desktop scenarios.
- **Ease-of-Deploy [Low]**: Pre-built Docker images simplify setup on one GPU.
- **Ease-of-Use [Low]**: Basic scripts are provided, though neuron-level tuning remains manual.
- **Latency-Aware [Medium]**: Hot-cold neuron separation removes some transfers but PCIe overhead persists.
- **Throughput-Aware [Medium]**: Neuron-aware sparse operators moderately raise tokens per second.
- **Scalability [Low]**: Designed for a single GPU with CPU assist and no cluster capability.

4.16 LMDeploy

LMDeploy [213] is an inference and serving engine that incorporates several optimization techniques, including continuous batching [380], dynamic split and fuse, and high-performance CUDA kernels. In addition to facilitating efficient inference, it provides features such as quantization, fine-tuning, and multi-model services across multiple machines and cards, enabling straightforward and effective service deployment in various contexts.

To support high throughput in interactive LLM inference, LMDeploy offers an engine called `TurboMind`, which is built on NVIDIA FasterTransformer [249]. `TurboMind` includes efficient LLM implementations, a Persistent Batch module, and a **KV** Cache Manager, all accessible through a simple API. The Persistent Batch module manages continuous batching with a fixed number of batch slots. When a request arrives, it occupies one of these slots, and upon completion, the slot is freed. Meanwhile, the **KV** Cache Manager functions as a memory pool, applying a Least Recently Used (LRU) policy to decide which sequence cache to evict when additional memory is required.

In addition to `TurboMind`, LMDeploy provides a developer-friendly engine named `lmdeploy.pytorch`, which offers a PyTorch-like environment while sharing the same service interface as `TurboMind`. It performs model loading, adapter integration, cache management, and parallel processing through an Engine object composed of three components. `ModelAgent` encapsulates the model, `Scheduler` handles resource allocation and sequence tracking, and `RequestManager` manages input and output for requests. In particular, the Scheduler uses a mechanism similar to vLLM's PagedAttention [166] to allocate and release blocks based on the sequence length and supports S-LoRA [290], enabling multiple LoRA adapters to operate within limited memory.

Although LMDeploy features both `TurboMind` for high-performance inference and `lmdeploy.pytorch` for easier development, it currently supports only NVIDIA GPU environments.

Representative Characteristics Summary
- **General-Purpose [Low]**: Includes TurboMind and PyTorch engines but remains NVIDIA-only.
- **Ease-of-Deploy [High]**: Docker images and a serve script ease installation, though driver matching is needed.
- **Ease-of-Use [Medium]**: A unified API toggles between high-performance and development modes.
- **Latency-Aware [Medium]**: KV-cache LRU and dynamic split-and-fuse significantly reduce prompt latency.
- **Throughput-Aware [Medium]**: Persistent batching and continuous scheduling keep GPUs fully occupied.
- **Scalability [High]**: Supports multiple GPUs per node; multi-node orchestration is still experimental.

4.17 LightLLM

LightLLM [191] is a Python-based, lightweight, and highly scalable LLM inference engine that addresses performance, scheduling, and memory inefficiencies in existing solutions. Using a three-process asynchronous collaboration approach, it separates tokenization, model inference, and detokenization to boost GPU utilization.

LightLLM replaces PagedAttention [166] with `TokenAttention` and introduces `Efficient Router Scheduling`. LightLLM uses an `Efficient Router` to manage GPU memory at a fine-grained, token-level granularity depending on whether it is in the prefill or decode phase. This router employs a custom algorithm to batch tokens appropriately. Additionally, the scheduling and model inference stages are merged, removing the communication overhead

Just Accepted

between the scheduler and the model-RPC. LightLLM also integrates OpenAI Triton [317] to optimize service scheduling kernels.

The inference engine consists of multiple modules, each running as a separate process (e.g., `Metric Server`, `Health Server`, `HTTP Server`, `Router`). These modules communicate via ZeroMQ [316] or RPC. The `Cache Manager` stores multimodal inference results, while the `Visual Server` handles multimodal requests.

LightLLM also features a `CacheTensorManager` class to handle the allocation and deallocation of Torch tensors. By maximizing inter-layer tensor sharing during runtime and permitting memory sharing across distinct CUDA graphs, it reduces overall memory usage. A `ModelBackend` defines the mechanism and operations needed for prefill or decode requests from the router. Each backend maintains its own model object, supporting parallel existence of multiple backends. The model class performs computations on the device and includes tensor parallelism support.

Representative Characteristics Summary
- **General-Purpose [Low]**: TokenAttention backend offers a lightweight footprint for NVIDIA GPUs.
- **Ease-of-Deploy [High]**: Manual source builds and custom dependencies increase setup complexity.
- **Ease-of-Use [Medium]**: Multi-process ZeroMQ architecture and minimal docs raise the learning barrier.
- **Latency-Aware [Medium]**: Triton-optimized kernels and router fusion shorten critical-path latency.
- **Throughput-Aware [Medium]**: Efficient router scheduling and memory sharing maintain high TPS.
- **Scalability [Medium]**: Multiple back ends can run concurrently; cluster scaling is manual.

4.18 NanoFlow

NanoFlow [405] is a high-performance inference engine that improves LLM throughput by introducing `Nano-batching` and supporting co-scheduling of operations for intra-device parallelism. Traditional systems process pipelines sequentially, often underutilizing hardware resources.

By dividing batches into smaller `nano-batches`, NanoFlow boosts optimization flexibility. It can also estimate GPU memory usage to check whether additional requests fit. If necessary, it offloads **KV** cache data to lower memory tiers—like system memory or disk—maximizing overall resource usage.

To implement `Nano-batching`, NanoFlow classifies LLM service operations into three types: `memory-bound operations` like self-attention computations, `compute-bound operations` such as General Matrix Multiplication (GEMM), and `network-bound operations` such as AllReduce. Then analyzes the resource requirements of each operation and the corresponding iterations or latencies to pinpoint performance characteristics and bottlenecks. Based on these findings, NanoFlow maximizes hardware parallelism to achieve higher throughput.

NanoFlow consists of three primary components. The `global batch scheduler` collects all incoming requests, creates dense batches in high-performance sizes (determined by offline profiling), and uses continuous batching [380] technique to fill these batches dynamically. It also applies chunked prefill [11] operations and a discrete batching approach, selecting only the batch sizes that were identified as optimal rather than arbitrary ones. By prioritizing throughput rather than focusing solely on latency, this method exploits available memory to process more requests in parallel.

Next, the intra-device parallelism engine enables fine-grained parallel operations for `Nano-batching`, along with execution unit scheduling to reduce interference among tasks. Lastly, the `KV cache manager` oversees the decoding status of every request, estimates future memory usage (assuming an average decode length), and manages GPU memory to prevent out-of-memory issues. If predicted usage does not exceed the GPU limits, the request is accepted; otherwise, it is deferred.

However, NanoFlow's `Nano-batching` mechanism requires additional setup—such as per-model schedule optimization—and may need pipeline adjustments or kernel re-implementation for new models. It also introduces overhead, potentially lowers efficiency for individual operations due to smaller batch sizes, and remains dependent on NVIDIA GPUs.

JIST Accepted

Representative Characteristics Summary
- **General-Purpose [Low]**: Operates solely on NVIDIA GPUs and demands per-model nano-schedule tuning.
- **Ease-of-Deploy [Low]**: Research-grade code requires custom schedule files and environment tweaks.
- **Ease-of-Use [Low]**: Sparse documentation and pipeline modifications limit accessibility.
- **Latency-Aware [Medium]**: Memory forecasting and KV offload avoid OOM stalls, indirectly cutting latency.
- **Throughput-Aware [Medium]**: Nano-batching plus intra-device parallelism greatly boost throughput.
- **Scalability [Medium]**: Confined to a single node without distributed scheduling.

4.19 DistServe

DistServe [402] is a serving system designed to efficiently run LLM inference across multiple GPU clusters while keeping latency low. It breaks down LLM inference requests at a granular level to enable parallel execution, thereby boosting throughput and resource utilization. Traditional inference engines handle prefill and decode on a single device, causing resource interference and pipeline inefficiencies. By decoupling them and applying both intra-operation and inter-operation parallelization via SwiftTransformer [295], DistServe reduces overhead.

DistServe also addresses large model sizes, such as a 175B-parameter model that can require 350GB of memory. It uses a low node-affinity placement algorithm for batch allocation, relying on NVLink when computations for a given stage remain on the same node. Online scheduling further manages workloads in real time to meet latency SLO requirements.

DistServe consists of a batching algorithm module, a `RESTful API frontend`, an `orchestration layer`, and a `parallel execution engine`. The batching module provides a simulator and algorithms to optimally distribute requests based on particular models and cluster setups. The `RESTful API frontend` supports an OpenAI-compatible interface and accepts user inputs such as maximum output length and temperature. The `orchestration layer` manages prefill and decode instances, handles request dispatching, and coordinates **KV** cache transfers. For inter-node GPU communication, DistServe uses NCCL [247], while intra-node transfers rely on asynchronous memory copy. Individual instances run as GPU workers through Ray [240], driven by a `parallel execution engine`.

Because DistServe is intended for large GPU clusters, its parallel strategies and resource allocations can be difficult to adapt to smaller-scale or resource-constrained settings (e.g., single or few-GPU systems), potentially limiting performance in those scenarios.

Representative Characteristics Summary
- **General-Purpose [Low]**: Aims at very large models across multi-GPU clusters.
- **Ease-of-Deploy [Low]**: Requires Ray cluster setup and NVLink topology awareness.
- **Ease-of-Use [Low]**: Placement-algorithm tuning and orchestration add complexity for operators.
- **Latency-Aware [Medium]**: Decoupled prefill and decode phases reduce tail latency under load.
- **Throughput-Aware [High]**: Intra- and inter-operation parallelization plus low node-affinity batching maximize throughput.
- **Scalability [High]**: Designed for multi-node clusters, scaling to hundreds of GPUs.

4.20 vAttention

vAttention [268] is a inference engine for dynamically managing **KV** cache memory during LLM inference. Built on Sarathi-Serve [11], it includes components such as `sarathi-lean`, a `vattention memory allocator`, and a `custom Unified Virtual Memory (UVM) driver`. These elements support both PagedAttention [166] and vAttention-style memory management.

vAttention addresses the complexity and performance limitations linked to virtual contiguity in PagedAttention—commonly used in transformer-based LLMs. It enhances performance (especially in prefill-bound workloads) while staying compatible with existing kernels. To achieve this, vAttention modifies PyTorch [263]

Just Accepted

caching allocator to introduce virtual tensors, reserving virtual memory buffers without allocating physical memory from the start.

Unlike PagedAttention, where LLM serving systems must manually handle mappings between **KV** cache and dynamic memory blocks, vAttention integrates memory allocation and computation and enables predictive page allocation. It separates virtual and physical memory usage via low-level CUDA APIs (rather than `cudaMalloc`), and supports optimizations that target NVIDIA's Hopper architecture through FlashAttention-3 [285], restricting it to NVIDIA GPUs.

vAttention is implemented as a Python library that wraps CUDA/C++ extension libraries that interfacing with the CUDA driver. During model serving, each worker sets up vAttention based on model parameters and page group sizes, allocating virtual tensors as needed. It checks whether the **KV** cache is mapped to physical memory before launching kernels, tracking page allocations during both prefill and decode. Only when all current pages are used does it allocate new pages and it frees or reclaims pages once a request ends.

Representative Characteristics Summary
- **General-Purpose [Low]**: Tailored for NVIDIA Hopper GPUs, limiting portability.
- **Ease-of-Deploy [Low]**: CUDA driver patches and custom UVM setup complicate installation.
- **Ease-of-Use [Low]**: Experimental wrapper and minimal docs hamper quick adoption.
- **Latency-Aware [Medium]**: Predictive page allocation hides memory-map costs and speeds prefill.
- **Throughput-Aware [Medium]**: Integrated KV memory and compute paths provide moderate gains.
- **Scalability [Medium]**: Currently supports only single-node, single-GPU execution.

4.21 Sarathi-Serve

Sarathi-Serve [10] is a high-performance inference scheduler built on vLLM [166] to address the trade-off between throughput and latency in LLM inference. It relies on FlashAttention-2 [63] and FlashInfer [375] as backends to enhance decode-stage throughput in multi-GPU and multi-node environments.

Previous systems, such as Orca [380] and vLLM [166], faced generation stalls—where decode requests wait because of prolonged prefill—and pipeline inefficiencies—where insufficient parallelism at the request level left GPU resources underused. Sarathi-Serve tackles these problems via chunked prefill and stall-free scheduling, cutting down TBT while offering high throughput and minimal TBT latency.

Sarathi-Serve decides the maximum number of tokens (`token budget`) in each batch based on TBT SLOs and chunked prefill overhead. Under strict latency requirements, it sets a smaller token budget and splits prompts into smaller chunks, lowering tail latency at the cost of some overall system efficiency. Under looser latency constraints, it raises the token budget to improve prefill efficiency. With token budgets like 2,048 or 512, Sarathi-Serve provides efficient inference for varying SLO conditions.

Representative Characteristics Summary
- **General-Purpose [Low]**: Extends vLLM scheduling to multiple model categories.
- **Ease-of-Deploy [Low]**: A simple CLI launches servers, yet CUDA and NCCL versions must align.
- **Ease-of-Use [Low]**: Interactive SLO slider lets users trade latency for throughput with ease.
- **Latency-Aware [Medium]**: Chunked prefill and stall-free scheduling keep TBT consistently low.
- **Throughput-Aware [Medium]**: Token-budget batching adapts to workload for maximum throughput.
- **Scalability [Medium]**: Multi-GPU and multi-node deployments are supported via FlashAttention-2.

4.22 Friendli Inference

Friendli Inference [87] is a commercial LLM inference engine built on top of Orca [380] and designed to enhance inference through features such as iteration batching. It supports both web- and API-based serving via `Friendli`

Just Accepted

`Container` and `Friendli Serverless/Dedicated Endpoints`, with the latter focusing on stable service by managing traffic and adhering to service-level agreements (SLAs). Users can integrate Friendli AI solutions with Amazon SageMaker [18] and gRPC [112] inference servers, while monitoring is facilitated through tools like Grafana [47].

For optimization, Friendli Inference allows serving multiple LoRA [133] models on a single GPU, maximizing utilization of different user-defined models. It introduces `TCache`, a GPU load reduction technique that caches frequently accessed results to maintain a high TTFT compared to conventional frameworks. Quantization techniques are also used to further improve the inference performance. However, Friendli Inference primarily targets NVIDIA GPUs, limiting its support on other hardware platforms.

Representative Characteristics Summary
- **General-Purpose [Low]**: Focuses on hosting multiple LoRA models on NVIDIA GPUs.
- **Ease-of-Deploy [High]**: Container and serverless options plus SageMaker integration simplify rollout.
- **Ease-of-Use [Medium]**: Web console with Grafana metrics streamlines monitoring and management.
- **Latency-Aware [Medium]**: TCache lowers time-to-first-token, though deeper latency tooling is not described.
- **Throughput-Aware [Medium]**: Iteration batching and quantization deliver strong tokens-per-second performance.
- **Scalability [Medium]**: Dedicated and serverless GPU instances scale horizontally, but non-GPU back ends are absent.

4.23 Fireworks AI

Fireworks AI [83] is a inference platform for rapid and efficient serving of both LLMs and image models, supporting inference, fine-tuning, and deployment. It offers a simple interface and APIs—compatible with services like LangChain [167] and the OpenAI API—and can operate in serverless, on-demand, or enterprise environments. Beyond NVIDIA GPUs, Fireworks AI also supports LLM inference on AMD Instinct MI300X [300], broadening its hardware compatibility.

To meet diverse throughput and latency demands, Fireworks AI uses multiple parallelization and optimization techniques, including multi/group query attention optimization, sharding, quantization, and continuous batching. It provides deployment configurations specifically tailored for low latency, high throughput, or long input/output sequences. In particular, Fireworks AI employs its own MQA [288] model and a custom CUDA kernel (`FireAttention`) to further accelerate inference.

To ensure service reliability, Fireworks AI is SOC 2 Type II [12] and HIPAA [224] certified, ensuring privacy, security, availability, processing integrity and confidentiality.

Representative Characteristics Summary
- **General-Purpose [Low]**: Serves LLMs and vision models on NVIDIA GPUs and AMD MI300X accelerators.
- **Ease-of-Deploy [High]**: Cloud console offers API keys and quick templates for inference or fine-tuning.
- **Ease-of-Use [Medium]**: OpenAI-compatible endpoints and LangChain adapters reduce integration effort.
- **Latency-Aware [Medium]**: FireAttention kernels and low-latency deployment profiles minimize response time.
- **Throughput-Aware [Medium]**: Multi-query attention, sharding, and continuous batching provide high TPS.
- **Scalability [High]**: Serverless and enterprise clusters scale elastically with SOC 2 and HIPAA compliance.

4.24 GroqCloud

GroqCloud [111] is an AI infrastructure platform focused on delivering high-performance, low-latency inference for LLMs through a specialized hardware architecture and software stack. It aims to resolve bottlenecks and non-deterministic behaviors commonly found in conventional GPU systems by offering inference services powered by the Groq Language Processing Unit (LPU) [3, 4]. Built specifically for AI inference, the Groq LPU achieves lower latency and higher throughput than traditional GPUs. Its Tensor Streaming Processor (TSP) architecture [4]

Just Accepted

statically schedules and locks the model execution path at compile time, eliminating runtime variability and enabling predictable response times.

A key advantage of Groq LPU is its ability to maintain optimal performance even with a batch size of one, making it well suited for latency-sensitive applications like financial trading and autonomous driving. Through its `TruePoint` technology, the Groq LPU delivers near-FP32 precision when using FP16 or INT8 computations. For high-throughput workloads, GroqCloud provides an asynchronous batching API, Flex Processing for scalable throughput, and deterministic QoS scheduling to meet a range of SLAs. Additionally, its kernel-less compiler approach removes the need for manual kernel optimization, simplifying development, and lowering maintenance overhead.

However, because GroqCloud relies on static compilation, it may have limited flexibility in dynamically adjusting batch sizes or handling complex runtime branching.

Representative Characteristics Summary
- **General-Purpose [Low]**: Runs exclusively on Groq LPUs, limiting hardware flexibility.
- **Ease-of-Deploy [High]**: Fully managed cloud API hides all compilation and runtime details.
- **Ease-of-Use [Low]**: Deterministic QoS and simple REST calls facilitate rapid integration.
- **Latency-Aware [Low]**: TSP architecture yields sub-millisecond latency even at batch size 1.
- **Throughput-Aware [Medium]**: Flex Processing sustains high throughput without hurting single-request latency.
- **Scalability [High]**: LPU pods scale horizontally under the same deterministic schedule.

4.25 Together Inference

Together Inference [318] is part of the Together AI platform, offering high-performance LLM inference with an emphasis on speed, cost, and accuracy. To enhance LLM serving, it implements transformer-optimized CUDA kernels, quantization [76], and speculative decoding [173]. Together Inference provides different model configurations to meet diverse needs, from maximum performance and full-precision accuracy to lower cost and higher throughput. It supports dedicated instances, serverless deployments, and multi-GPU environments; however, it is optimized exclusively for NVIDIA GPU-based services.

Representative Characteristics Summary
- **General-Purpose [Low]**: Offers multiple accuracy-versus-cost tiers, but only on NVIDIA GPUs.
- **Ease-of-Deploy [High]**: Provides serverless endpoints and dedicated instances through a web UI.
- **Ease-of-Use [Medium]**: OpenAI-style API eases migration for existing clients.
- **Latency-Aware [Medium]**: Speculative decoding and custom CUDA kernels reduce median and tail latencies.
- **Throughput-Aware [Low]**: Quantization, optimized attention, and continuous batching significantly raise TPS.
- **Scalability [High]**: Multi-GPU instances scale vertically; adding endpoints enables horizontal growth.

5 LLM Inference Optimization

LLM inference performance depends not only on the model size and hardware environment but also on various inference optimization techniques. The specialized LLM inference engines introduced earlier seek low latency and high throughput by employing parallelism, efficient memory management, kernel optimization, and quantization.

This section explains key inference optimization strategies, including parallel processing, memory optimization, latency, and throughput optimization. In addition to the methods provided by existing inference engines, we also examine recent research findings on inference optimization. Table 8 summarizes the optimization techniques supported by each LLM inference engine.

Figure 9 presents full system-level layering of the LLM inference optimization techniques summarized in Table 8. Effective LLM inference optimization is not attainable by improving a single hardware or software

Just Accepted

Table 8. Optimizations of LLM Inference Engines

Engines	Batch Optimization				Parallelism					Compression			Fine-tuning	Caching			Attention Optimization		Sampling Optimization	Output Optimization
	Dynamic Batching	Continuous Batching	Nano Batching	Chunked-prefills	Data Parallelism	Fully Sharded Data Parallel	Expert Parallelism	Tensor Parallelism	Pipeline Parallelism	Quantization†/ Support Quantized Model	Pruning	Sparsity‡	LoRA	Prompt Caching	Prefix Caching	KV Caching	PagedAttention	FlashAttention	Speculative Decoding	Structured Outputs*
Ollama [255]	✘	✘	✘	✘	✘	✘	✘	✔	✔	✔	✔	✔ (MoE, SL)	✔	✔	✘	✔	✘	✔	✔	✔ (GG)
llama.cpp [101]	✘	✔	✘	✘	✘	✘	✘	✔	✔	✔	✘	✔ (MoE, SL)	✔	✔	✘	✔	✘	✔	✔	✔ (GG)
vLLM [166]	✘	✔	✘	✔	✔	✔	✔	✔	✔	✔ (A, A*, B, D, G, L, M)	✔	✔ (BS, MoE, N:M)	✔	✘	✔	✔	✔	✔ (v3)	✔	✔ (LM, OA, OL, XG)
DeepSpeed-FastGen [129]	✘	✔	✘	✔	✔	✔	✔	✔	✔	✔	✔ (S, U)	✔ (N:M, NxM, MoE, SA)	✔	✘	✘	✔	✔	✔ (v2)	✘	✘
Unsloth [325]	✘	✘	✘	✘	✘	✘	✘	✘	✘	✔ (B)	✘	✘	✔	✘	✘	✔	✘	✔	✘	✘
MAX [237]	✘	✔	✘	✔	✘	✘	✔	✔	✘	✔	✘	✔ (MoE)	✔	✘	✔	✔	✔	✔ (v3)	✔	✔ (XG)
MLC-LLM [234]	✘	✔	✘	✔	✘	✘	✘	✔	✔	✔ (A)	✘	✔ (MoE)	✘	✘	✔	✔	✔	✘	✔	✔ (XG)
llama2.c [24]	✘	✘	✘	✘	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘
bitnet.cpp [336]	✘	✘	✘	✘	✘	✘	✘	✘	✘	✔ (A, G)	✘	✔ (MoE)	✘	✘	✘	✔	✘	✘	✘	✘
SGLang [399]	✘	✔	✘	✔	✔	✔	✔	✔	✘	✔ (A, B, G, L, M)	✔	✔ (DSA, N:M, MoE)	✔	✘	✔	✔	✔	✘	✔	✔ (LL, OL, XG)
LitGPT [194]	✘	✔	✘	✘	✔	✔	✘	✔	✘	✔ (B)	✘	✔ (MoE)	✔	✘	✘	✔	✘	✔ (v2)	✔	✘
OpenLLM [37]	✘	✔	✘	✘	✔	✘	✘	✘	✘	✔ (B, G)	✘	✘	✘	✘	✘	✘	✘	✘	✘	✘
TensorRT-LLM [252]	✔	✔	✘	✔	✔	✘	–	✔	✔	✔ (A, G, S, W)	✔	✔ (BSA, MoE)	✔	✔	✘	✔	✔	✘	✔	✔ (XG)
TGI [139]	✘	✔	✘	✘	✘	✘	✔	✔	✘	✔ (A, E, E*, G, M)	✔	✔ (N:M, MoE)	✘	✘	✔	✔	✔	✔ (v2)	✔	✔ (OL)
PowerInfer [302, 362]	✘	✔	✘	✘	✔	✘	✘	✘	✔	✔	✘	✔	✔	✔	✘	✔	✘	✔	✔	✔ (GG)
LMDeploy [213]	✘	✔	✘	✔	✘	✘	✔	✔	✘	✔ (A, G, S)	✔	✔ (MoE)	✔	✘	✔	✔	✔	✘	✘	✔ (PT)
LightLLM [191]	✔	✘	✘	✔	✘	✘	✔	✔	✘	✔	✘	✔ (MoE)	✘	✔	✘	✔	✘	✔ (v1)	✘	✔ (OL, XG)
NanoFlow [405]	✘	✔	✔	✔	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘
DistServe [402]	✔	✔	✘	✔	✘	✘	✘	✔	✔	✘	✘	✘	✘	✘	✘	✔	✔	✔ (v1)	✘	✘
vAttention [268]	✔	✘	✘	✘	✔	✘	✘	✔	✔	✔	✔	✔ (N:M)	✔	✘	✘	✔	✔	✔ (v2)	✘	✘
Sarathi-Serve [10]	✘	✘	✘	✔	✘	✘	✘	✔	✔	✘	✘	✔ (MoE)	✘	✘	✘	✔	✘	✔ (v2)	✘	✘
Friendli Inference [87]	–	✔	–	–	–	–	–	✔	✔	✔	–	✔ (MoE)	✔	–	–	–	–	–	✔	✔
Fireworks AI [83]	–	✔	–	–	–	–	–	–	–	✔	✔	✔ (MoE)	✔	✔	✔	✔	–	–	✔	✔ (OA)
Groq Cloud [111]	✘	–	–	–	✔	–	–	✔	✔	✔	✔	✔ (MoE)	–	–	–	–	–	–	✔	✔ (OA)
Together Inference [318]	–	–	–	–	–	✔	–	–	–	✔	–	✔ (MoE)	✔	✔	–	–	–	✔ (v3)	✔	✔

†A: AWQ, A*: AQLM, B: bitsandbytes, D: DeepSpeedFP, E: EXL2, E*: EETQ, G: GPTQ, L: LLM Compressor, M: Marlin, S: SmoothQuant, W: Weight-only
‡BS: Block Sparse, BSA: Block Sparse Attention, DSA: Double Sparse Attention, SL: SparseLLM
*GG: GBNF, LL: llguidance, LM: lm-formet-enforcer, OA: OpenAI API, OL: outlines, PT: PyTorch, XG: XGrammar

TLSI Accepted

component, it requires an end-to-end strategy that spans from the service layer through the hardware layer, embracing co-design and cross-layer coordination.

At the `Service` layer, techniques such as dynamic batching [15, 59] and continuous batching [126, 380] are used to efficiently handle multiple user requests. These batching optimizations balance latency and throughput and serve as critical factors to ensure quality in large-scale service environments.

The `Model` and `Serving System` layer, optimizations are applied to align with model architecture and serving methods. Prompt Caching [404], Prefix Caching [202, 261], and **KV** Caching [266] reduce memory usage and computation for long-context processing, while MoE [46, 74, 81] and sparsity-based methods [89, 369] decrease internal computation to allow efficient inference.

The `Inference Engine` layer directly improves token-level generation. Speculative Decoding and Guided Decoding [45, 55, 183] accelerate token prediction, while fine-tuning methods such as LoRA [67, 133] reduce computational resource demands.

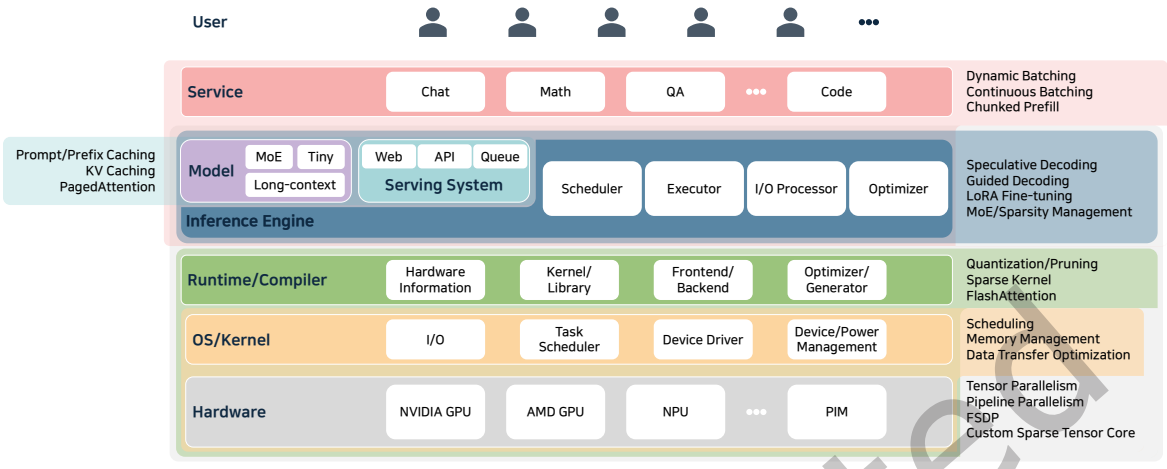


Fig. 9. Optimization Techniques across the LLM Inference

Table 9. Optimization Techniques in LLM Inference: Points and Suitable Workloads

Optimization Technique	Inference Optimization Point	Suitable Workload
Batch Optimizations (§5.1)	Increase GPU utilization by grouping requests; balance latency vs. throughput	Multi-user chatbot services, API serving, large-scale concurrent requests
Parallelism (§5.2)	Distribute model/compute across GPUs and nodes; enable handling of ultra-large models	70B+ model serving, distributed clusters, enterprise/research-scale LLMs
Compression (§5.3)	Reduce memory footprint and FLOPs; enable execution on resource-limited hardware	Edge devices, mobile deployment, personal or lightweight inference servers
Inference-aware Fine-tuning (§5.4)	Train only small adapter layers for domain-specific adaptation	Enterprise-specific chatbots, domain QA, customized LLMs with low training cost
Caching (§5.5)	Reuse prefix and past tokens to avoid redundant computation	RAG-based QA, conversational assistants with repeated context, long-context summarization
Attention Optimization (§5.6)	Reduce attention complexity from $O(n^2)$ to $O(n)$ or $O(n \log n)$; efficient long-sequence processing	Long-context document QA, summarization, code assistants, multi-turn dialogue
Decoding Algorithm (§5.7, §5.8)	Accelerate token generation via prediction and guidance strategies	Real-time chatbots, math, voice assistants, interactive applications requiring low latency

At the `Runtime` and `Compiler` layer, optimizations are performed at the graph and kernel level, including quantization [50, 77, 178, 355], pruning [89, 251, 309], sparse kernels [93, 353, 389, 390], and kernel fusion [82, 310, 395]. These techniques maximize FLOPs utilization and memory bandwidth efficiency, ensuring stable high performance on actual hardware.

The `OS` and `Kernel` layer manages resources and data transfer. Task scheduling and memory management enhance the utilization of GPU, I/O, and network resources, while ensuring that memory-intensive structures, such as the **KV** cache, operate efficiently.

The `Hardware` layer provides the foundation for large-scale parallel execution. Parallelism [11, 278, 305, 396] and device-to-device interconnection distribute model parameters and computation across multiple accelerators, while specialized features such as sparse tensor cores [80] provide direct support for sparsity optimization.

Each optimization technique must be considered not only by its placement, but also in terms of the bottlenecks it alleviates and the workloads it best supports. Table 9 categorizes the commonly used optimization techniques

Just Accepted

Table 10. Summary of Main Optimization Focus by LLM Inference Engine

Engine	Main Optimization Focus
vLLM [166], TGI [139], TensorRT-LLM [252], DeepSpeed-FastGen [129], LMDeploy [213], DistServe [402]	Batch scheduling, KV caching, attention optimization, decoding acceleration; optimized for large-scale serving
Ollama [255], llama.cpp [101], llama2.c [24], bitnet.cpp [336], PowerInfer [302]	Compression (quantization, pruning), lightweight KV caching; optimized for edge and personal inference
SGLang [399], MLC-LLM [234], MAX [237], LitGPT [194], OpenLLM [37], unsloth [325], NanoFlow [405], vAttention [268]	Experimental attention variants and decoding strategies; suitable for research and flexible customization
LightLLM [191], Sarathi-Serve [10], Friendli Inference [87]	Long-context attention optimization and speculative decoding; optimized for real-time long-sequence serving
GroqCloud [111], Fireworks AI [83], Together Inference [318]	Ultra-low latency decoding; optimized for real-time interactive services

and maps them to their performance targets and suitable workloads. In practice, real-world LLM applications or services often combine multiple techniques rather than relying on a single optimization.

Simply categorizing optimization techniques based on their primary performance objectives is insufficient. It is also important to understand how each technique is applied in concrete downstream tasks, applications, or service scenarios. In real-world serving environments, factors such as the scale and configuration of available hardware resources, interaction patterns with users, input context length, and acceptable latency constraints vary widely. As a result, the selection and combination of inference optimization techniques must be adapted to these conditions. Therefore, beyond the theoretical performance gains of individual techniques, it is essential to consider their practical applicability and impact within actual service contexts.

For example, interactive chatbots and conversational assistants typically operate in environments with multiple concurrent users, making it essential to achieve both low latency and high throughput by combining batch processing, caching, and decoding optimizations. In such applications, TTFT and ITL have a direct impact on user experience, rendering the efficiency of scheduling and decoding phases critical factors in system performance.

In contrast, tasks such as document summarization and long-form question answering often involve extended input sequences, which require efficient memory management and computational optimizations to handle long contexts. In these cases, reducing the complexity of attention operations or applying caching techniques to minimize redundant computations during the prefill phase can be effective in lowering both memory usage and computational cost.

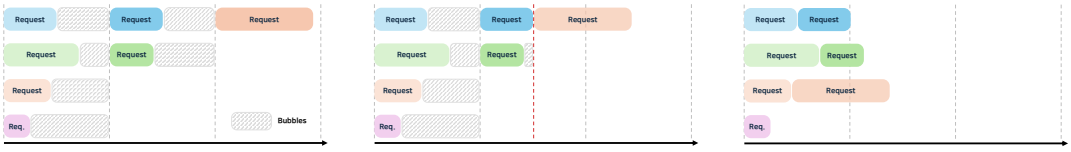
In RAG workloads, multiple queries often share common retrieval results or similar contextual information. Applying techniques such as prefix caching can therefore reduce redundant prefill computations, leading to lower end-to-end inference latency and overall serving cost.

Beyond service-level considerations, the requirements of the execution environment in which LLM inference is performed must also be taken into account. In environments such as on-device assistants or lightweight personal inference services, strict constraints on memory capacity and power budget necessitate prioritizing model compression techniques such as quantization and pruning to reduce model size. In these environments, reliable responsiveness and resource efficiency are more important goals than peak performance, and choosing a lightweight model along with an inference engine optimized for these constraints plays a critical role in determining overall system performance.

In large-scale server environments processing large models, parallel processing and batch processing across multiple GPUs play a key role in overcoming the limitations of a single device. By leveraging model parallelism and data parallelism, computational and memory workloads can be distributed across devices, enabling the system to handle multiple requests concurrently and thereby maximize overall throughput.

These examples demonstrate that an efficient LLM inference system should not rely on a single optimization technique, but should be designed by combining multiple optimization strategies tailored to the characteristics of a specific downstream and the constraints of the service environment.

Just Accepted



(a) Static Batching (b) Dynamic Batching (c) Continuous Batching

Fig. 10. Comparison of Batching Strategies

Based on Table 10, the LLM inference engines discussed in this paper can be classified as follows: serving-oriented engines such as vLLM [166] TGI [139], and TensorRT-LLM [252] provide batch scheduling, caching, attention optimization, and decoding acceleration, making them suitable for large-scale services. Lightweight execution engines such as llama.cpp [101], Ollama [255], and PowerInfer [302] leverage quantization and pruning to provide efficient inference on personal devices or edge environments. Research- and experiment-focused engines such as SGLang [399] and MLC-LLM [234] support diverse attention variants and decoding strategies, while LightLLM [191] and Sarathi-Serve [10] are optimized for long-context processing and real-time interaction. Commercial engines such as GroqCloud [111], Fireworks AI [83], and Together Inference [318] employ low-latency decoding optimizations to enable industrial-scale real-time services.

Ultimately, LLM inference optimization is determined by the interaction of technologies across all layers, and this paper systematically presents which layers of the system stack are supported by each inference engine.

5.1 Batch Optimization

In LLM inference, batching groups multiple input requests for simultaneous processing, boosting hardware utilization and throughput. Efficient batch processing is essential to maximize computational parallelism and latency.

Hence, finding the optimal batch size is essential. Smaller batches reduce response time but may underuse hardware resources, while larger batches yield higher throughput but risk longer response times. Various methods have been proposed to select optimal batch sizes [118, 264, 291], and inference engines typically provide mechanisms to explore the best batch size based on workload and SLOs.

Beyond batch size, the scheduling method also significantly influences inference performance. As shown in Fig. 10 (a), static batching processes a fixed number of requests, potentially increasing latency as new requests must wait until a batch completes. Dynamic batching [15, 59] and continuous batching [126, 380], in contrast, adapt the batch in real time, often reducing latency and increasing overall efficiency.

5.1.1 Dynamic Batching. Dynamic batching [15, 59] alleviates the latency and hardware underutilization issues of static batching. As shown in Fig. 10 (b) new requests are immediately added to an ongoing batch, enabling more flexible and efficient inference.

Unlike static batching, dynamic batching reconstructs batches based on incoming requests and available hardware resources, adaptively determining batch sizes. When a new request arrives, it can be merged with an existing batch or appended to an ongoing process to optimize resource usage.

Several parameters must be tuned to implement dynamic batching effectively, including the maximum batch wait time, minimum batch size, and batch size limits. Although dynamic batching can minimize latency by reducing batch size in real time, new requests can only be added after the current batch finishes. It may also introduce overhead for dynamically resizing batches and degrade performance if requests have widely varying prompt or output token lengths.

Just Accepted

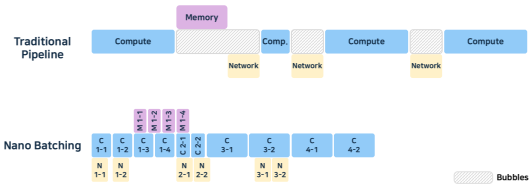


Fig. 11. Nano-batching

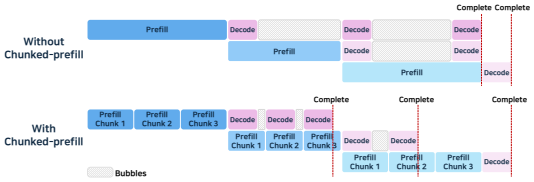


Fig. 12. Chunked-prefills

5.1.2 Continuous Batching. Continuous batching [126, 380] is similar to dynamic batching [15, 59] but allows new requests to join an ongoing batch without interruption, minimizing latency. Fig. 10 (c) shows how requests are continuously inserted to maximize GPU and memory efficiency.

Orca [380] implements continuous batching via Iteration-Level Scheduling and Selective Batching. Iteration-Level Scheduling forms batches each iteration while accommodating new requests on the fly. Selective Batching focuses only on batchable transformer operations, enabling immediate results for completed requests and lowering both the average response time and the waiting time.

However, continuous batching requires sophisticated scheduling that new requests can be integrated without disrupting active processing. Efficient **KV** cache management is crucial, often involving methods such as PagedAttention [166]. Inference engines such as llama.cpp [101], DeepSpeed-FastGen [129], and vLLM [166] use continuous batching techniques derived from Orca.

5.1.3 Nano-batching. Nano-batching, introduced by NanoFlow [405], maximizes resource utilization and throughput by running `computation`, `memory`, and `network-bound operations` in parallel on a single device. Traditional inference engines batch tasks at the request level, but NanoFlow divides them at the operation level, as illustrated in Fig. 11.

Operation units include attention and **KV** generation, GEMM, and collective communication for multi-GPU synchronization. NanoFlow dynamically adjusts nano-batch sizes to optimize each resource type, employing a scheduling approach that merges topological sorting and greedy search based on hardware resources and kernel optimizations.

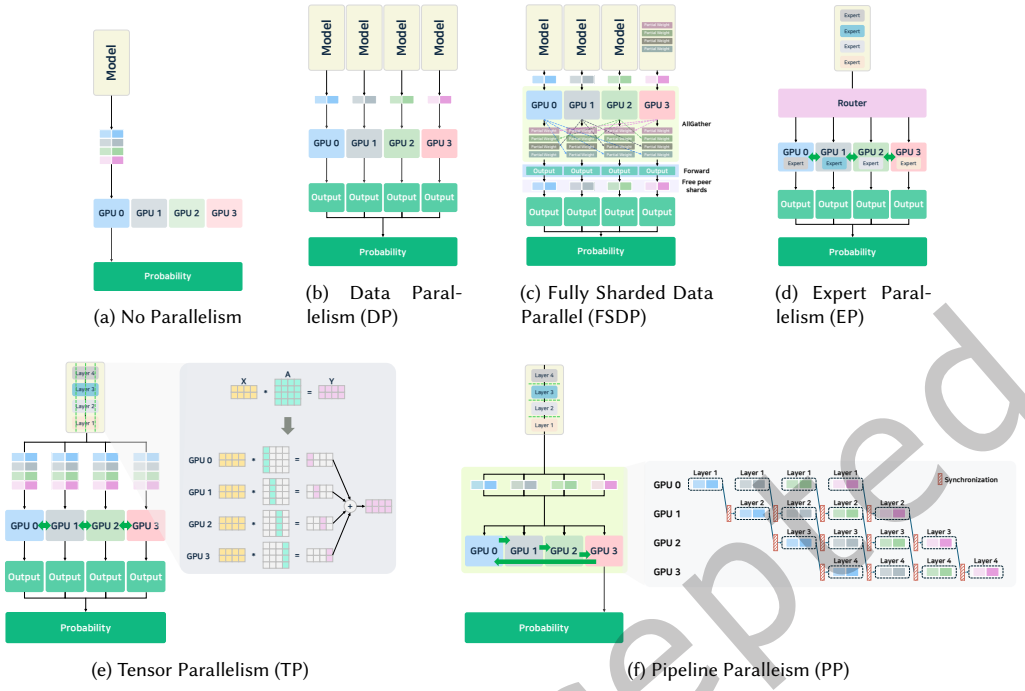
By breaking operations into smaller nano batches, tasks can overlap and run concurrently with user-server network operations, boosting resource utilization and increasing throughput. However, nano-batching demands complex scheduling and can incur additional communication overhead when operations are spread across multiple GPUs.

5.1.4 Chunked-prefills. Chunked prefills [11] addresses pipeline inefficiencies that arise in dynamic or continuous batching, especially in multi-GPU environments where the memory-bound decode phase might sit idle if a batch is empty. Processing long prompts in one step can also increase latency and hinder shorter requests.

Fig. 12 illustrates chunked prefills, which splits long prompts into multiple segments and processes them incrementally. The decoding for the first segment can begin immediately while subsequent segments undergo prefill, allowing these phases to run concurrently and improving resource usage.

However, chunked prefills adds scheduling complexity by requiring more granular batch management. **KV** cache usage can also surge due to simultaneous prefill and decode execution. For example, DeepSpeed-FastGen [129]'s Dynamic SplitFuse splits prompts to generate tokens earlier, while Sarathi-Serve [10] stall-free scheduling immediately admits new requests, eliminating wait time and boosting efficiency.

Just Accepted



(a) No Parallelism (b) Data Parallelism (DP) (c) Fully Sharded Data Parallel (FSDP) (d) Expert Parallelism (EP)

(e) Tensor Parallelism (TP) (f) Pipeline Paralleism (PP)

Fig. 13. Comparison of Parallelism Strategies

5.2 Parallelism

Because LLMs may contain billions or even trillions of parameters, relying on a single GPU or similar hardware for inference has become increasingly challenging. As a result, distributed and parallel processing across multiple devices or nodes is vital for reducing latency and maximizing hardware utilization.

Parallelism strategies in LLM inference differ in their implementation and performance based on server architecture and hardware configuration. Factors such as the number of available GPUs or accelerators, interconnect bandwidth, memory hierarchy, and computational capacity influence how effectively tensor parallelism (TP) [267, 305], data parallelism (DP) [278], FSDP [396], and pipeline parallelism (PP) [11, 135] can be employed. With the widespread adoption of MoE models, the expert parallelism (EP) [33, 197, 406] has also been introduced, in which the expert modules selected by each token are distributed across multiple devices and executed in parallel. Furthermore, hybrid approaches can combine multiple strategies to further improve performance [54, 361].

For example, in multi-node clusters, inter-node communication latency may become a bottleneck, necessitating techniques like communication compression or asynchronous scheduling to maintain high performance. Conversely, in single-node, multi-GPU setups, shared memory, and high-speed interconnects (e.g., NVLink, NVSwitch) allow for more efficient synchronization and workload distribution. Ultimately, the success of any parallelization strategy depends on balancing computation and communication overhead, emphasizing the need to tailor the approach to each unique hardware environment and model architecture. To address these challenges, researchers are actively developing methods to automatically explore and identify optimal parallelism strategies for each system [185, 232]. Various parallelism mechanisms are shown in Fig. 13.

In addition to inter-device parallelization, internal device parallelization strategies also significantly impact LLM inference performance. A representative example is the distribution of GEMM operations across multiple

Just Accepted

thread blocks, where the conventional approach partitions the matrix into fixed-size tiles and maps each block to a single tile. However, when the number of tiles does not divide evenly by the number of thread blocks, idle threads emerge, leading to wasted computational resources. To mitigate this inefficiency, the Stream-K [260] method has been proposed. Instead of partitioning by tiles, Stream-K decomposes the multiply-accumulate (MAC) loop into finer-grained units and distributes them evenly, thereby maximizing hardware utilization. Experiments implemented in CUTLASS report up to a 14.7× performance improvement for FP16 operations on NVIDIA GPUs. Currently, several inference engines, including vAttention [268], llama.cpp [101], Ollama [255], TensorRT-LLM [252], and SGLang [399], support Stream-K-based GEMM parallelization.

5.2.1 Data Parallelism. As shown in Fig. 13 (b), DP [278] replicates the same model across multiple GPUs or nodes. A mini-batch is split among available hardware devices, each performing inference independently on its share of the data. Once the computations are completed, the outputs (or weights) are collected into a single device to produce the final results.

Although this method is straightforward to implement and features relatively low communication overhead—since synchronization happens only after inference—DP can become impractical if the entire model must reside on each device, especially for massive LLMs. Furthermore, if hardware devices differ significantly in performance, the overall system may experience bottlenecks.

5.2.2 Fully Sharded Data Parallelism. FSDP [396] is a parallelism technique designed to reduce memory usage and improve training efficiency when working with LLMs. Unlike traditional data parallelism, where each device holds a full copy of the model parameters and optimizer states, FSDP shards the model's parameters, gradients, and optimizer states across multiple devices. This removes duplicated memory usage and allows larger models to be trained on the same hardware resources.

As shown in Fig. 13 (c), FSDP works by gathering all the parameters of a layer on each GPU right before that layer is executed. This allows full computation to happen on each GPU. The full parameters are temporarily loaded into memory only during this operation and are removed right after the layer finishes. This approach does not split the operation itself, making it simple to implement and compatible with most models.

However, since parameters must be all-gathered at every layer, there is a communication overhead. This can lead to performance issues during inference, especially for workloads with small batch sizes or where low latency is important. Also, if a layer requires more memory than what a single GPU can handle during the all-gather phase, it cannot be run.

During training, FSDP brings large memory savings by sharding activations and parameters. But during inference, there is no gradient or activation recomputation, thus memory savings are smaller. Therefore, the use of FSDP during inference should be decided based on model size.

FSDP is natively supported in PyTorch and works well with its autograd engine, checkpointing, and mixed precision training. It can also be combined flexibly with other parallel strategies such as hybrid parallelism. Among the LLM inference engines we studied, vLLM[166], DeepSpeed-FastGen[129], and SGLang [399], LitGPT [194] support FSDP.

5.2.3 Expert Parallelism. As MoE models [46] have become more widespread, the expert parallelism (EP) [33, 197, 406] has been proposed to enable efficient inference for this architecture. In MoE models, only a subset of experts with the highest scores are activated for each token; therefore, replicating all experts across all devices, as done in traditional parallelization methods, leads to unnecessary memory usage and computational overhead.

As illustrated in Fig. 13 (d), EP distributes the set of experts across multiple devices, where each device retains only the weights of its assigned experts and processes only the tokens corresponding to those experts. Specifically, the router (or gate) network computes scores for all experts, selects the top-k experts, and transfers the input

Just Accepted

tokens to the devices hosting those experts via `All-to-All` communication. This approach significantly reduces memory and computation overhead compared to full model replication.

However, because the frequency of experts calls varies across tasks, load imbalance can occur among devices. DeepSeek-v3 [197] addresses this issue by introducing the redundant experts strategy, which replicates frequently used experts, and by employing Hierarchical Load Balancing and Global Load Balancing mechanisms to maintain balanced workloads even in multi-node environments.

Both vLLM [166] and SGLang [399], TensorRT-LLM [252] experimentally support EP to enhance the efficiency of MoE model inference.

5.2.4 Tensor Parallelism. TP [267, 305], also known as model parallelism or sharding, divides specific LLM operations (e.g., matrix multiplication, attention, fully connected (FC) layers) across multiple hardware devices. Each device processes a slice of the operation, and intermediate results are merged afterward.

For example, Fig. 13 (e) shows a situation in which four GPUs handle the matrix operation $X \times A = Y$. The matrix A is partitioned among the GPUs, either row-wise or column-wise, and the computations are reconciled via collective communication (e.g., All-Reduce or All-Gather).

By distributing large computations, tensor parallelism speeds up inference and reduces the memory footprint of each device, since individual GPUs do not need to store all weights. However, frequent inter-device communication can increase overhead, and suboptimal partitioning may reduce efficiency. Inference engines such as vLLM [166], DeepSpeed-FastGen [129], and TensorRT-LLM [252] often integrate techniques to address these challenges [254, 297, 361].

To mitigate communication bottlenecks in tensor parallelism—particularly the performance degradation observed in TTFT—recent research has proposed communication compression techniques that reduce overhead and enhance inference speed [122].

5.2.5 Pipeline Parallelism. PP [11, 135] assigns different parts (layers) of an LLM to different GPUs. The input data are split into micro-batches which traverse this `pipeline` of layers sequentially. As illustrated in Fig. 13 (f), if a transformer model has four layers and there are four GPUs available, each GPU is responsible for one layer.

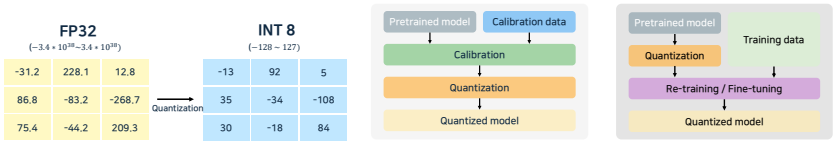
This arrangement can reduce memory usage by distributing layers across devices and can also accelerate inference by overlapping operations. However, communication overhead occurs when intermediate results move between devices, and the initial pipeline stages remain underutilized until the pipeline is `warmed up`. Various pipeline optimization techniques have been proposed to mitigate these concerns [217, 379].

Various inference frameworks support PP, including Ollama [255], llama.cpp [101], vLLM [166], and Friendli Inference [87].

5.3 Compression

As LLMs grow larger, conducting inference on a single GPU or server node becomes increasingly difficult. To mitigate this issue, model compression techniques—such as quantization [76], KD [364], and pruning [158, 407], sparsity optimization [80, 296]—have emerged. Among these, quantization is particularly important for saving memory and increasing inference speed, thereby reducing power consumption and cost. Pruning and sparsity optimization can enhance computational efficiency and inference speed, and several inference engines offer support for these techniques. Although they are closely tied to training or fine-tuning, inference engines must still ensure proper kernel selection and execution when running quantized models. The effectiveness of such compression techniques depends not only on whether they are supported by the inference engine, but ultimately on whether the underlying inference hardware can natively recognize the corresponding data formats and provide optimized kernels and execution paths to process them efficiently.

Just Accepted



(a) Quantization Scheme (b) Post Training Quantization (PTQ) (c) Quantization-Aware Training (QAT)

Fig. 14. Quantization Methods

In edge environments, where constraints on computing power and memory capacity are severe, inference with lightweight models or techniques for model compression is essential. Inference engines designed for edge deployment, such as llama.cpp [101] and bitnet.cpp [218], primarily aim to efficiently execute compressed models, including 1-bit, 4-bit or 8-bit quantized models. Consequently, to accommodate diverse edge hardware platforms and execution environments, inference engines must provide broad support for inference optimization techniques, including quantization, memory management, and computational optimization, at the engine level.

5.3.1 Quantization. **Quantization Algorithm.** Quantization converts pretrained FP32 or FP16 models into lower-precision floating-point formats (e.g., FP4, FP8) or integer formats (e.g., INT4, INT8), as illustrated in Fig. 14 (a). By representing fewer distinct numerical values, quantization can substantially accelerate matrix multiplications and reduce memory requirements with only minor performance trade-offs.

To convert high-precision models into lower-bit representations, methods like Absolute Max Quantization [66] calculate a scale factor from the tensor's absolute maximum value. During dequantization, the model's values are approximated by applying the scale factor to the quantized data. Although this method is straightforward and hardware-efficient, it can be sensitive to outliers. Approaches like Affine Quantization [220] address this issue by adjusting the distribution more flexibly.

Depending on the workflow, several methods can be applied, such as post-training quantization (PTQ) [178, 355], which applies quantization after model training (Fig. 14 (b), or quantization-aware training (QAT) [50, 210], which integrates quantization into the training process (Fig. 14 (c) PTQ can be used with pretrained models, making it straightforward and quick to implement. When applying PTQ, a small calibration dataset of representative sample data is used to refine the quantization parameters in the pretrained model. This dataset helps determine the activation distributions within each layer, which are then used to define the quantization parameters, such as clipping ranges and scale factors. However, it may result in accuracy degradation due to quantization effects. In contrast, QAT incorporates quantization operations into the training process, allowing gradient information to be considered and thereby preserving accuracy more effectively. However, QAT involves additional steps—such as fine-tuning quantization parameters, retraining, and adjusting training strategies—which increase overall training costs [50]. Consequently, even in services where high accuracy is required, PTQ is used more frequently in practice, given these realistic constraints.

A range of quantization methods can be used for LLMs and are supported by various inference engines. Generalized Post-Training Quantization (GPTQ) [85] offers weight-only quantization, optimizing per-layer scales with error compensation to minimize accuracy degradation. Activation-Aware Weight Quantization (AWQ) [195] improves weight quantization by grouping weights according to activation distributions, increasing accuracy.

Additive Quantization of Language Models (AQLM) [77] quantizes both weights and activations using PTQ only, avoiding QAT and achieving high performance with lower overhead than GPTQ. SmoothQuant [355] normalizes activation and weight distributions to reduce clipping during quantization, leading to stable PTQ-based activation quantization while reducing latency and memory usage.

Just Accepted

Table 11. Data Type Support in LLM Inference Engines

Engines	Data Type											
	FP32	FP16	FP8	FP4	NF4	BF16	INT8	INT4	MXFP8	MXFP6	MXFP4	MXINT8
Ollama [255]	✔	✔	✔	✘	✘	✔	✔	✘	✘	✘	✘	✘
llama.cpp [101]	✔	✔	✘	✘	✘	✘	✔	✔	✘	✘	✘	✘
vLLM [166]	✔	✔	✔	✔	✔	✔	✔	✔	✘	✘	✘	✘
DeepSpeed-FastGen [129]	✔	✔	✘	✘	✘	✘	✔	✔	✘	✘	✘	✘
Unsloth [325]	✔	✔	✔	✘	✔	✔	✔	✔	✘	✘	✘	✘
MAX [237]	✔	✔	✔	✘	✘	✔	✔	✔	✘	✘	✘	✘
MLC LLM [234]	✔	✔	✔	✘	✘	✘	✔	✔	✘	✘	✘	✘
llama2.c [24]	✔	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘
bitnet.cpp [336]	✔	✔	✘	✘	✘	✔	✔	✘	✘	✘	✘	✘
SGLang [399]	✔	✔	✔	✘	✔	✔	✔	✔	✘	✘	✘	✘
LitGPT [194]	✔	✔	✘	✔	✔	✘	✔	✘	✘	✘	✘	✘
OpenLLM [37]	✔	✔	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘
TensorRT-LLM [252]	✔	✔	✔	✘	✘	✔	✔	✔	✔	✘	✔	✘
TGI [139]	✔	✔	✔	✔	✔	✔	✘	✘	✘	✘	✘	✘
PowerInfer [302]	✔	✔	✘	✘	✘	✔	✔	✔	✘	✘	✘	✘
LMDeploy [213]	✔	✔	✔	✘	✘	✔	✔	✔	✘	✘	✘	✘
LightLLM [191]	✔	✔	✘	✘	✘	✔	✔	✘	✘	✘	✘	✘
NanoFlow [405]	✘	✔	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘
DistServe [402]	✔	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘	✘
vAttention [268]	✔	✔	✔	✘	✘	✔	✔	✔	✘	✘	✘	✘
Sarathi-Serve [10]	✔	✔	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘
Friendli Inference [87]	✔	✔	✔	✘	✘	✔	✔	✔	✘	✘	✘	✘
Fireworks AI [83]	✘	✔	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘
GroqCloud [111]	✔	✔	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘
Together Inference [318]	✘	✔	✔	✘	✘	✘	✘	✔	✘	✘	✘	✘

Additionally, **KV** cache quantization [131, 212] has been proposed to minimize memory usage in long-context scenarios, allowing a balance between memory efficiency and generation speed while minimizing the impact on model quality. KVQuant [131] applies ultra-low precision quantization with minimal accuracy drop using Pre-Channel and Pre-RoPE key quantization, non-uniform **KV** cache quantization, and Per-Vector Dense-and-Sparse Quantization. KIVI [212] quantizes key caches per channel and value caches per token, achieving 2-bit quantization.

Kernel Code and Hardware Support. Many inference engines integrate external quantization tools. bitsandbytes [40] is a CUDA-based Python library that supports 8-bit and 4-bit quantization; it supported in engines such as vLLM [166] and SGLang [399], LitGPT [194]. DeepSpeed FP is DeepSpeed's library for 6-bit and 8-bit weight-only quantization and is partially supported in vLLM [166] on NVIDIA GPUs.

ExLlamaV2 (EXL2) [323] is an inference library for consumer GPUs, offering flexible 2-bit to 8-bit quantization similar to GPTQ, along with the option to mix quantization at different bit-levels per layer. EETQ [242] provides a straightforward and efficient approach to INT8 weight-only PTQ for transformer models. Both EXL2 and EETQ are supported in TGI [139].

LLM Compressor [329] is a quantization library designed for the vLLM environment, allowing both weight-only and activation quantization. It supports mixed-precision modes (e.g., W4A16, W8A16) and integrates techniques such as simple PTQ, GPTQ [85], and SmoothQuant [355]. Inference engines such as vLLM [166] and SGLang [399] can employ LLM Compressor for quantization.

Mixed Auto-Regressive Linear Kernel (Marlin) [86] is a highly optimized kernel for FP16×INT4 matrix multiplication. Designed to maximize inference speed, it can theoretically deliver up to four times the performance of FP16 by fully utilizing GPU global memory, cache, shared memory, and tensor cores. Implemented at the NVIDIA Parallel Thread Execution (PTX) [246] assembly level, Marlin depends on NVIDIA GPUs and is supported by vLLM [166], SGLang [399], and TGI [139].

Just Accepted

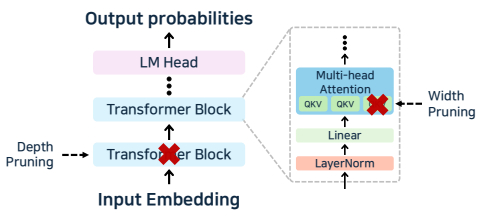


Fig. 15. Pruning in Transformer-based Model

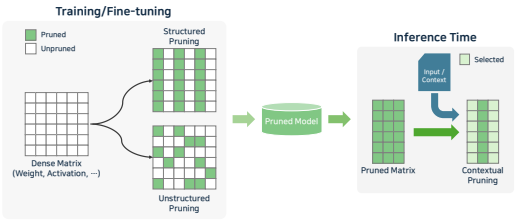


Fig. 16. Three Types of Pruning: Structured, Unstructured, and Contextual

These quantization techniques are closely connected to the hardware supported by each inference engine. As indicated in Table 11, every engine accommodates specific data types, which in turn govern how quantized models are executed.

In particular, data types based on block-level scaling in the 4-8-bit range—such as the microscaling (MX) format [279]—have been proposed to balance training and inference performance, accuracy, and framework compatibility. Examples include MXFP8 and MXINT8. Each MX block consists of a single scale value X and a set of compressed values $(P_1, P_2, \ldots, P_k)$. The scale format (e.g., E8M0) and element format (e.g., FP4, FP6, FP8, INT8) can be independently configured.

Unlike traditional FP8 or INT8 formats, which require a single tensor-level scaling factor to match the dynamic range of the entire tensor, MX formats split the tensor into smaller sub-blocks and assign separate scale values to each, thereby circumventing the limitations of sub-8-bit formats. Hardware platforms such as the Qualcomm Cloud AI 100 [48] and NVIDIA GPUs based on the Hopper architecture support MX formats, and some inference engines (e.g., TensorRT-LLM [252]) offer support for them.

Support in Inference Engines. Most inference engines either support the execution of quantized models or provide built-in functionality to quantize models directly. Ollama [255] supports quantization of FP16 and FP32 models to 4-bit to 8-bit precision and provides compatibility with GGUF-based quantized models and adapters. vLLM [166] supports both weight-only quantization and weight-activation quantization, and further offers quantization schemes optimized for specific hardware platforms, such as Intel Neural Compressor [143] and AMD Quark [20].

DeepSpeed [371] supports QAT-based quantization [50] through techniques such as Mixture-of-Quantization (MoQ), which gradually adapts model precision during training rather than reducing it in a single step. This approach adjusts the rate of precision reduction based on layer-wise sensitivity. Unsloth [325] provides QAT support based on PyTorch TorchAO [319], while MAX [237] offers low-precision formats, including 4-bit quantization, through its graph-level quantization classes.

SGLang [399] supports offline quantization using tools such as Unsloth [325] or LLM Compressor [329], as well as FP8 online quantization based on AWQ [195] and GPTQ [85] algorithms. LMDeploy [213] supports both weight-only quantization and **KV**-cache quantization, and llama.cpp [101] provides a wide range of quantization options from 1-bit to 8-bit precision for BF16 and FP32 models. In addition, several other inference engines, including MLC-LLM [234], TGI [139], and TensorRT-LLM [252], support either weight-only or weight-activation quantization. The specific scope and mechanisms of quantization vary across engines depending on their design goals and target hardware platforms.

5.3.2 Pruning. **Overview and Inference Benefits of Pruning.** Pruning [158, 407] is a model compression technique that removes less important parameters to reduce model size, often targeting weights or attention heads, as shown in Fig. 15 in Transformer-based LLMs. It can be applied after training by zeroing out and

Just Accepted

removing certain weights, applied dynamically during training/fine-tuning [209], or performed through one-shot pruning [84, 286] to rapidly shrink large models.

From an inference perspective, pruning reduces the number of parameters, thereby improving memory utilization, bandwidth efficiency, and cache usage. As more weights become zero, sparse computation can reduce actual computation costs. However, to fully exploit sparse computation, the inference engine or compute library must support kernels capable of skipping or efficiently handling zero weights.

Three Types of Pruning. Pruning methods in LLMs generally fall into three categories: structured pruning, unstructured pruning, and contextual pruning [32], and each pruning method is illustrated in Fig. 16. Structured pruning eliminates groups of parameters with fixed structures, such as convolutional filters or neuron channels. Since the matrix dimensions are physically reduced, the inference speed can be improved even without specialized sparse computation kernels. However, inference graphs or kernels must be adjusted to match the pruned architecture. One of the structured pruning methods is LLM-Pruner [219], which prunes low-importance structures based on gradient information.

Unstructured pruning removes individual weights according to their importance scores. Although this reduces model size and FLOPS, the use of random sparsity patterns can limit performance gains on dense matrix multiplication kernels, requiring optimized sparse kernels for effective acceleration. For example, NVIDIA CUDA provides sparse operations via cuSPARSE [251]. Among the unstructured pruning methods, Wanda [309] prunes weights based on the product of weight magnitude and input activation.

Contextual pruning dynamically assesses the importance of weights depending on the input context or domain, selectively removing or retaining weights. This approach adapts the model to specific inputs, skips unnecessary computation paths, and improving inference efficiency. Although sparse computation may yield smaller performance gains compared to other methods, it can enhance domain-specific accuracy. Implementing contextual pruning requires inference engines to support conditional branching or precompiled kernels with logic for bypassing certain layers. For example, Mini-GPTs [326] applied contextual pruning to Phi-1.5 [180] and Opt-1.3 [386] using legal and medical QA datasets to prune linear, activation, and embedding layers.

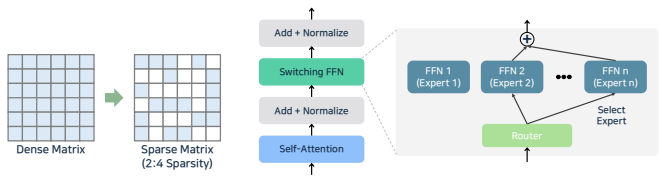
Recent Advances: Post-Training and Token Pruning. Like quantization, post-training pruning has been a research focus for LLMs, given the high cost of training or fine-tuning. Recent efforts involve unstructured and semi-structured post-training pruning algorithms to address the Multiple Removal Problem (MRP) by pruning large quantities of weights at the LLM layer level [394].

In LLM inference, as the input token length increases, the TTFT generally increases as well. One proposed solution is token pruning [89], which selectively computes **KV** representations solely for tokens judged important for next-token prediction—without requiring extra training or fine-tuning. The remaining tokens are deferred and only computed if needed, reducing initial computation costs and improving TTFT.

Engine-Level Support for Pruned Models. Among the inference engines discussed in this paper, fewer than half directly support pruning. Most rely on NVIDIA pruning libraries, with DeepSpeed-FastGen [129] explicitly supporting row, head, sparse, and structured/unstructured pruning through the DeepSpeed backend. Other engines generally only support running pre-pruned models.

5.3.3 Sparsity Optimization. **Sparsity Optimization Overview.** Sparsity optimization [80, 296] is a technique that reduces computational costs and speeds up inference by increasing the number of zero values in model weights or activations. This approach can reduce memory usage and improve compute performance when supported by hardware or inference engines that allow sparse operations. Although it shares the same goal as pruning, sparsity optimization focuses on designs sparse model structures or applies predefined sparse patterns to achieve computational efficiency. Sparsity can be applied to the attention mechanism (e.g., sparse attention patterns) or to the weights of individual heads. Pruning can induce sparsity and models that are already sparse

Just Accepted



(a) Structured Sparsity - N:M sparsity

(b) Dynamic Sparsity - MoE

Fig. 17. Sparsity Optimizations

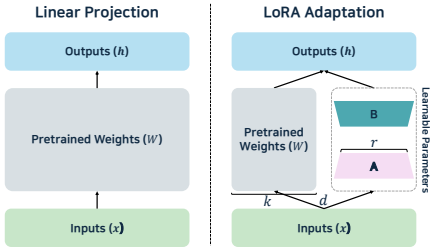


Fig. 18. LoRA

can be further refined through pruning. Sparsity optimization techniques include structured sparsity [70, 398], dynamic sparsity [390], and kernel-level sparsity [41, 353].

Structured Sparsity. Structured sparsity [70, 398] imposes sparsity on weights or tensor values in fixed patterns, simplifying hardware-level optimizations. Typical examples are N:M sparsity [389] as shown in Fig. 17 (a), where n values within an m-sized block remain active; and block sparsity [93] which divides weight matrices into blocks and removes values to create sparsity. On NVIDIA GPUs and similar hardware, these static patterns can be optimized at the time of model compilation. However, rigid patterns may negatively affect model performance.

Dynamic Sparsity and Mixture-of-Experts. Dynamic sparsity [390] activates only the computations required at run time based on input tokens, skipping unnecessary operations to improve efficiency. A prominent example is MoE [46], which replaces the MLP with multiple FFNs (experts) but activates only a subset of them, depending on the input tokens, as shown in Fig. 17 (b). This reduces the number of computations per token and enables large models to run more efficiently. To support MoE, the inference engine must offer gating mechanisms and flexible architectures for dynamic routing. Example MoE models include Mixtral 8x7B [149], DeepSpeed-MoE [277], and DeepSeek-R1 [120], which allow more tokens to be processed or trained with limited time or resources. Most inference engines provide MoE support and related optimizations.

Enhancements to Sparse MoE Training. In dense expert models, all experts are activated for every input, leading to a more complicated computation. Sparse MoE [74, 81] complements this by using only some of them, such as selecting only top-k experts. To address reduced specialization or unstable training due to sparse-gating, SMoE-Dropout [52] employs randomized router networks that gradually increase the number of active experts throughout training to refine the model.

Token and Contextual Sparsity. Another example of dynamic sparsity is dynamic token sparsity [89, 369] which avoids computing attention for all tokens by focusing on a subset. In addition to contextual pruning [326], contextual sparsity [14, 211] has also been proposed, which selectively activates only a subset of attention heads or MLP parameters depending on the input. In contextual sparsity research such as Deja Vu [211], a lightweight sparsity method was employed to dynamically skip computations based on the input context, addressing the high cost of verifying whether contextual sparsity truly exists for each input.

Kernel-Level Sparsity. Kernel-level sparsity checks for zero values in the computation kernels and skips them. For example, sparse matrix-dense matrix multiplication (SpMM) [41] kernels or rely on libraries like cuTeSpMM [354] to utilize NVIDIA GPU Tensor Cores. The xFormers [228] library also includes CUDA kernels for memory-efficient attention, sparse attention, and block-sparse attention.

Support in Inference Engines. Several engines already integrate such methods. vLLM [166], SGLang [399] and TGI [139] support N:M sparsity, with vLLM also offering block sparsity. SGLang applies Double Sparse Attention [369], a post-training sparse attention approach that prioritizes essential tokens (token sparsity) in self-attention while determining important feature channels offline (channel sparsity). DeepSpeed-FastGen [129]

Just Accepted

adopts the Sparse Attention technique to introduce block-level sparsity in self-attention, reducing compute and memory usage through a variety of patterns and custom modifications. TensorRT-LLM [252] employs Block Sparse Attention to accelerate SpMM through block-structured sparsity, relying on the Sparse Tensor Cores featured in NVIDIA GPUs from the Ampere architecture onward.

5.4 Inference-Aware Fine-tuning

LLMs typically rely on foundation models pretrained on large-scale datasets to perform diverse inference tasks. For example, Llama 4 Behemoth [227], which contains approximately 2 trillion parameters, was trained using 32,000 GPUs (NVIDIA H100) at FP8 precision. Training such ultra-large-scale models requires enormous hardware resources and massive training datasets, making it impractical for individual organizations to train models from scratch. As a result, a common approach is to adopt foundation models that have already been trained with large-scale resources and then apply quantization [76] or fine-tuning [337] to customize them for specific service requirements.

Recently, with growing demand for personalized and service-oriented models as well as on-device inference, there has been increasing interest in improving both performance and efficiency by supporting model fine-tuning at the inference engine level. For example, training a 22B-parameter model requires approximately 308 GB of memory when accounting for parameters, gradients, and optimizer states [65]. In contrast, by applying low-precision training techniques such as 4-bit Quantized LoRA (LoRA) [67], even a 34B-parameter model can be fine-tuned using only about 24 GB of memory [348]. This indicates that fine-tuning large-scale models is becoming feasible on consumer-grade GPUs with under 32 GB memory.

In real-world services, models require continuous updates even after deployment to incorporate domain-specific knowledge, introduce new functionalities, and reflect user feedback. If an inference engine cannot promptly integrate the latest fine-tuning results, improved models may fail to be deployed in production environments, leading to degraded performance, poorer user experience, and ultimately increased operational costs. Consequently, support for fine-tuning within inference engines has emerged as an important consideration when selecting and designing LLM serving systems.

In addition to large-scale training frameworks targeting massive clusters, such as TorchTitan [186] and NVIDIA Megatron-LM [297], fine-tuning-focused tools like Axolotl [31] and Torchtune [229] are also rapidly evolving. These frameworks support a variety of post-training methods on pretrained models, including full fine-tuning [216], LoRA [133], and GPTQ [85]. Many of them either provide built-in inference modules or leverage inference engines such as SGLang [399] and vLLM [166] as backends, enabling fine-tuned models to be served immediately. As a result, in service environments, inference engines must integrate seamlessly with these fine-tuning frameworks to allow continuous deployment of model updates and maintain up-to-date performance without service interruption.

Parameter-Efficient Fine-Tuning. Fine-tuning is a technique originally applied in CNN that modifies the parameters of a pre-trained model. It can be divided into full-parameter fine-tuning [216], where every parameter is updated, and parameter-efficient fine-tuning (PEFT) [68], where only a subset of parameters is adjusted. Because full fine-tuning requires substantial hardware resources, LLMs generally favor PEFT methods that update only part of the model. Although fine-tuning mainly targets model training, it directly affects LLM inference performance as well. Fine-tuning can be implemented by inserting adapter networks (additional layers) to recalibrate parameters [136] or by supplying domain-specific data through prompt engineering [374].

Low-rank adaptation. LoRA [133, 290] is a representative PEFT approach. Instead of updating the entire model, LoRA keeps the original weights frozen and trains additional low-rank matrices to adjust the model parameters. Research indicates that despite the large dimensions of the FC layers in LLMs, the effective dimensionality required for adaptation is relatively low. LoRA exploits this by approximating weight updates with low-rank

Just Accepted

matrices, greatly reducing the training cost. As shown in Fig. 18, LoRA retains the pre-trained weight matrix and trains only two small matrices, A and B. Rather than directly computing updates for the full weight matrix ($d \times k$, d : input dimension, k : output dimension), LoRA approximates $A \times B$ by low rank (r) matrix multiplication ($d \times r$ and $r \times k$).

A major advantage of LoRA is the ability to swap in different LoRA modules for varying tasks, facilitating quick adaptation without retraining the entire model. Moreover, since LoRA only updates the small low-rank matrices, training is faster and demands less memory. Merging these trained modules with the original weights generally does not degrade inference speed, although merging may limit the capacity for multi-task processing in real time.

Quantized Low-rank adaptation. As LLM sizes increase, additional methods like QLoRA [67, 387] have been introduced, combining 4-bit quantization with LoRA fine-tuning. QLoRA backpropagates through a 4-bit quantized model, preserving LoRA's benefits while reducing memory usage. This enables running large models on a single device, offering efficient options for both training and inference.

Support in Inference Engines. Ollama [255], llama.cpp [101], Friendli Inference [87], and Fireworks AI [83] support LoRA, morevoer DeepSpeed-FastGen [129], SGLang [399], TensorRT-LLM [252], Unsloth [325], and vLLM [166] support QLoRA. Especially, vLLM [166], TensorRT-LLM [252], TGI [139], LMDeploy [213], Friendli Inference [87], and Together Inference [318] also provide Multi-LoRA functionality, enabling simultaneous serving of multiple user-customized models.

In particular, Unsloth [325] is designed to enable direct fine-tuning of LLMs. Instead of relying on PyTorch's automatic differentiation, Unsloth manually derives gradients for core operations and restructures the order of matrix multiplications to exploit the structural properties of LoRA layers. It further rewrites all major kernels using Triton [317], optimizing the entire training path. Owing to this redesign, Unsloth achieves several-fold to orders-of-magnitude faster training speed on the same GPU hardware, while reducing GPU memory consumption by approximately 50 percent. As a result, it enables fine-grained control over model tone, style, and behavior, as well as efficient injection of domain-specific knowledge, facilitating the construction of customized inference pipelines tailored to specific service or domain requirements.

5.5 Caching

In LLMs containing billions to trillions of parameters, repeatedly generating hundreds to millions of tokens requires substantial computation and memory resources. To address this, most inference engines employ various caching strategies that reduce redundant computations and lower latency. Caching can be applied to several LLM components, and different caching optimizations can be combined and used together.

Recently, approaches such as retrieval-augmented generation (RAG) [174], which improve LLM accuracy by referencing external trusted technical documents or knowledge bases, have become increasingly prevalent. As these approaches further evolve into multi-agent workflows [124], the importance of caching mechanisms has grown substantially. In multi-agent settings, a single user request is often addressed by multiple agents, each of which repeatedly invokes an LLM. As a result, the prefill phase, which processes the initial portion of the prompt, may be redundantly executed for each invocation. This leads to repeated processing of identical or highly similar prompts, causing prefill duplication, which in turn becomes a major contributor to increased end-to-end latency and inference cost.

To mitigate these issues, prefix caching [202] or prompt caching [102] can be applied. By reusing the computation results of previously processed prompt segments, unnecessary prefill operations can be eliminated, thereby reducing both latency and resource consumption. Furthermore, in long-horizon environments [311] where multiple agents operate collaboratively over extended periods, an important challenge emerges regarding how **KV** caches used by different agents should be shared and managed [365]. Consequently, in multi-agent

Just Accepted

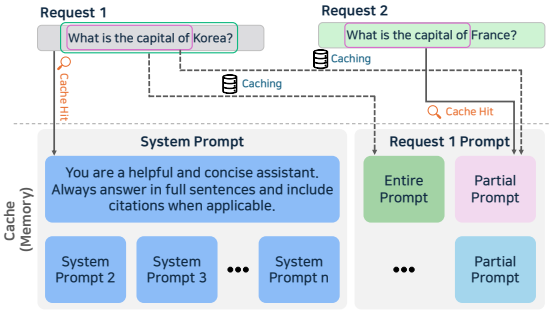


Fig. 19. Prompt Caching

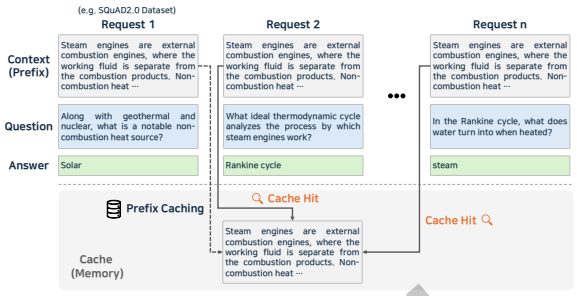


Fig. 20. Prefix Caching

inference scenarios, efficient **KV** cache management and optimization mechanisms become critical factors that determine overall system performance and scalability.

These caching techniques involve different trade-offs depending on the granularity of cache sharing and the mechanisms used to ensure consistency. Shared prefix caching offers substantial performance benefits by enabling reuse of identical prompt prefixes across multiple requests or agents. However, in multi-agent environments where agents may have distinct roles or privileges, cache sharing can introduce reliability and governance concerns [124]. In particular, there is a risk that policies or access controls may be inadvertently leaked through shared caches, potentially undermining isolation guarantees among agents.

In addition, prompt caching and semantic caching [238] can improve efficiency by reusing computations for similar queries. However, when queries are not exactly identical, such reuse may lead to degradation in response quality. Furthermore, if the **KV** cache capacity is insufficient, older information may be evicted or critical context may not be retained, potentially resulting in accuracy loss [388]. Therefore, these caching techniques should not be treated merely as static performance optimizations; instead, they must be dynamically managed in accordance with SLOs. Therefore, this requires inference engine-level support to control and coordinate caching behavior across the inference and serving pipeline.

5.5.1 Prompt Caching. A large portion of LLM prompts may include frequently reused text. Identical content, such as system messages or common instructions, often appears multiple times—particularly in conversational agents, coding assistants, or extensive document processing. To optimize this, a technique known as Prompt Cache [102, 404] has been introduced. The Prompt Cache stores attention states for frequently used text segments in advance, and when the same segment reappears in a prompt, it reuses those stored attention results and only computes the new segments, thereby expediting inference. However, because the transformer architecture applies positional encoding, the attention states depend on the position of each segment, meaning that reuse is only possible if the segment appears in the same position. To overcome this limitation, Prompt Markup Language (PML) was proposed. It explicitly defines a prompt's structure, identifies reusable segments (prompt modules), and assigns unique position IDs to each module. PML functions as a schema for module positions and hierarchy, offering an interface for generating and reusing attention states at the module level. An example of prompt caching is shown in Fig. 19.

Prompt Caching in Commercial AI Services. Commercial services such as ChatGPT [257] and Claude [26] also employ prompt caching. ChatGPT routes API requests with the same prompt to the same server that previously handled them, allowing reuse of cached results instead of recomputing from the first token. This approach has reduced latency by as much as 80% for long prompts. To increase cache hit rates, ChatGPT places static components (e.g., instructions, examples) at the beginning of the prompt, while dynamic user content goes

Just Accepted

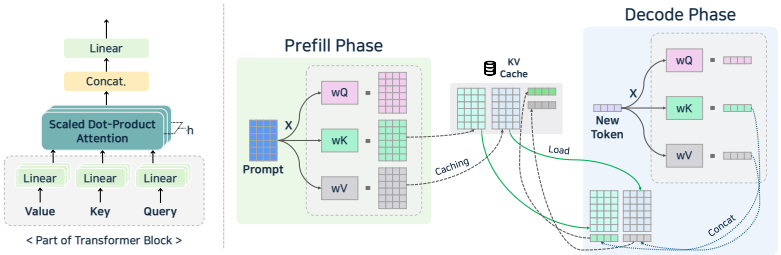


Fig. 21. KV Caching

at the end. ChatGPT applies caching to prompts exceeding 1,024 tokens and triggers cache hits in 128-token segments. It retains cached prompts for five to sixty minutes, depending on system load, automatically evicting any prompts that remain unused. Claude employs a similar format, providing up to four cache breakpoints that split prompts into multiple cacheable segments.

Cache Hit Prediction. A method has also been proposed to improve the accuracy of prompt caching [404]. This study recommends predicting the effectiveness of caching based on embedding similarity. In single-turn question-answer scenarios, the study uses embeddings refined through knowledge distillation to determine whether cached responses can be reused. Cosine similarity is computed between prompt embeddings, and a model is trained to decide if the same response can be reapplied. This research also presents finite sample guarantees for loss functions like Binary Cross Entropy (BCE) and Squared Log Difference (SLD).

Support in Inference Engines. Several inference engines, including Ollama [255], llama.cpp [101] and TensorRT-LLM [252], support prompt caching. TensorRT-LLM [252] provides system prompt caching, while Ollama [255] offers optimized prompt caching even in multi-user environments.

5.5.2 Prefix Caching. Prefix Caching [202, 261] is conceptually similar to prompt caching [102, 404] but focuses on caching only the common prefix segments that reappear across multiple requests, rather than caching the entire prompt, as shown in Fig. 20. During batched inference, when multiple prompts share the same prefix, the computation for that shared segment can be reused to improve efficiency. For instance, in question-answering tasks where the same system prompt or few-shot examples are repeatedly used, caching these portions of the prompt during the prefill phase can reduce overall inference time.

However, prefix caching typically accelerates only the prefill phase and does not affect the decode phase. Consequently, if the primary bottleneck stems from extended decoding for very long responses, the performance gains from prefix caching may be limited. In addition, if a new request does not share a prefix with any existing request, the caching advantage diminishes.

Support in Inference Engines. vLLM [166] provides Automatic Prefix Caching (APC), which stores the **KV** cache from previous requests and reuses it whenever a new request shares a prefix with an existing one, thus skipping attention computations for the shared segments. TGI [139] employs high-performance data structures instead of basic string matching to speed up prefix lookups and applies chunking code to optimize memory usage; it also integrates prefix caching with Flashdecoding [130] kernels to support rapid inference for lengthy sequences. MAX [237] employs a PagedAttention [166]-based mechanism to apply prefix caching and improve inference efficiency, and other engines, such as LMDeploy [213], also include prefix caching features.

5.5.3 KV Caching. In self-attention of the Transformer, each token attends to all preceding tokens, resulting in a time complexity of $\mathcal{O}(n^2)$. To reduce this overhead, **KV** Caching [94, 266, 343] was proposed. By storing the K and V matrices produced in each token step and reusing them for subsequent tokens, inference can run in $\mathcal{O}(n)$

Just Accepted

time. The **KV** Cache operation in the prefill and decode phases is as shown in Fig. 21. This provides significant efficiency gains when large batch size processing or multi-turn conversation scenarios. **KV** caching also works well alongside other optimizations such as Prefix Caching [202, 261] and Speculative Decoding [173, 204, 303].

Increasing KV Cache Size. However, the memory needed for **KV** caching increases significantly with longer contexts because both **K** and **V** must be stored in memory. The memory needed for LLM inference can be expressed as $S_{KV}/\text{token} = 2 \times n_{\text{layers}} \times (n_{\text{heads}} \times d_{\text{head}}) \times precision$ and the total size of KV Cache can be expressed as $\Sigma\ S_{KV} = s_{\text{batch}} \times l_{\text{seq}} \times 2 \times n_{\text{layers}} \times s_{\text{hidden}} \times precision$.

In the equation, n_{layers} represents the number of layers, n_{heads} represents the number of attention heads, and d_{head} represents the dimension of the heads. s_{batch} is the batch size, l_{seq} is the sequence length, s_{hidden} is the hidden size, *precision* is precision bytes such as FP16, and each KV cache size (S_{KV}, $\Sigma\ S_{KV}$ is in bytes. Multiplication by 2 in the equation accounts for both the **K** and **V** components.

Reducing batch size to decrease **KV** cache memory lowers memory usage but also reduces hardware utilization and throughput. Limiting the sequence length forces the recomputation of some **KV** caches, causing inefficiency. Reducing the depth of the model may compromise performance; hence, current research focuses on attention mechanisms, cache compression, and quantization techniques.

KV Cache Optimizations. As shown in Fig. 3, attention mechanisms like GQA [13] and MQA [288] reduce the number of **Q** heads or reuse **KV** heads, naturally shrinking **KV** cache size. Research also focuses on compressing **KV** caches. MiniCache [198] uses depth-wise compression, observing that middle-to-deep layer **KV** caches are highly similar. H2O [391] leverages sparsity of the attention matrix to reuse only essential parts while retaining the same output tokens. FlexGen [292] profiles the structural properties of the attention heads and proposes adaptive **KV** cache compression. As explained in Section 5.3.1 above, quantization can also be applied to reduce the **KV** cache size [131, 212].

As context windows and model sizes grow, GPU memory alone can become insufficient to store **KV** caches, leading to offloading solutions. InfiniGen [171] prefetches necessary **KV** segments into CPU memory speculatively, while CacheGen [208] encodes and streams **KV** caches as compressed bitstreams in distributed systems, reducing network transfer latency.

Support in Inference Engines. Many inference engines implement **KV** caching. LMDeploy [213] and Ollama [255] use INT8 or FP16 quantization to reduce the cache size. DeepSpeed-FastGen [129] applies ZeRO-Inference [21]-based offloading and 4-bit quantization to increase throughput in resource-constrained settings. TensorRT-LLM [252] offers various optimizations, including a paged **KV** cache, **KV** cache quantization, a circular **KV** cache, and **KV** cache reuse. It saves memory using an LRU-based eviction strategy and **KV**-aware routing or scheduling that forwards requests to instances with the required **KV** cache already in place. vLLM [166] implements **KV** Cache Preemption, allowing some requests to be preempted to free up space. Although preempted requests required recomputation and may increase end-to-end latency, this approach can enhance overall system stability.

5.6 Attention Optimization

Attention lies at the core of transformer-based LLMs, but its high memory and computational costs make optimization essential for efficient service deployment. As the sequence length grows, the time required for attention operations rises quadratically, and calculating and storing **Q**, **K** and **V** matrices consumes substantial memory. Increasing memory efficiency and batch size is therefore, critical for managing multiple requests and boosting overall throughput.

To improve inference performance, most LLM inference engines utilize a **KV** Cache mechanism [208, 306] that stores the **K** and **V** vectors from previously generated tokens. This approach prevents redundant computations when producing subsequent tokens, thereby reducing both inference latency and computational overhead.

Just Accepted

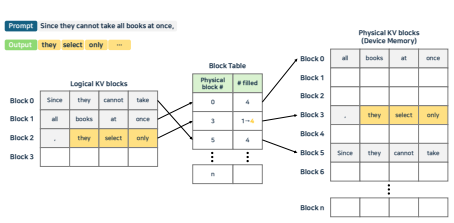


Fig. 22. PagedAttention

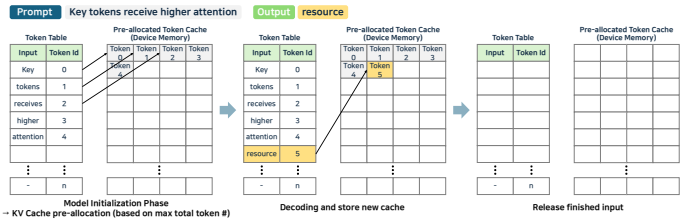


Fig. 23. TokenAttention

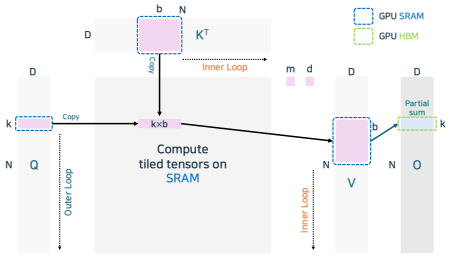


Fig. 24. FlashAttention

Algorithm 1 Simple FlashAttention

for $k \leftarrow 1$ to N **do**
  for $i \leftarrow 1$ to #tiles **do**
    $x_i \leftarrow Q[k, :]K^\top[:, (i-1)b : ib]$ ▷ Copy to SRAM
    $m_i^{(\text{partial})} \leftarrow \max_{j=1}^{b}(x_i[j])$ ▷ Compute on SRAM
    $m_i \leftarrow \max(m_{i-1}, m_i^{(\text{partial})})$ ▷ Compute on SRAM
    $d_i' \leftarrow d_{i-1}' e^{m_{i-1}-m_i} + \sum_{j=1}^{b} e^{x_i[j]-m_i}$ ▷ Compute on SRAM
    $o_i' \leftarrow o_{i-1}' \frac{d_{i-1}'}{d_i'} e^{m_{i-1}-m_i} + \sum_{j=1}^{b} \frac{e^{x_i[j]-m_i}}{d_i'} V[j+(i-1)b, :]$ ▷ Compute on SRAM
  end for
  $O[k, :] \leftarrow o_i'$ ▷ Copy to HBM
end for

LLM inference engines offer a range of optimization techniques for efficiently storing and retrieving the **KV** cache, as well as methods to improve the use of **Q**, **K** and **V** vectors are utilized during inference.

5.6.1 KV Cache Optimization: PagedAttention. Effective **KV** cache management is crucial for enhancing LLM inference performance. Conventional inference engines often allocate a contiguous **KV** cache based on the maximum sequence length, which can lead to internal and external fragmentation as sequence lengths vary. These fragmentation issues reduce memory efficiency and limit parallelism.

PagedAttention [166] addresses this problem by adopting a Linux-style paging mechanism, as shown in Fig. 22. It partitions the **KV** cache into smaller pages and uses a page table to map logical blocks to physical blocks. Newly needed blocks are allocated on demand, and memory from completed requests is quickly reclaimed for new requests. Requests with identical prompts share the same **KV** cache block, further saving memory. Various inference engines, such as DeepSpeed-FastGen [129] and vLLM [166], MAX [237], SGLang [399] integrate PagedAttention to improve inference efficiency.

LightLLM [191] introduces TokenAttention, which manages **KV** cache at the token level (Fig. 23). Instead of a page table, TokenAttention uses a token table to track each token's actual storage location. It preallocates **KV** cache based on a user-defined maximum token limit and assigns continuous memory regions to new requests. This strategy minimizes fragmentation and improves resource utilization through fine-grained memory management.

Other approaches, such as ChunkedAttention [373], reduce **KV** cache duplication by recognizing that system prompts often repeat. ChunkedAttention splits the *KV* cache into smaller chunks and organizes them with a Prefix-Aware **KV** Cache (PAKV) structure, enabling requests with the same prefix to share cache blocks. This further enhances memory efficiency.

5.6.2 I/O Optimization: FlashAttention. During LLM inference, attention requires $\mathcal{O}(n^2)$ computations for a sequence of length n. This involves forming a score matrix through the dot product of **Q**, **K**, and **V**, which is highly memory-intensive due to frequent data transfers between memory hierarchies on GPUs.

Just Accepted

FlashAttention [64] reduces unnecessary data transfers by splitting **Q**, **K**, and **V** into smaller blocks. Unlike approaches that compute the entire attention matrix before applying softmax, FlashAttention applies an online softmax per tile, avoiding redundant writes to memory. It also fuses matrix multiplication and softmax into a single pass, thereby decreasing kernel invocation overhead.

In order to illustrate the core concept of attention operation fusion in FlashAttention, we present the essential idea in a simplified manner, as shown in Algorithm 1 and Fig. 24. In fused attention, only a small subset of intermediate results is maintained on-chip at each step, which enables memory-efficient self-attention. This design scales linearly with the sequence length N while respecting GPU shared memory limits.

We define b as the block size (also referred to as the tile width). The total number of tiles along the sequence dimension is given by $\#\text{tiles} = \left\lceil \frac{N}{b} \right\rceil$. The term $x_i \in \mathbb{R}^b$ denotes the pre-softmax logits for tile i. We let m_i be the global maximum value over all tiles from 1 to i, and $m_i^{(\text{partial})}$ represent the maximum value within the partial tile i. The variable d_i is the cumulative denominator for the softmax computation up to tile i. Finally, o_i is the partial output vector (corresponding to $O[k, :]$) that accumulates results up to tile i. Each iteration computes the local logits x_i for the current tile and updates the softmax scaling factor m_i, cumulative denominator d_i, and partial output o_i.

Because x_i, m_i, d_i, and o_i have small and fixed sizes ($\mathcal{O}(b)$ or $\mathcal{O}(D)$), they can reside in shared memory during kernel execution. This formulation ensures that the computation is both numerically stable and compatible with parallel tiling, making it ideal for long-sequence Transformer inference or training.

FlashAttention-2 [63] further optimizes GEMM and related non-matrix operations. It merges certain scaling steps and allows parallelization along the sequence length dimension, which is beneficial when the batch size and number of attention heads are small. This strategy maximizes GPU utilization for long sequences, but initially suffered from low GPU usage in GEMM on NVIDIA H100 GPUs.

To address that, FlashAttention-3 [285] introduces asynchronous computation and low-precision arithmetic. By splitting data transfer and computation into separate GPU warps and using a producer-consumer model, it overlaps softmax operations with Warp Group Matrix Multiply-Accumulate (WGMMA), reducing latency.

Many LLM inference engines now include support for FlashAttention variants, with vLLM [166] compatible up to FlashAttention-3 [285]. Because these optimizations are closely related to NVIDIA GPU architectures, researchers are exploring ways to adapt similar principles for other hardware [42, 130], aiming to generalize attention acceleration across diverse platforms.

5.6.3 KV Cache Reuse: RadixAttention. RadixAttention is an optimization technique proposed by SGLang [399] that enables the automatic reuse of the **KV** cache across multiple operations. Traditional inference engines flush all related **KV** caches once a request finishes, which prevents reuse between requests and slows performance. To address this limitation, SGLang manages **KV** caches with a radix tree-based LRU mechanism that allows fast matching, insertion, and deletion, and it applies cache-aware scheduling to handle diverse reuse patterns efficiently. The approach is compatible with continuous batching [380], PagedAttention [166], and tensor parallelism, and it adds only minimal time and memory overhead, even for cache misses.

RadixAttention maps token sequences to their corresponding **KV** cache tensors through a radix tree, while the cache itself is stored in a non-contiguous, page-based memory layout. The tree resides in CPU memory and incurs little maintenance cost. When continuous batching is enabled, the nodes referenced by active batches cannot be deleted. Each node maintains a reference counter that tracks how many active requests use it, and nodes are removed only when this counter reaches zero.

RadixAttention also applies in multi-GPU environments. Under tensor parallelism, each GPU keeps its own shared **KV** cache and sub-tree without inter-GPU synchronization. The SGLang Router builds and manages a meta-tree by combining all GPU sub-trees. When a new batch arrives, the Router performs prefix matching on

Just Accepted

the meta-tree and selects dispatch policies based on request affinity. After processing, the router and the workers update their local trees independently, and the Router updates the meta-tree during periods of low system load to maintain consistency.

5.6.4 Attention Programming Model: FlexAttention. Numerous attention optimizations have been proposed to accelerate attention operations, but most rely on manually writing hardware-specific kernels, complicating implementation and testing. To improve applicability and flexibility, FlexAttention [71] was introduced.

FlexAttention is a general-purpose, flexible programming model that allows developers to implement diverse attention optimizations with only minimal additional code in PyTorch [263]. Observing that most attention variants can be expressed by modifying the intermediate score matrix before the softmax stage, FlexAttention accepts two callable functions—`score_mod` and `mask_mod`—together with the tensor input. During compilation, PyTorch automatically converts these functions into template-based handwritten attention kernels, and both forward and backward graphs are generated through the PyTorch autograd machinery. Operator fusion occurs automatically in this process, producing optimized kernel code without any low-level kernel development.

FlexAttention also supports sparsity optimization through BlockMask, which records block-level sparsity in the mask matrix to reduce computational load and memory usage while enabling flexible composition of multiple attention variants. The framework allows for independent or combined implementation of techniques such as Relative Positional Embedding [287], ALiBi [269], and FlashAttention [64].

FlexAttention is supported in PyTorch 2.5 and later, and Unsloth [325] provides application of various kernels based on FlexAttention.

5.6.5 MQA Optimization: FireAttention. FireAttention is an FP16- and FP8-based optimization technique developed by Fireworks AI [83] to improve the performance of MoE models. Implemented as a custom CUDA kernel for MQA, it efficiently leverages memory bandwidth across a wide range of batch sizes and sequence lengths on NVIDIA H100 GPUs. Designed for multi-GPU environments, FireAttention achieves higher requests per second (RPS) and lower token-generation latency than traditional LLM inference engines.

FireAttention V2 adds FP16 and FP8 prefill-kernel support for the NVIDIA Hopper architecture and introduces multi-host deployment optimizations. In long-context, online inference workloads, it delivers up to an 8× increase in throughput and up to a 12× reduction in latency relative to existing engines.

FireAttention V3 further refines key matrix-multiplication and attention operations by providing dedicated kernels that extend support beyond NVIDIA GPUs to AMD MI300 hardware.

5.7 Sampling Optimization

Because LLMs generate text autoregressively, longer input sequences increase the amount of computation and prolong user wait times. Moreover, memory I/O latency rather than raw computation often becomes the main bottleneck, affecting performance in interactive AI systems or real-time translation scenarios, where quick responses are critical.

Speculative decoding [173, 352] accelerates token generation by leveraging an optimization concept inspired by speculative execution in computer processors. In speculative execution, operations are performed in parallel before it is confirmed whether they are needed—much like branch prediction.

Fig. 25 illustrates this concept in LLMs through two models: a high-accuracy target model (the original LLM) and a lighter and faster draft model. The draft model generates candidate tokens that the target model later validates. This mechanism, known as speculative sampling, allows the target model to check up to K tokens at once and either accept or reject them. If certain tokens are rejected, additional tokens are sampled from an adjusted probability distribution. Parallel validation of these candidate tokens constitutes speculative decoding, speeding up generation without altering the architecture of the original model. If, during the verification stage,

Just Accepted

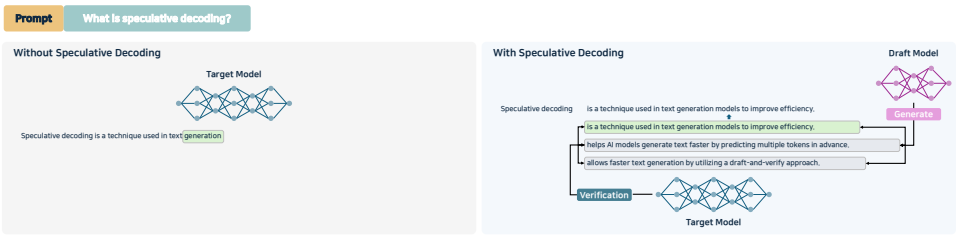


Fig. 25. Speculative Decoding

some of the tokens generated by the draft model are rejected, all draft tokens generated after that point are discarded. In this case, multiple forward-pass computations performed to generate those tokens are effectively wasted. Therefore, the choice of the draft model and the token generation policy must be carefully selected to maximize the efficiency of speculative decoding.

The effective speedup of speculative decoding is highly dependent on how closely the probability distributions produced by the draft model and the target model align. When the draft and target models are trained on substantially different data domains, the acceptance rate of draft-generated tokens decreases, which can instead increase overall response latency. Therefore, in practical systems, selecting model pairs from similar families or with closely related training characteristics is essential to maximize both acceptance rate and performance gains.

In addition, for a similar parameter budget, reducing layer depth while increasing layer width leads to an almost linear reduction in per-token generation latency. In other words, shallow and wide draft models are more favorable for speculative decoding than deep and narrow models, indicating that identifying an optimal layer configuration is a key factor in performance tuning [363].

The efficiency of speculative decoding is strongly influenced by the number of tokens K proposed by the draft model at each step, as well as the sampling parameters such as temperature, top-k, and top-p. As K increases, the frequency of target-model invocations decreases. However, if a rejection occurs early in the verification process, all remaining draft tokens must be discarded and recomputed, resulting in wasted computation. Therefore, rather than using a fixed constant, K should be dynamically adjusted by monitoring factors such as the real-time acceptance rate, available hardware memory, and prompt length.

Regarding sampling behavior, a lower temperature reduces randomness, leading the draft and target models to follow more similar decision paths and thus increasing the acceptance rate. Conversely, higher temperatures can produce more diverse and creative outputs, but often widen the distribution gap between the two models, increasing rejection rates and diminishing speedup. The configuration of the candidate set size (top-k) or accumulative probability threshold (top-p) must also be jointly optimized with temperature. If the candidate space is too restrictive, acceptance rates may be high but output diversity degrades, whereas overly permissive settings can sharply increase rejection rates.

Ultimately, to fully exploit the benefits of speculative decoding, K and the sampling parameters must be adaptively tuned by jointly monitoring target output quality (e.g., accuracy versus creativity), hardware bottlenecks (such as TBT and **KV** cache capacity), and real-time acceptance statistics. These considerations should be treated as part of the operational policy of the inference engine, rather than as static model-level configurations.

Speculative Decoding Model. Several inference engines can employ multiple draft models for speculative decoding. Although lightweight LLMs are commonly used as drafts, some systems adopt models built on the Extrapolation Algorithm for Greater Language-model Efficiency (EAGLE) [183]. EAGLE is a speculative sampling framework that reduces feature-level uncertainty by using a one-step-ahead token sequence and performs accurate feature prediction with minimal overhead. It achieves speculative sampling through a tree-structured

Just Accepted

draft that employs tree attention, and it is lightweight enough for real-world deployment by adding only one transformer-decoder layer as a plug-in module.

Because the acceptance rate of draft tokens depends on both position and context, EAGLE-2 [182] estimates this rate with a confidence score from the draft model and dynamically adjusts the draft-tree structure to increase the number of accepted tokens.

Following recent test-time scaling trends, EAGLE-3 [184] was introduced to overcome the feature-prediction constraints in the original EAGLE [183], which limited token-prediction flexibility and reduced the benefits of data augmentation. EAGLE-3 removes these constraints and directly predicts tokens. By simulating multi-stage generation during training, it maximizes input flexibility for draft models and achieves higher speed-up ratios as training data scales.

Optimization for Speculative Decoding. Research continues to refine speculative execution. Tree-based Speculative Inference [231] simultaneously generates multiple candidate sequences, increasing the likelihood that the target model will approve tokens. To address difficulties in adapting draft models to changing input distributions, online speculative decoding [204] gradually aligns a draft model with the target model using knowledge distillation. This allows training and deployment of different draft models for various input patterns, and routes queries to the most suitable draft model, increasing token acceptance rates. One study proposes accelerating draft model inference by applying MXFP4 weight-only quantization [97]. This research introduces Multi-Level Speculative Decoding with Quantized Draft Models (ML-SpecQD), a method that combines quantization with staged speculative decoding by delegating token generation in the draft model to an even smaller, quantized draft model. Google also applied speculative decoding to its AI search function, which improved the inference speed more than two times.

Most existing speculative decoding methods have been trained and evaluated primarily on short contexts, which can lead to a significant performance drop when applied to inputs consisting of thousands to tens of thousands of tokens. To address this limitation, LongSpec [368] has been proposed. LongSpec is designed to mitigate several bottlenecks encountered by conventional speculative decoding in long-context scenarios. First, it introduces a lightweight draft model that fixes the **KV** cache size to a constant upper bound, thereby preventing excessive memory usage for long contexts. Next, it adopts a hybrid strategy in which the prefix region is computed rapidly in a single pass and stored in the cache, while the subsequent regions are processed using standard tree attention. This approach reduces the computational cost of tree attention in long token sequences. By combining these two techniques, LongSpec enhances both memory efficiency and decoding speed, enabling stable speculative decoding even for tasks that require extended contexts.

Support in Inference Engines. Several LLM inference frameworks already incorporate speculative decoding. Ollama [255], PowerInfer [302], MAX [237] and llama.cpp [101] each expose this capability directly. vLLM [166] performs offline speculative decoding, generating up to five tokens at a time and-including an n-gram-based suggestion module-splits the input string into n-grams to compute similarity scores. MLC LLM [234] supports speculative decoding with lightweight draft models, as well as Medusa [45] and EAGLE [183]-family drafts, and TGI [139] also offers Medusa [45] and n-gram methods. SGLang [399] implements speculative decoding with EAGLE [183], EAGLE-2 [182], and EAGLE-3 [184] and integrates smoothly with Radix Cache and chunked-prefill. TensorRT-LLM [252] accepts draft models such as EAGLE [183], EAGLE-2 [182], and Medusa [45] and additionally supports Recurrent Drafter (ReDrafter) [55], which recursively predicts drafts, and look-ahead decoding, which performs parallel n-gram prediction and verification. Commercial engines, including Friendli Inference [87] and Fireworks AI [83], also leverage speculative decoding to accelerate inference.

SpecForge [293] is a specialized training framework for speculative decoding models, supporting online and offline training as well as training parallelism to efficiently train draft models at various scales. Models trained with SpecForge can be directly loaded into SGLang [399] without additional conversion, enabling fast inference by fully leveraging SGLang's optimized speculative decoding execution path.

Just Accepted

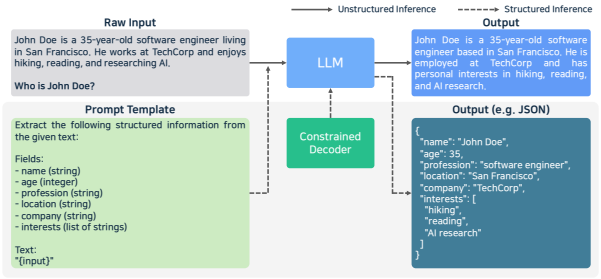


Fig. 26. Unstructured Outputs and Structured Outputs

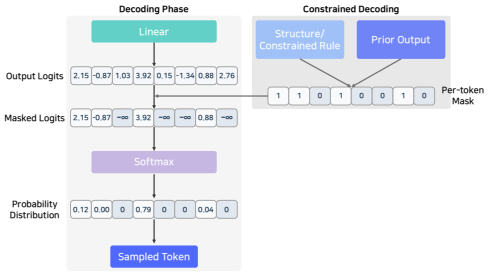


Fig. 27. Constrained Decoding in Decoding Phase

5.8 Structured Outputs

In autoregressive LLMs, the generated tokens function as both the model's inputs and its outputs. During tokenization, however, meaningful units can be divided or Unicode characters fragmented. This limitation becomes especially problematic in applications that require problem solving or planning, such as reasoning tasks or AI agents, where structured outputs like JSON, function calls, or code blocks are essential.

As shown in Fig. 26, structured output [145, 201] refers to generating text that follows a predefined format—JSON, XML, or another structured schema. Unlike typical free-form LLM output, structured generation ensures that the produced content conforms to constraints expected by downstream systems. For example, if a database entry or an API call requires JSON, structured output enables the LLM to return a valid JSON.

Modern LLMs have evolved beyond basic text generation to support tasks such as code creation, function invocation, and autonomous decision making. To enable these applications, inference engines must provide machine-readable structured outputs that integrate smoothly with other systems. Structured generation produces output that complies with explicit schemas—such as JSON or SQL—thus improving correctness and consistency, simplifying interpretation and integration, and reducing hallucinations by eliminating unnecessary or invalid information. Consequently, domain-specific output requirements are often satisfied without additional fine-tuning [332].

Constrained decoding [332, 349] is commonly used to generate structured outputs. As Fig. 27 illustrates, the entire vocabulary is evaluated at each decoding phase and tokens that violate the output schema are masked out. After the logits are computed, a token-level mask invalidates any token that would break the structure; those logits are then set to zero before the softmax operation. This masking procedure directly influences both the performance and the speed of structured generation.

In structured generation with a finite-state machine (FSM) [349], the LLM generates tokens sequentially while moving through the states defined by the machine. The FSM records the current state and assigns a probability of zero to any token that violates the required format, thereby filtering it out. Structures such as JSON schemas can be modeled as directed-graph FSMs, where each node represents a valid partial sequence and each edge represents an allowed next token. FSM decoding is fast, but it handles only simple patterns and cannot easily express recursive structures such as nested JSON.

Using a context-free grammar (CFG) [34, 96] supports more complex structured generation. A CFG defines language structure through production rules and can capture formats that an FSM cannot. Runtime guidance with a CFG, however, requires recursively checking these rules against the entire vocabulary and maintaining multiple parser stacks, which adds significant computational overhead.

Library/Frameworks for Structured Outputs. The Outlines [73] library enables guided generation through a FSM based regular expression parser that can be started or stopped at any point in the decoding process. To

JUST Accepted

efficiently identify valid tokens, it builds an index that lets each step run in amortized $\mathcal{O}(1)$ time. The library expands sequences with multinomial sampling until an EOS token appears, then applies a Boolean mask to produce an unnormalized conditional distribution; the same mask is reused in subsequent steps. In addition to FSM guidance, Outlines supports multiple-choice prompts and structured generation from CFGs written in Extended Backus-Naur Form (EBNF).

XGrammar [72] is a structured generation library designed for efficiency, flexibility, and portability. It supports CFGs for complex formats and includes system-level optimizations for high-speed execution. A C++ backend simplifies integration into diverse runtimes. To reduce recursion overhead, XGrammar converts each CFG into a byte-level pushdown automaton (PDA). The PDA separates context-independent tokens—validated by position alone—from context-dependent tokens that need entire stack inspection. Context-independent tokens are prevalidated and cached for reuse, while context-dependent tokens are dynamically checked by the PDA. XGrammar also maintains a persistent execution stack for rapid branching and rollback and expands context windows to decrease context-dependent tokens, achieving up to 100× lower per-token latency.

The LM Format Enforcer [245] guarantees output formatting through token-level filtering while preserving the expressive freedom of the model. It works independently of the base model and tokenizer, supports batch decoding and beam search, and enforces formats such as JSON Schema, JSON, or regular expressions. Internally, it merges a `Character-Level Parser`, which used to read character sets, with a `Tokenizer Prefix Tree` that stores every token the tokenizer can be generated. A token is accepted only if both structures allow it. After each token is generated, the parser and tree advance together, preparing the constraints for the next step. This approach maintains strict adherence to the target format, while allowing the model to manage details such as spacing and word order.

Low-level Guidance (llguidance) [116] is a library for fast structured output generation using CFGs. It accepts a CFG, a tokenizer, and a token prefix, then computes the set of valid next tokens (a token mask) that can follow the prefix while still yielding strings valid under the grammar. The library is highly efficient, taking only about 50 µs of CPU overhead per token for a 128 k-entry tokenizer. This speed comes from a CFG parser based on the Earley algorithm on top of regular expression derivatives [117] and a token prefix trie for mask computation. Supported grammar formats include the JSON Schema, regular expressions, CFGs derived from Lark [168], and the llguidance's own grammar syntax.

GGML Backus-Naur Form (GBNF) [100] is a formal grammar format introduced in llama.cpp [101] to constrain the output. It merges traditional BNF notation with modern regex features, specifying grammars as production rules that link non-terminal and terminal symbols. GBNF is compatible with JSON Schema and can be used via CLI or API, allowing straightforward control of structured generation.

OpenAI Structured Outputs [258] enables structured generation through function calling and response-format specifications. Function calling permits a model to invoke external system functions safely, while response-format specification ensures that the output matches the desired JSON schema. Both features provide type safety, eliminating post-processing or retries due to format errors, and maintaining consistent formatting without extensive prompt engineering. Built-in safety policies allow the system to reject unsafe or inappropriate requests.

Performance Evaluation of Structured Output. Benchmarks such as The Beyond the Imitation Game Benchmark (BIG-Bench) [304] and Massive Multitask Language Understanding (MMLU)-Pro [342] were used to evaluate multilingual and multitask performance of the LLM. As the quality of structured output has become critical for real-world LLM deployment, newer benchmarks target this capability more directly. JSONSchemaBench [95] provides 10,000 real-world JSON schemas of varying complexity and constraints, enabling the evaluation of constrained-decoding techniques on metrics such as schema adherence, output efficiency, and generalization. Using the official JSON Schema test suite, JSONSchemaBench evaluates llguidance [116], GBNF [100], Outlines [73], XGrammar [72], OpenAI [258], and Gemini [107], providing detailed functional and accuracy analyzes.

Just Accepted

Table 12. Server Specifications for Experimental Environment

Category		Server		Edge
		Server 1 (high-spec)	Server 2 (mid-spec)	
CPU		Intel Xeon Platinum 8480+ × 2	Intel Xeon Gold 6426Y × 2	8-core Arm Cortex A78AE
Memory		2.0 TiB (DDR5)	1.5 TiB (DDR5)	32 GiB (Unified LPDDR5)
OS		Ubuntu 22.04 LTS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS (JetPack 6.0)
GPU	Name	NVIDIA H100 × 8	NVIDIA RTX A6000 × 6	NVIDIA Ampere c GPU
	Memory	640 GB (80 GB per GPU)	288 GB (48 GB per GPU)	32 GB (Unified LPDDR5)
	Interconnection	SXM	NVLink	-

StructTest [49] is a rule-based evaluator that measures the ability of LLM to follow complex instructions while producing structured output. Emphasizing cost efficiency, ease of use, bias reduction, and robustness to data contamination, StructTest reports both accuracy and consistency and indirectly gauges instruction decomposition and reasoning capabilities. It has been applied to models such as GPT-3.5 and GPT-4 [5], the Claude 3 family [25], and DeepSeek-v3 [197] for tasks including summarization, code generation, HTML creation, and mathematical reasoning.

To further study structured generation, SoEval [207] offers a structured output benchmark dataset covering 13 output types—such as JSON and XML—across more than 20 domains, including science, technology, literature, and healthcare.

Support in Inference Engines. From the inference engine perspective, vLLM [166] supports guided decoding with Outlines [73], LM Format Enforcer [245] and XGrammar [72] in both online such as OpenAI Completions and Chat APIs [258] and offline modes. SGLang [399] integrates Outlines [73], XGrammar [72], and llguidance [116], while LightLLM [191] supports Outlines [73] and XGrammar [72]. MAX [237], MLC LLM [234], and TensorRT-LLM [252] adopt XGrammar [72], whereas Ollama [255], llama.cpp [101], and PowerInfer [302] rely on GBNF [100] for format enforcement. LMDeploy [213] provides structured output through its PyTorch [263] interface, and major commercial engines offer equivalent capabilities via proprietary solutions or OpenAI-compatible APIs [258].

6 Empirical Evaluation

In this section, we evaluated the performance of the inference engines introduced in Section 4. Table 2 summarizes the four key metrics used in our experiments (TTFT, TBT, latency, and throughput) which are critical in real-world service environments and served as the basis for our comparison. The analysis covered 21 open-source inference engines and examined how the optimization techniques discussed in Section 5 influenced their practical performance.

All inference engines were installed and benchmarked under the environments described in Table 12. To provide a comprehensive efficiency analysis, experiments were conducted separately on server-class hardware and edge devices.

6.1 Environment Setup

We installed 21 open-source LLM inference engines on both server and edge hardware environments summarized in Table 12, and verified successful installation by running example workloads. Most engines (e.g., Ollama [255], vLLM [166]) supported installation via pip [270] or uv [29], while engines such as MAX [237], TensorRT-LLM [252], and TGI [139] provided official container images (Docker [69], Podman [58]) which facilitated deployment. Lightweight C or C++-based engines such as LLaMA.cpp [101], llama2.c [24], and bitnet.cpp [336] were built directly using make [104] or ninja [244] to generate executable binaries. DeepSpeed-FastGen [129] was distributed as an internal module of DeepSpeed-MII [233], which required the installation of DeepSpeed-MII.

Just Accepted

Table 13. Inference Engines Execution Environments

Inference Engine		Installation			API	
Name	Version	Install Method	Environment	Install Ease	Support	Type
Ollama	0.11.10	curl	venv	Easy	✔	OpenAI-compatible
LLaMA.cpp	b6423 (7057faf6*)	source build	-	Easy	✔	OpenAI-compatible
vLLM	0.10.1.1	pip/uv	venv	Easy	✔	OpenAI-compatible
DeepSpeed-FastGen (DeepSpeed-MII)	0.3.3	pip	venv	Easy	✔	RESTful API (OpenAI-compatible)
unsloth	2025.9.2	pip	venv	Easy	✘	✘
MAX	25.4.0	docker	venv	Easy	✔	OpenAI-compatible
MLC LLM	nightly-cu12	pip	conda	Medium	✔	RESTful API (OpenAI-compatible)
llama2.c	350e04f*	source build	-	Easy	✘	✘
bitnet.cpp	404980e*	source build	conda	Easy	✘	✘
SGLang	0.5.2rc2	uv	venv	Easy	✔	OpenAI-compatible
LitGPT	0.5.10	pip	venv	Easy	✔	Python API (OpenAI-compatible)
OpenLLM	0.6.30	pip	venv	Easy	✔	OpenAI-compatible
TensorRT-LLM	0.21.0	docker	docker	Medium	✔	OpenAI-compatible
TGI	3.3.5	docker	docker	Medium	✔	OpenAI-compatible
PowerInfer	d3ebd7c*	source build	venv	Easy	✘	✘
LMDeploy	0.10.1	pip	conda	Easy	✔	OpenAI-compatible
LightLLM	1.1.0	prebuilt	docker/conda	Easy	✘	✘
NanoFlow	8a28a8c*	source build	docker	Hard	✘	✘
DistServe	82831f1*	source build	conda	Hard	✘	✘
vAttention	ef3fff2*	prebuilt	docker	Easy	✔	OpenAI-compatible
Sarathi-Serve	786d144*	source build	docker	Easy	✘	✘

*Commit number

Some engines required additional configurations. MLC LLM [234] encountered build errors with third-party libraries, which were resolved by using a conda [22] environment. TensorRT-LLM [252] and TGI [139] exhibited instability during source builds; therefore, we used their official container environments instead. DistServe [402] supported only Hugging Face model weights in binary (.bin) format, which required modifying the loader code. NanoFlow [405] required the commercial Gurobi Optimizer [121] for execution graph exploration, which meant that in environments without an academic license, the code had to be modified to use alternative libraries such as Google OR-Tools [106]. After modification, both DistServe and NanoFlow successfully built and ran example workloads, but due to lack of correctness verification, they were excluded from the experimental evaluation.

In the edge environment, Ollama [255] and LLaMA.cpp [101] supported native installation and were directly tested. On NVIDIA Jetson devices, unofficial containers [75] supported only limited inference engines and versions, so these were excluded from the performance comparison. The container-based execution of vLLM [166] and MLC LLM [234] on edge environment resulted in inference errors and was therefore also excluded from the analysis.

Based on these observations, Table 13 categorizes installation difficulty into three levels: `Easy`, `Medium`, and `Hard`. `Easy` refers to cases requiring no additional setup beyond dependency installation and example execution; `Medium` includes cases requiring adjustments to build options or environment configuration; `Hard` applies when extra procedures such as source code modification or replacement of proprietary libraries were necessary.

Documentation Quality vs. Real Installation Difficulty. In Fig. 6, we previously assigned `Ease-of-Deploy` scores for each inference engine based on official documentation, guides, and repositories. However, empirical results revealed discrepancies for certain engines. Unsloth [325], llama2.c [24], bitnet.cpp [336], LitGPT [194], OpenLLM [258], PowerInfer [302], vAttention [268], and Sarathi-Serve [11] received relatively low documentation-based scores, but were found to be easier to install in practice, with dependency setup and example execution proceeding smoothly. In contrast, NanoFlow [405] and DistServe [402] were as difficult to install as indicated by their documentation, requiring additional library configuration or code modification. Engines such as TensorRT-LLM [252], TGI [139], MLC LLM [234], and vAttention [268] which had high documentation-based scores,

Just Accepted

provided official container or development environments (e.g., conda) that effectively offset the complexity of source builds.

6.2 Evaluation of LLM Serving and Inference Performance

6.2.1 Evaluation Environments. In this study, we investigated the compatibility of each LLM inference engine with the OpenAI API, as summarized in Table 13, to evaluate performance. Although benchmark scripts can be implemented individually for each inference engine, differences in supported libraries and dependencies between engines may lead to inconsistencies in environment configuration and code behavior.

To ensure a fair and reproducible performance evaluation, this study adopted an OpenAI API-compatible interface-based approach. This approach minimizes implementation-dependent variability while enabling measurement of end-to-end serving performance, which reflects actual inference efficiency in real service environments.

However, API-based evaluation inherently includes overhead from serving operations, such as server-side request handling, scheduling, and communication latency, in addition to the model's computational performance. Nevertheless, this is considered a practical evaluation approach, as it represents the end-to-end responsiveness perceived by users. Given that most LLM inference engines are provided as API-based services and standalone executables (e.g., CLI), measuring performance under serving mode aligns more closely with real-world application scenarios.

Therefore, the goal of this study is not only to assess computational efficiency within the model itself but also to evaluate the overall inference service pipeline, the end-to-end serving performance.

For performance measurement, we employed GuideLLM [328], a tool capable of simulating inference workloads that closely resemble real-world service conditions. GuideLLM enables analysis of throughput, latency, resource efficiency, and scalability during model deployment. It follows the OpenAI API request-response protocol and supports various workload conditions, including single, concurrent, and asynchronous requests.

Using GuideLLM, we evaluated 13 inference engines compatible with the OpenAI API (Ollama [255], LLaMA.cpp [101], vLLM [166], DeepSpeed-FastGen (DeepSpeed-MII [233]) [129], MAX [237], MLC LLM [234], SGLang [399], LitGPT [194], OpenLLM [37], TensorRT-LLM [252], TGI [139], LMDeploy [213], and vAttention [268]) as listed in Table 13. All engines were tested in the server environment (Table 12 - `Server`), while only Ollama [255] and LLaMA.cpp [101] were evaluated in the `Edge` environment. To ensure comparability, models with similar parameter scales and training datasets were selected from the officially supported weights for each engine. As mentioned earlier, the default options of each inference engine were used in this experiment, and the detailed configurations are summarized in Table 14.

Performance testing was conducted using GuideLLM by generating workloads with different combinations of concurrent request counts (1-64) and input/output token lengths (64-4096). Each test was run for 30 seconds, with the initial 10% of the test period used for warmup and the final 5% for cooldown. Data from this period were excluded from the analysis. In the edge environment, performance measurement data were collected for 240 seconds considering the limited computational capacity. The measured metrics, as described in Table 2, include TTFT, TBT, latency, and throughput, which are critical indicators of both LLM inference and service-level performance. The input prompts consisted of 1,000 synthetic samples generated by GuideLLM, and the results were averaged across all conditions for comparison. The corresponding GuideLLM result metrics for the LLM inference performance metrics defined in Table 2 are summarized in Table 15.

To measure the performance metrics defined in Table 15, the following experimental procedures were applied.

- First, to examine how TTFT varies with prompt length, the concurrency level was fixed at 16 and the output length at 1,024 tokens, while the prompt length was gradually increased to 64, 256, 512, 1,024, 2,048, and 4,096 tokens. In the edge device environment, TTFT was measured by varying the prompt

Just Accepted

Table 14. LLM Inference Engine Serving Default Configuration Comparison

Inference Engine	Context Length	Attention	Max Batch	GPU	API Server	Model Repository
Ollama	4,096	Disable (FlashAttention)	–	1	Go Lang Server	Ollama
LLaMA.cpp	4,096	FlashAttention (Auto)	2,048	Visible devices	HTTP server	HuggingFace
vLLM	Model config	FlashAttention, Cascade Attention (FlashInfer)	–	1	vLLM APIServer	HuggingFace
DeepSpeed	Model config	–	–	1	RESTful / Uvicorn	HuggingFace
MAX	Model config	–	–	1	–	MAX / HuggingFace
MLC LLM	Model config	–	128	1	Uvicorn	MLC LLM / HuggingFace
SGLang	Model config	Triton / FlashAttention / FlashInfer	–	1	HTTP server	HuggingFace
litgpt	–	–	–	1	Uvicorn	HF
openllm	3,192	FlashAttention	2,048	1	–	Bento
TRT	Model config	–	2,048	1	Uvicorn	HuggingFace
TGI	8,192	FlashInfer	128	1	–	HuggingFace
LMDeploy	8,192	–	–	1	Uvicorn	HuggingFace
vAttention	8,192	FlashAttention	128	1	Uvicorn	HuggingFace

Table 15. LLM Inference Performance Metrics and GuideLLM Result Metrics

Category	GuideLLM Metric	Unit	Description
TTFT	TTFT	ms	The time it takes to generate the first token in the output. This reflects the model's initial response latency and the user's perceived responsiveness.
TBT / ITL	ITL	ms	The average time interval between consecutive output tokens, excluding the first token. This indicates the smoothness of generation and the speed of decoding.
End-to-End Latency	Request Latency	ms	End-to-end latency for each request is measured from submission to completion and is a key responsiveness metric for an inference system.
Throughput	Requests Rate (Requests Per Second)	req/s	The number of requests successfully processed per second. This indicates the overall system service capacity and scheduling efficiency.
	Total Tokens Per Second	Tot tok/s	The combined rate of prompts and output tokens processed per second. This measures the overall system efficiency, including input and output token processing.

length to 65, 128, and 256 tokens, while fixing the concurrency level at 2 and the output length at 512 tokens.

- Next, to observe how TBT changes as the output length increases, the prompt length was fixed at 1,024 tokens and the concurrency at 16, while the output length was varied across 512, 1,024, 2,048, and 4,096 tokens. In the edge environment, TBT was evaluated by fixing the prompt length at 128 tokens and the concurrency level at 2, while varying the output length to 256, 512, and 1,024 tokens.
- To evaluate request-handling capability in multi-user environments, both the prompt and output lengths were fixed at 1,024 tokens, and the concurrency level was increased to 1, 4, 8, 16, 32, and 64. For each setting, we collected the number of requests processed per second and the overall success rate. For performance evaluation on the edge device, the prompt length was fixed at 128 tokens and the output length at 512 tokens, while the concurrency level was increased to 1, 2, and 4 to measure request throughput and latency in multi-user scenarios.
- Under the same configurations, we additionally recorded the total number of tokens processed per second to measure token throughput and measured user-perceived latency at each concurrency level.

In the performance evaluation of this study, all inference engines were executed using their default configurations. Each engine differs in serving mechanisms and internal parameters, and although performance can be improved by adjusting these options, the range of supported configurations varies across engines, making it difficult to establish uniform conditions. To avoid potential bias from arbitrary tuning, the experiments focused on maintaining consistent evaluation conditions by preserving the default settings.

Just Accepted

Table 16. Inference Engine Model Support Matrix

Model	Ollama		LLaMA.cpp		vLLM		DeepSpeed-FastGen		MAX		MLC LLM		SGLang		LitGPT		OpenLLM		TensorRT-LLM		TGI		LMDeploy		vAttention	
	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100	A6000	H100
Llama-2-7b-hf	✔	✔	✘	✘	✔	✔	✔	✔	✘	✘	✔	✔	✔	✔	✔	✔	✘	✘	✔	✔	✔	✔	✔	✔	✘	✘
Llama-2-13b-hf	✔	✔	✘	✘	✔	✔	✔	✘	✘	✘	✔	✔	✔	✔	✔	✔	✘	✘	✔	✔	✔	✔	✔	✔	✘	✘
Llama-2-70b-hf	✔	✔	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✔	✔	✔	✔	✘	✘	✘	✘	✘	✔	✘	✔	✘	✘
Meta-Llama-3-8B-Instruct	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✔	✔	✔	✔	✔	✔	✔	✘
Meta-Llama-3-70B	✔	✔	✘	✘	✔	✔	✘	✘	✘	✔	✔	✔	✔	✔	✘	✔	✘	✘	✘	✘	✘	✔	✘	✔	✘	✘
Llama-3.1-8B	✔	✔	✔	✔	✔	✔	✔	✘	✘	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✘	✔	✔	✔	✘	✘
Llama-4-Scout-17B-16E	✔	✔	✔	✔	✘	✔	✘	✘	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘	✔	✘	✘
Mistral-7B-Instruct-v0.3	✔	✔	✘	✘	✘	✔	✘	✘	✘	✘	✔	✔	✔	✔	✔	✔	✘	✘	✔	✔	✔	✔	✔	✔	✘	✘
Mistral-Small-3.2-24B-Instruct-2506	✔	✔	✔	✔	✘	✔	✘	✘	✘	✔	✘	✘	✘	✔	✘	✘	✘	✘	✔	✔	✘	✘	✘	✘	✘	✘
Mixtral-8x7B-Instruct-v0.1	✔	✔	✘	✘	✘	✔	✘	✘	✘	✘	✔	✔	✔	✔	✔	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘	✘
Qwen2.5-7B	✔	✔	✘	✘	✔	✔	✔	✘	✘	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✘	✘
Qwen3-32B	✔	✔	✔	✔	✘	✔	✘	✘	✘	✔	✔	✔	✔	✔	✘	✔	✘	✘	✘	✘	✔	✔	✘	✔	✘	✘
Phi-3-mini-4k-instruct	✔	✔	✔	✔	✔	✔	✘	✘	✘	✘	✘	✘	✘	✘	✔	✔	✘	✘	✘	✘	✔	✔	✔	✔	✘	✘
falcon-40b	✔	✔	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✘	✔	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘	✘
Falcon3-1B-Instruct	✔	✔	✔	✔	✔	✔	✘	✘	✘	✔	✘	✘	✘	✘	✔	✔	✘	✘	✔	✔	✔	✔	✘	✘	✘	✘
gpt-oss-20b	✔	✔	✔	✔	✘	✔	✘	✘	✘	✘	✘	✘	✔	✔	✘	✘	✘	✘	✘	✔	✘	✘	✔	✔	✘	✘
DeepSeek-R1-Distill-Qwen-32B	✔	✔	✔	✔	✘	✔	✘	✘	✘	✘	✔	✔	✔	✔	✘	✘	✘	✘	✘	✔	✘	✔	✘	✘	✘	✘
DeepSeek-V2	✔	✔	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘
opt-6.7b	✘	✘	✘	✘	✔	✔	✔	✘	✘	✘	✘	✘	✔	✔	✘	✘	✘	✘	✘	✘	✘	✔	✘	✘	✘	✘

In this paper, as summarized in Table 16, we first verified which models each inference engine could successfully run on each GPU, and then conducted performance measurements. The experimental setup was standardized by applying a tensor parallelism factor of 4 for models with more than 20B parameters and 8 for models with more than 70B parameters.

As summarized in Table 16, we observed cases in which the same model and inference engine ran successfully on the NVIDIA A6000 but failed on the NVIDIA H100, and vice versa. These differences arise not only from architectural distinctions between the two GPUs but also from whether each inference engine provides the necessary runtime and kernel support required for the corresponding device. In general, the H100 offers greater memory capacity and computational performance than the A6000 and is widely adopted as a primary data-center GPU, resulting in broader support across many inference engines. However, some engines, such as vAttention [268], do not yet provide H100-compatible kernels, making it impossible to execute certain models on the H100 in such cases.

6.2.2 Evaluation Results on Servers with Quantized Models. In this section, we evaluate the performance of various language models using server-grade inference engines. We first distinguish between engines that primarily support quantized models such as Ollama [255], LLaMA.cpp [101], and MLC LLM [234], and engines that mainly operate on FP16/BF16 full-precision weights such as vLLM [166], SGLang [399], and TensorRT-LLM [252], and compare their performance separately. Since the evaluated inference engines support 4-bit weight-only quantization, all experiments were conducted using this format for consistency. To emulate real service workloads, most experiments were conducted with the concurrency level fixed at 16, while the prompt and output lengths were varied to measure the metrics defined in Table 15 including TTFT, TBT, and token throughput.

Six models were selected, including Meta-Llama-3.1-8B, Llama-4-Scout-17B-16E, Qwen3-32B, Phi-3-mini-4k-Instruct, gpt-oss-20b, and DeepSeek-R1-Distill-Qwen-32B, and these models were executed on Ollama [255], LLaMA.cpp [101], and MLC LLM [234].

TTFT variation with respect to prompt length. When measuring TTFT while increasing the prompt length from 64 to 4,096 tokens, all inference engines showed a general trend in which TTFT increased as the prompt length became longer. However, the magnitude of this increase differed depending on the model, the engine,

Just Accepted

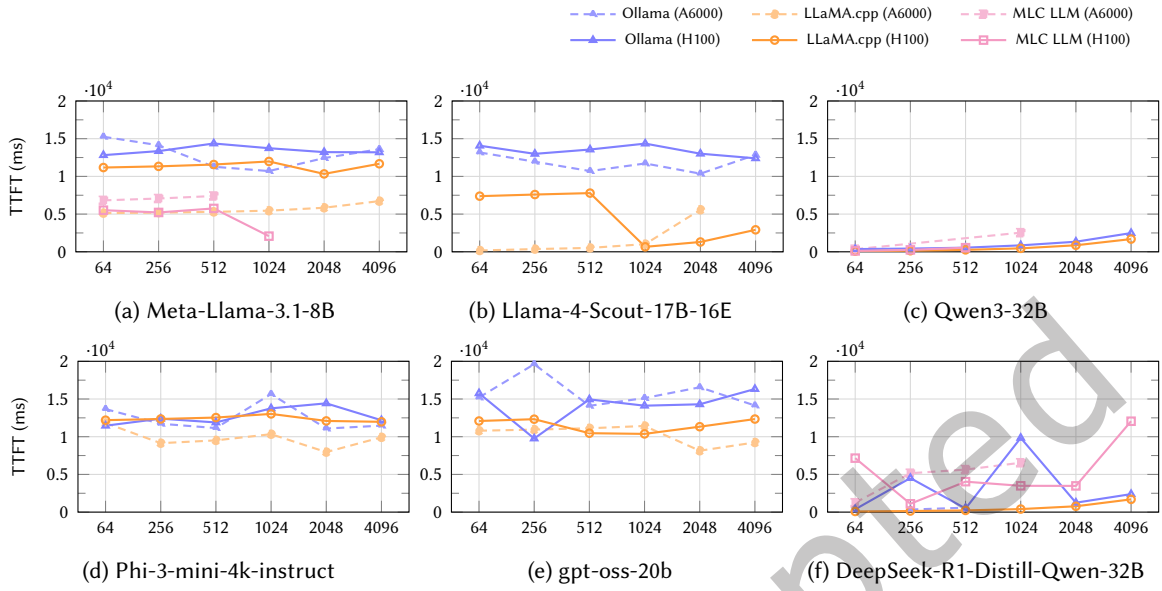


Fig. 28. TTFT variation with prompt length on server devices (4bit Quantized models, Concurrency=16, Output length=1024)

and the underlying GPU architecture. For example, as shown in Fig. 28, Qwen3-32B and DeepSeek-R1-Distill-Qwen-32B exhibited a relatively linear TTFT growth across nearly all engine-hardware combinations, whereas other models showed more gradual increases or, in some cases, nearly constant TTFT across segments. When running Meta-Llama-3.1-8B with a prompt length of 1,024 tokens, LLaMA.cpp (A6000) recorded an average TTFT of approximately 5,454 ms, providing nearly twice the initial response speed of Ollama (A6000) at 10,706 ms. The difference persisted in the H100 environment, where LLaMA.cpp (H100) measured 11,980 ms, compared to 13,737 ms for Ollama (H100). Under the same conditions, MLC LLM (H100) achieved the lowest TTFT at 2,076 ms. For the Llama-4-Scout-17B-16E, the performance gap was even more pronounced. LLaMA.cpp (H100) recorded 658 ms, delivering a substantially faster response than Ollama (H100), which measured 14,349 ms. In certain large-scale models, the absolute TTFT values were unexpectedly low. This appears to be due to specific engines including optimized kernels for those models, or because some requests failed during execution, resulting in a smaller number of samples being included in the final statistics.

TBT variation with respect to output length. When measuring TBT as the output length increased from 512 to 4,096 tokens, the absolute decoding speed varied across GPU-engine combinations, but the overall trend remained consistent. As shown in Fig. 29, when running Meta-Llama-3.1-8B with an output length of 1,024 tokens, Ollama and LLaMA.cpp achieved similar performance on the A6000, recording 11.24 ms and 10.12 ms per token, respectively. In contrast, on the H100, Ollama recorded 6.52 ms, and LLaMA.cpp recorded 4.54 ms, meaning both engines achieved more than double the decoding speed compared to the A6000. A similar trend was observed for the mid-sized model Phi-3-mini-4k-instruct, with LLaMA.cpp (H100) providing the lowest TBT among all tested combinations. Conversely, for larger models such as DeepSeek-R1-Distill-Qwen-32B, TBT remained relatively high even on the H100. For example, Ollama (H100) recorded 17.69 ms, and MLC LLM (H100) recorded 13.34 ms, both noticeably slower than other combinations. In addition, several inference failures occurred on the A6000 when running Qwen3-32B and DeepSeek-R1-Distill-Qwen-32B with output lengths exceeding 512 tokens. On the H100, Qwen3-32B consistently failed once the output length exceeded 1,024 tokens. These results indicate that

Just Accepted

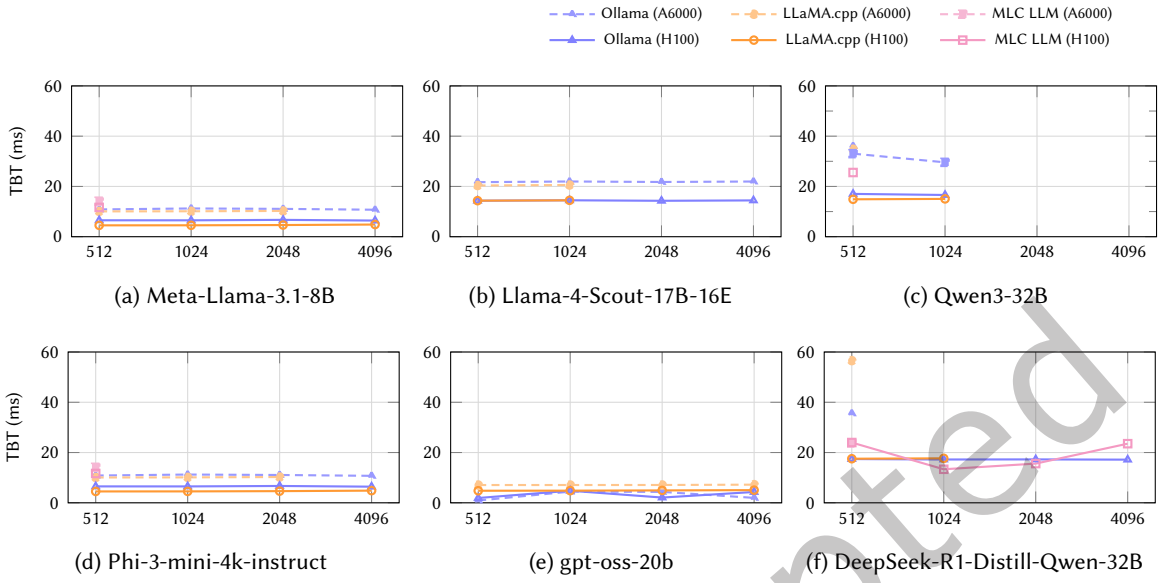


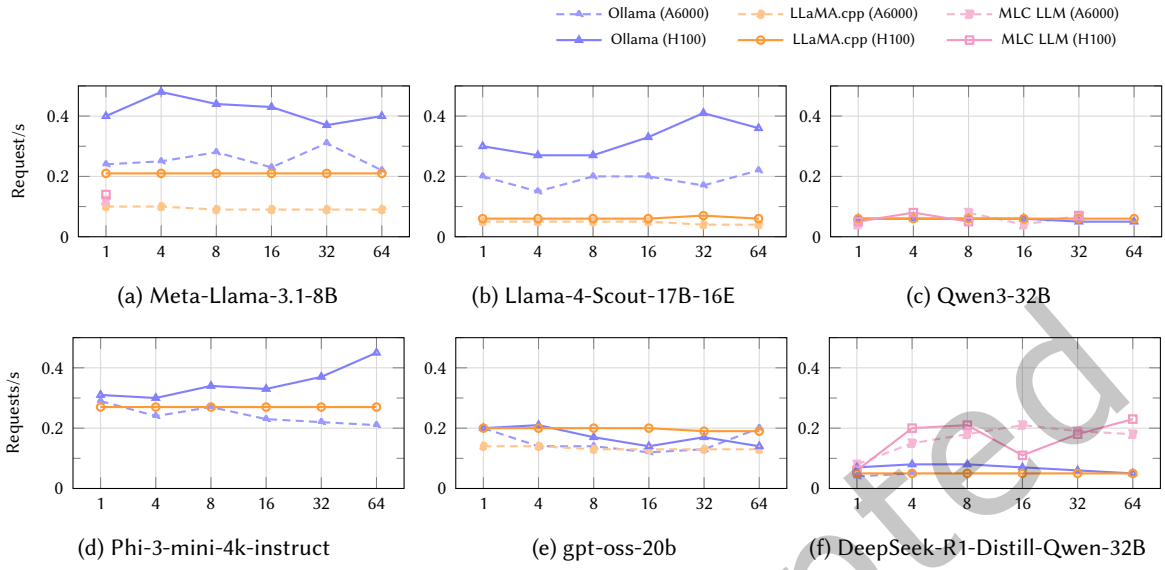
Fig. 29. TBT on server devices with varying output length (4bit Quantized models, Concurrency = 16, Prompt length = 1024)

Just Accepted

for large models, memory and bandwidth requirements increase rapidly with output length, and the kernels and runtime configurations provided by some inference engines may be insufficient to handle such demands.

Request throughput (Requests/s) with respect to concurrency. As shown in Fig. 30, the request throughput measured under varying concurrency levels differed across inference engines due to differences in batch scheduling strategies and internal pipeline designs. For Meta-Llama-3.1-8B with the concurrency level fixed at 16, Ollama (H100) achieved the highest throughput at approximately 0.43 requests/s, followed by Ollama (A6000) at 0.23 requests/s and LLaMA.cpp (H100) at 0.21 requests/s. The throughput improvement on the newer H100, which was roughly twice that of the A6000 for the same engine, indicates that faster GPU decoding speed directly translates into higher batch processing performance. For the Llama-4-Scout-17B-16E, the gap widened further. Under the same test conditions, Ollama (H100) reached 0.33 requests/s, whereas LLaMA.cpp (H100) achieved only 0.06 requests/s. This highlights the increasing importance of batch scheduling strategies and kernel optimizations as model size grows. For the smaller model Phi-3-mini-4k-instruct, the differences between engines were relatively small. Ollama (H100) achieved 0.33 requests/s, and LLaMA.cpp (H100) recorded 0.27 Requests/s, indicating similar levels of performance. This is likely because smaller models require fewer computations per token, making engine-level kernel differences less pronounced.

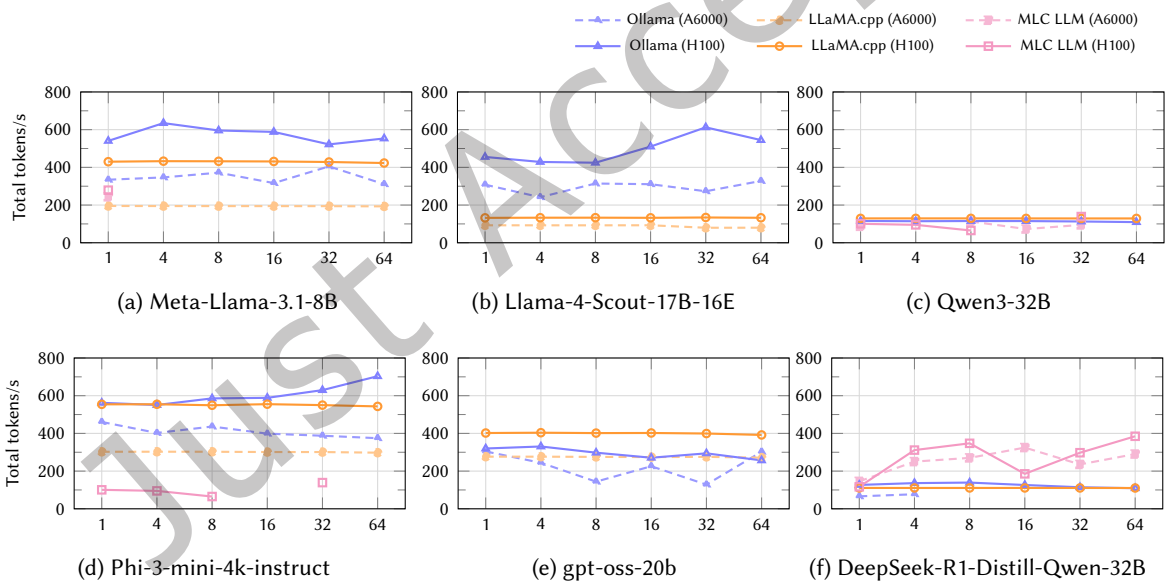
Token Processing Capability. Total tokens/s represents the overall token processing throughput of the system and directly reflects hardware resource utilization and the efficiency of engine implementation. As shown in Fig. 31, for the Meta-Llama-3.1-8B model with concurrency set to 16, Ollama (H100) achieved approximately 588 tokens/s, while LLaMA.cpp (H100) recorded 431 tokens/s. LLaMA.cpp (A6000) reached only 194 tokens/s, which is less than half of the throughput observed on the H100. For Llama-4-Scout-17B-16E, Ollama (H100) achieved 510 tokens/s, which is nearly four times higher than LLaMA.cpp (H100) at 132 tokens/s. In contrast, for the smaller model Phi-3-mini-4k-instruct, the two engines showed almost identical performance, reaching 589 tokens/s and 555 tokens/s, respectively. For the gpt-oss-20B model, LLaMA.cpp (H100) delivered 402 tokens/s, surpassing Ollama (H100), which achieved 271 tokens/s.



(a) Meta-Llama-3.1-8B (b) Llama-4-Scout-17B-16E (c) Qwen3-32B

(d) Phi-3-mini-4k-instruct (e) gpt-oss-20b (f) DeepSeek-R1-Distill-Qwen-32B

Fig. 30. Requests per second on server devices with varying concurrency (4bit Quantized models, Prompt length = 1024, Output length = 1024)



(a) Meta-Llama-3.1-8B (b) Llama-4-Scout-17B-16E (c) Qwen3-32B

(d) Phi-3-mini-4k-instruct (e) gpt-oss-20b (f) DeepSeek-R1-Distill-Qwen-32B

Fig. 31. Total tokens per second on server devices with varying concurrency (4bit Quantized models, Prompt length = 1024, Output length = 1024)

Just Accepted

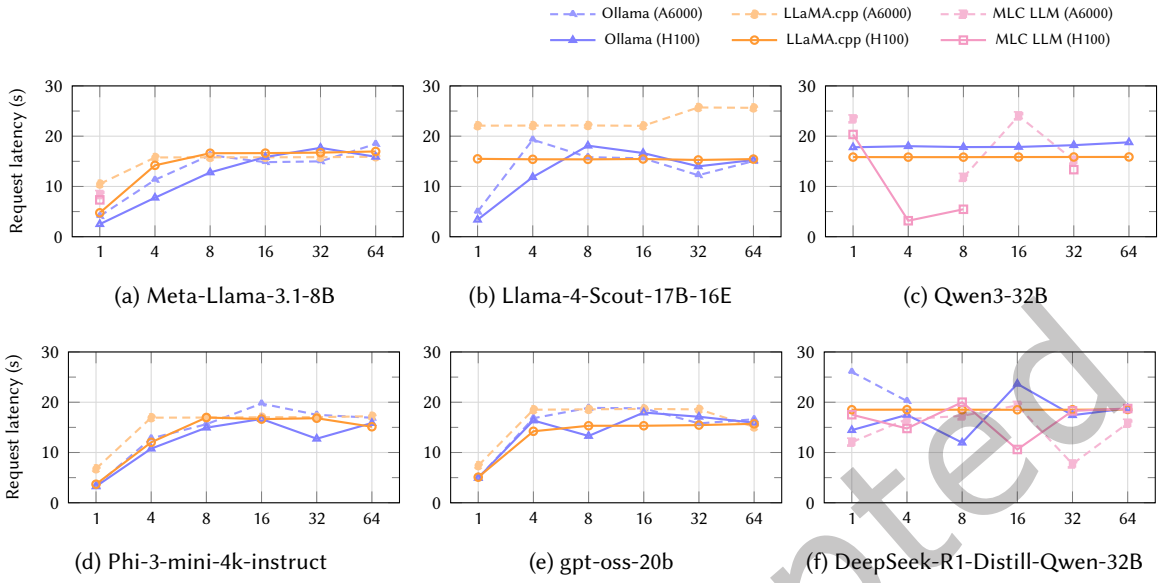


Fig. 32. Request Latency on server devices with varying concurrency (4bit Quantized models, Prompt length = 1024, Output length = 1024)

Just Accepted

End-to-End Latency. End-to-end latency represents the combined effect of TTFT, TBT, scheduling delays, and serving-layer delays, and therefore provides the most accurate indication of the response time perceived by end users. As shown in Fig. 32 the Meta-Llama-3.1-8B model measured at a concurrency level of 16, request latency showed similar values across engines and GPU configurations, generally falling within the range of 15-17 seconds. Specifically, Ollama (A6000) recorded 14.8 s, LLaMA.cpp (A6000) recorded 15.8 s, Ollama (H100) measured 15.9 s, and LLaMA.cpp (H100) recorded 16.6 s, with differences among engines being relatively small compared to those observed in TTFT and TBT. For Phi-3-mini-4k-instruct, Ollama (H100) measured 16.7s, while LLaMA.cpp (H100) measured 16.6 s, indicating nearly identical end-to-end latency. However, detailed metrics show that LLaMA.cpp achieved slightly faster token streaming due to its lower TBT. Overall, most model–engine–hardware combinations exhibited increasing request latency as concurrency increased. However, DeepSeek-R1-Distill-Qwen-32B showed intermittent fluctuations in latency. MLC LLM successfully executed the Qwen3-32B model only under specific configurations on both the A6000 and H100, while other configurations failed to run.

Capacity under increasing concurrency. To evaluate stability as concurrency increased from 1 to 64, we measured success rates and partial-response rates. As shown in Fig. 33, all engines exhibited a common pattern in which success rates dropped sharply and partial responses increased substantially as concurrency increased. This behavior became more pronounced for larger models. For example, with Meta-Llama-3.1-8B at concurrency 16, Ollama (H100) maintained a relatively stable success rate of 48.1%, whereas LLaMA.cpp (A6000) dropped to 11.1%. At concurrency 64, the success rate of Ollama (H100) further decreased to 16.2%, and LLaMA.cpp (A6000) fell to approximately 3%. MLC LLM failed to maintain high concurrency, reaching 0% success starting from concurrency 4. For the Qwen3-32B model, at concurrency 16, Ollama (H100) and LLaMA.cpp (H100) reported success rates of 6.3% and 6.0%, respectively, and both fell below 1.5 at concurrency 64. DeepSeek-R1-Distill-Qwen-32B showed a similar pattern, with Ollama (H100) reaching only 11.8% even at concurrency 16. These results indicate that when

Table 17. Hands-on Experience with Inference Engines for Quantization models)

Inference Engine	Feature Items
Ollama	- Easy installation and execution - Supports model hot-swap and multi-model serving during server operation - Allocates only the memory required for inference
LLaMA.cpp	- By default, utilizes multiple GPUs to optimize computation - Stability degrades when prompt or output length becomes large - Uses relatively low memory
MLC LLM	- The server performance is slow because it wraps the RESTful API into an OpenAI compatible server - Allocates only the necessary memory - Lacks model-checking when receiving client requests, reducing reliability - Offers strong synergy for users proficient with compiler-level optimizations

deploying large models in quantized form on a single server under high concurrency, a substantial portion of requests may fail or return incomplete responses regardless of the inference engine or hardware used.

Key Takeaways

- **Small Models (e.g., Phi-3, Meta-Llama-3.1-8B):** Ollama and LLaMA.cpp both demonstrated performance levels suitable for real service deployment. LLaMA.cpp (H100) achieved exceptionally low decoding latency (4.54 ms/token), enabling responsive streaming, while Ollama (H100) provided higher throughput and more stable success rates.
- **Medium-Scale Models (e.g., Llama-4-Scout-17B-16E, gpt-oss-20B):** Ollama (H100) maintained relatively stable throughput and reliability. In contrast, LLaMA.cpp delivered fast decoding but suffered from significantly lower success rates under concurrency, suggesting limited suitability for high-throughput multi-user scenarios.
- **Large Models (e.g., Qwen3-32B, DeepSeek-R1-Distill-Qwen-32B):** Even with quantization, these models were unable to sustain a concurrency level of 16 or higher on a single server. Success rates remained extremely low (1–10%), indicating that practical deployment would require distributed serving, reduced concurrency, or careful tuning of batching and scheduling parameters.
- **MLC LLM Stability Considerations:** Although MLC LLM achieved very fast TTFT in specific configurations, its overall low success rate makes it unsuitable for production workloads in its current state. This emphasizes that backend stability, scheduler design, and memory management strategies are essential factors that quantization alone cannot compensate for.

6.2.3 Evaluation Results on Servers. For server-side performance analysis, models were selected according to their parameter scale. Small-scale models included Falcon-3-1B-Instruct; medium-scale models included Llama-2-7B-hf, Meta-Llama-3-8B-Instruct, and Qwen-2.5-7B; medium-large models included gpt-oss-20B, DeepSeek-R1-Distill-Qwen-32B, and Meta-Llama-3-70B; large models included Llama-4-Scout-17B-16E; and the extra-large model category was represented by DeepSeek-V2.

The selection also included models with knowledge distillation (e.g., DeepSeek-R1-Distill-Qwen-32B) and MoE architectures (e.g., gpt-oss-20B and Llama-4-Scout-17B-16E) to evaluate how effectively each inference engine can accommodate diverse service requirements. In addition, because large- and larger-scale models often cannot be executed on a single GPU, this experiment also examines the degree of parallelization support and the effectiveness of optimization provided by each inference engine.

TTFT variation with respect to prompt length. In the H100-based environment, TensorRT-LLM, vLLM, LMDeploy, and TGI exhibited the shortest TTFT across most models. As shown in Fig. 34, when a 1,024-token prompt was provided to the Llama-2-7B-HF model, TensorRT-LLM achieved approximately 133 ms, followed by TGI at 186 ms, LMDeploy at 290 ms, and vLLM at 320 ms, making TensorRT-LLM the fastest in terms of initial response. Under the same configuration, A6000-based engines generally showed TTFT values 1.5× to

Just Accepted

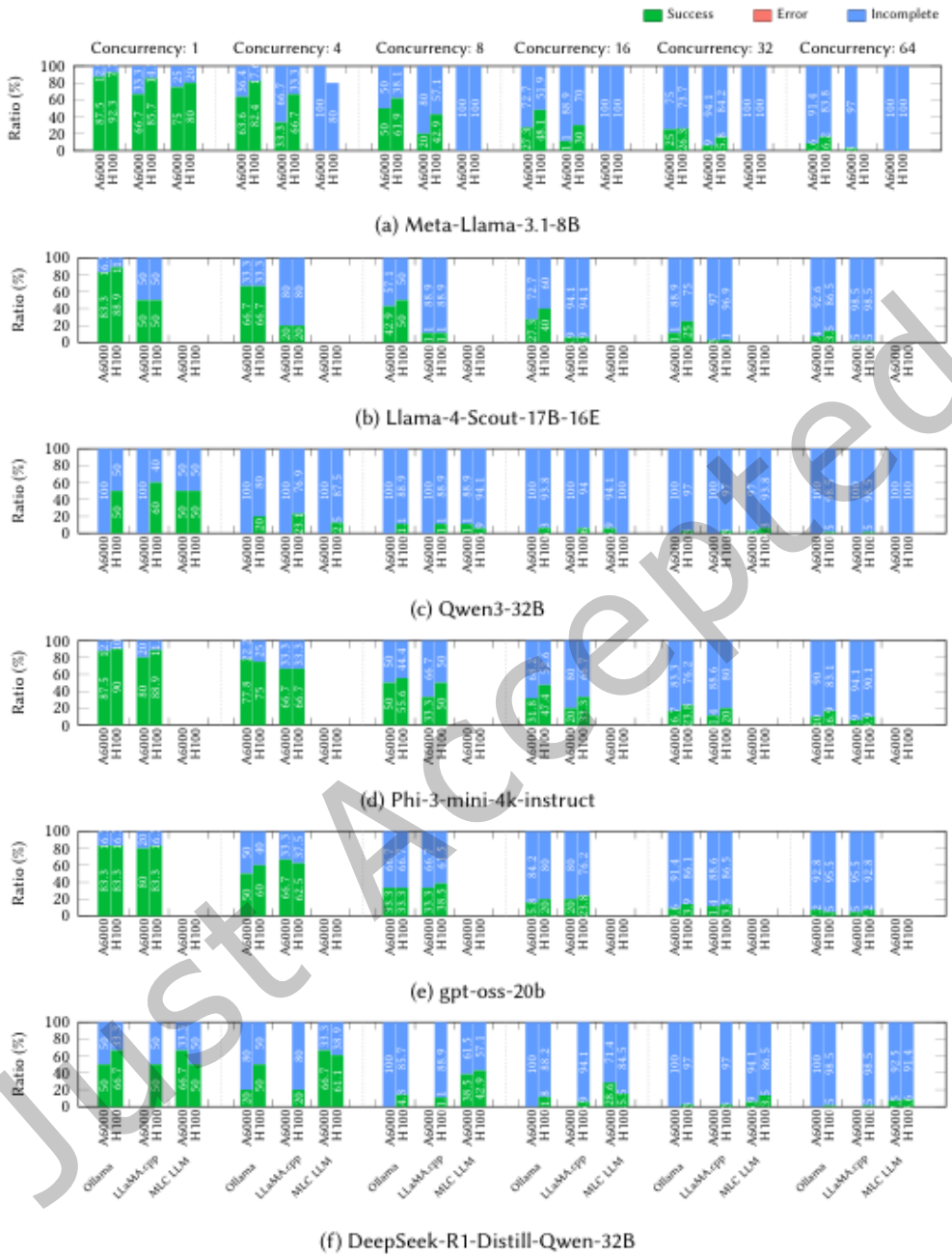


Fig. 33. Impact of concurrency scaling on inference capability (4bit Quantized models, Prompt length = 1024, Output length = 1024)

Just Accepted

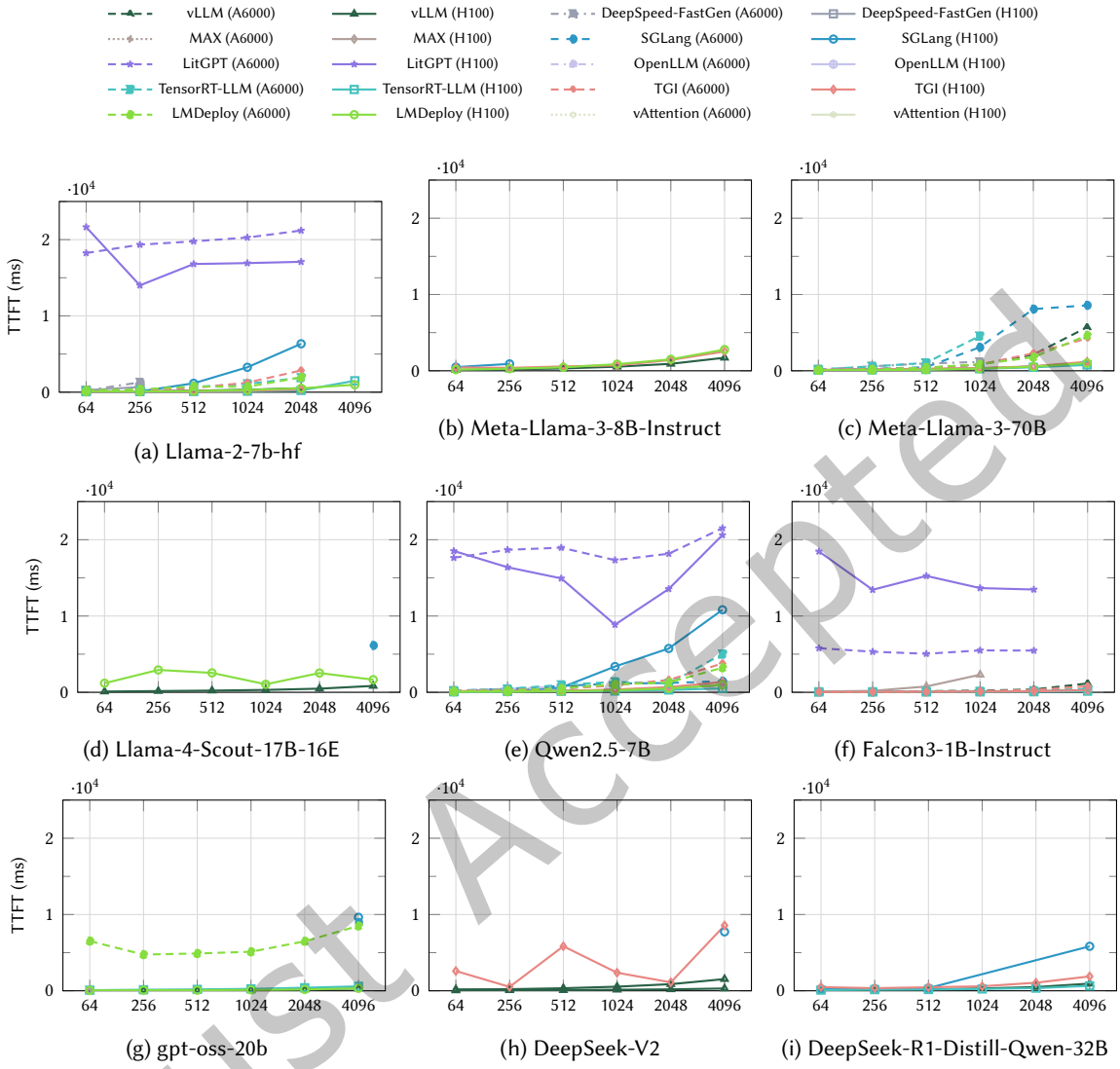


Fig. 34. TTFT variation with prompt length on server devices (Concurrency=16, Output length=1024)

Just Accepted

3× longer than their H100 counterparts. Notably, LitGPT (A6000) recorded delays of 18,000-19,000 ms even for prompt lengths between 64 and 512 tokens, indicating that additional optimization is required for real-time service scenarios. A similar trend was observed with the Qwen 2.5-7B model. On the A6000, vLLM and LMDeploy showed TTFT values of 898 ms and 1,053 ms, respectively, for a 1,024-token prompt, whereas on the H100 these values were reduced to 299 ms and 258 ms. For larger models such as Meta-Llama-3-70B and DeepSeek-R1-Distill-Qwen-32B, the absolute TTFT naturally increased, but TensorRT-LLM and vLLM still recorded the lowest TTFTs on the H100, followed by LMDeploy and TGI, maintaining the same ranking across configurations.

TBT variation with respect to output length. TBT directly reflects the level of kernel and **KV**-cache optimization in the decoding stage. For the Llama-2-7B-HF model, TBT was measured by fixing concurrency at 16 and the prompt length at 1,024 tokens while increasing the output length. The results are as shown in Fig. 35. In the H100 environment, TensorRT-LLM showed an average TBT of approximately 5.0 ms at an output length of 1,024 tokens, 2.65 ms at 2,048 tokens, and 1.23 ms at 4,096 tokens, indicating a decreasing per-token latency as output length increased. Under the same conditions, vLLM (H100) recorded 10.6 ms, 11.3 ms, and 12.6 ms at 512, 1,024, and 2,048 tokens, showing a gradual increase. TGI (H100) produced a relatively flat curve with values of 14.6 ms, 15.1 ms, 15.2 ms, and 14.5 ms at 512, 1,024, 2,048, and 4,096 tokens. LMDeploy (H100) measured 9.6 ms, 10.1 ms, and 11.4 ms at 512, 1,024, and 2,048 tokens, which is moderately lower than TGI but still roughly twice that of TensorRT-LLM. These results indicate that TensorRT-LLM applies the most aggressive kernel and cache optimizations on the H100, whereas vLLM, TGI, and LMDeploy provide more stable performance profiles. For medium-scale models such as Qwen 2.5-7B, the absolute TBT values were slightly lower, but the relative performance ranking among engines remained largely consistent.

Request throughput (Requests/s) with respect to concurrency. Examining the throughput metrics Requests/s show that H100-based engines provide excellent scalability, particularly for small and medium-size models. As illustrated in Fig. 36, for the Llama-2-7B-HF model, TensorRT-LLM (H100) achieved approximately 0.21, 1.06, 1.83, 2.74, 3.52, and 3.68 req/s at concurrency levels of 1, 4, 8, 16, 32, and 64, respectively. Under the same conditions, vLLM (H100) recorded 0.14, 0.50, 0.88, 1.34, 1.87, and 2.00 req/s, corresponding to about 50-70% of TensorRT-LLM. LMDeploy (H100) achieved 1.50, 2.07, and 2.57 req/s at concurrency levels of 16, 32, and 64, slightly outperforming vLLM but still falling below TensorRT-LLM. TGI (H100) produced approximately 2.37 req/s at concurrency 64, placing it between vLLM and LMDeploy. In contrast, engines running on the A6000, such as DeepSpeed-FastGen, MAX, OpenLLM, and vAttention, showed substantially lower throughput, typically in the range of 0.03-0.10 req/s, and in some cases produced no measurable results. These observations indicate that, even on the same hardware, the amount of real-world service traffic that can be supported varies significantly depending on the inference engine being used.

Token Processing Capability. The total tokens per second metric provides a clearer indication of decoding efficiency across inference engines. As shown in Fig. 37, when running the Llama-2-7B-HF model on the H100, TensorRT-LLM achieved approximately 7,535 tokens/s at concurrency 64 and 7,206 tokens/s at concurrency 32. Under the same conditions, vLLM reached 4,107 tokens/s, TGI achieved 3,058 tokens/s, and LMDeploy delivered 4,246 tokens/s. In other words, TensorRT-LLM provides roughly 1.8× the throughput of vLLM and 2.5× that of TGI. For the medium-scale Qwen 2.5-7B model, overall throughput was higher. At concurrency 64, LMDeploy achieved approximately 12,020 tokens/s, vLLM reached 10,928 tokens/s, TensorRT-LLM recorded 10,596 tokens/s, and TGI achieved 9,242 tokens/s. All four engines maintained between 7,000 and 12,000 tokens/s in the concurrency 32-64 range, suggesting that these engines may be suitable for high-concurrency production environments. In contrast, LitGPT and SGLang exhibited sharp drops in throughput beyond certain concurrency levels, with some configurations failing to produce any measurable output. For large models such as DeepSeek-R1-Distill-Qwen-32B, several engines were unable to achieve meaningful throughput, indicating the substantial challenges of serving large models efficiently at scale.

End-to-End Latency. Analysis of request latency shows that, across most models, TensorRT-LLM, vLLM, LMDeploy, and TGI on the H100 maintain average latencies of approximately 10 seconds up to concurrency 32, and converge to around 10-12 seconds even at concurrency 64. For example, in Fig. 38, with Qwen-2.5-7B, LMDeploy (H100) recorded 6.4 s, 6.7 s, 6.9 s, 7.4 s, 8.3 s, and 10.6 s at concurrency levels of 1, 4, 8, 16, 32, and 64, respectively. Under the same conditions, vLLM (H100) showed 6.6 s, 7.0 s, 7.4 s, 8.1 s, 9.0 s, and 12.0 s, while TensorRT-LLM (H100) achieved the lowest latencies at 5.6 s, 6.0 s, 6.2 s, 6.7 s, 7.6 s, and 10.1 s. A similar pattern was observed for Llama-2-7B-HF, where TensorRT-LLM and vLLM demonstrated both high Requests/s and Total tokens/s while keeping latency at comparatively low levels, highlighting their effectiveness as high-efficiency

Just Accepted

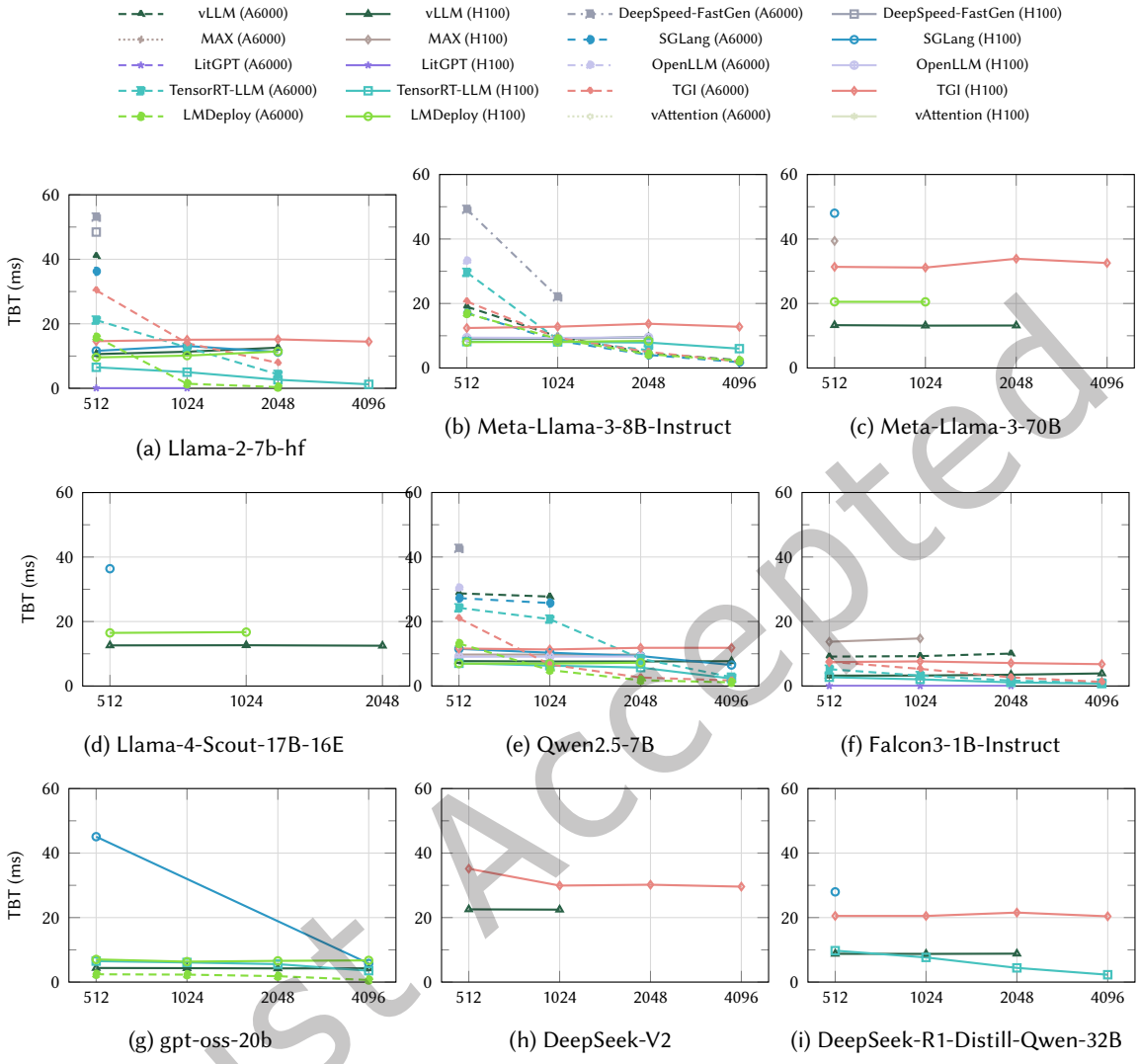


Fig. 35. TBT on server devices with varying output length (Concurrency = 16, Prompt length = 1024)

Just Accepted

serving engines. In contrast, engines such as DeepSpeed-FastGen, MAX, OpenLLM, and vAttention exhibited low success rates or high partial-response ratios, making it difficult to assess service quality based solely on average latency.

Capacity under increasing concurrency. In terms of overall serving capacity, the impact of increasing concurrency shows relatively stable behavior for vLLM, TGI, TensorRT-LLM, and LMDeploy when evaluated with the Llama-2-7B-hf model. As shown in Fig. 39, in the H100 environment, vLLM maintains success rates of 80% at concurrency 1, 75% at concurrency 4 and 8, and continues to record approximately 75% at concurrency 16. When concurrency increases to 32 and 64, the success rates gradually drop to 66.7% and 50%, respectively, yet more than half of the requests are still completed successfully. TGI (H100) records success rates of 83.3% and

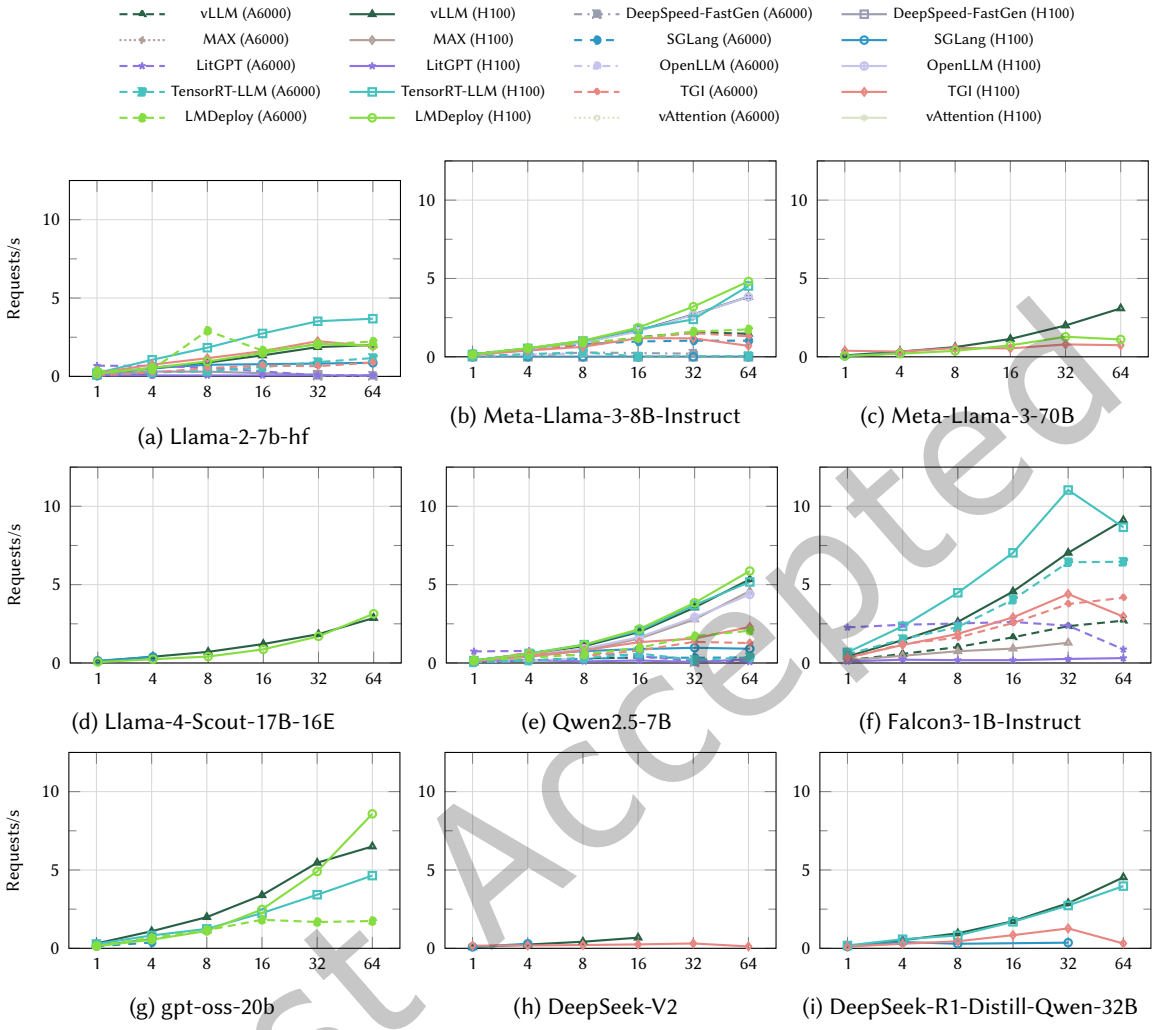


Fig. 36. Requests per second on server devices with varying concurrency (Prompt length = 1024, Output length = 1024)

87.5% at concurrency 1 and 4, 82.5% at 8, and then declines to 67.1%, 56.5%, and 25.% at concurrency 16, 32, and 64. TensorRT-LLM (H100) achieves 85.7%, 89.2%, and 87.7% at concurrency 1, 4, and 8, respectively, and maintains relatively high success rates of 78.5%, 68.6%, and 67.2% at concurrency 16, 32, and 64. In contrast, some engines such as vAttention show near-zero success rates or a majority of partial responses for the same model, making them unsuitable for practical service deployment. For example, with Llama-2-7B-hf, DeepSpeed-FastGen (A6000) achieved only 5.9% success at concurrency 16, with 94.1% of requests resulting in incomplete responses. SGLang and LitGPT maintained success rates above 70% at lower concurrencies (1, 4, 8) on the A6000, but produced almost no valid responses when concurrency increased beyond 16. These patterns were consistently observed across other models, including Meta-Llama-3-8B-Instruct, Qwen-2.5-7B, Falcon-3-1B-Instruct, GPT-OSS-20B, and DeepSeek-V2, indicating that each inference engine has a different upper bound for stable concurrency.

Just Accepted

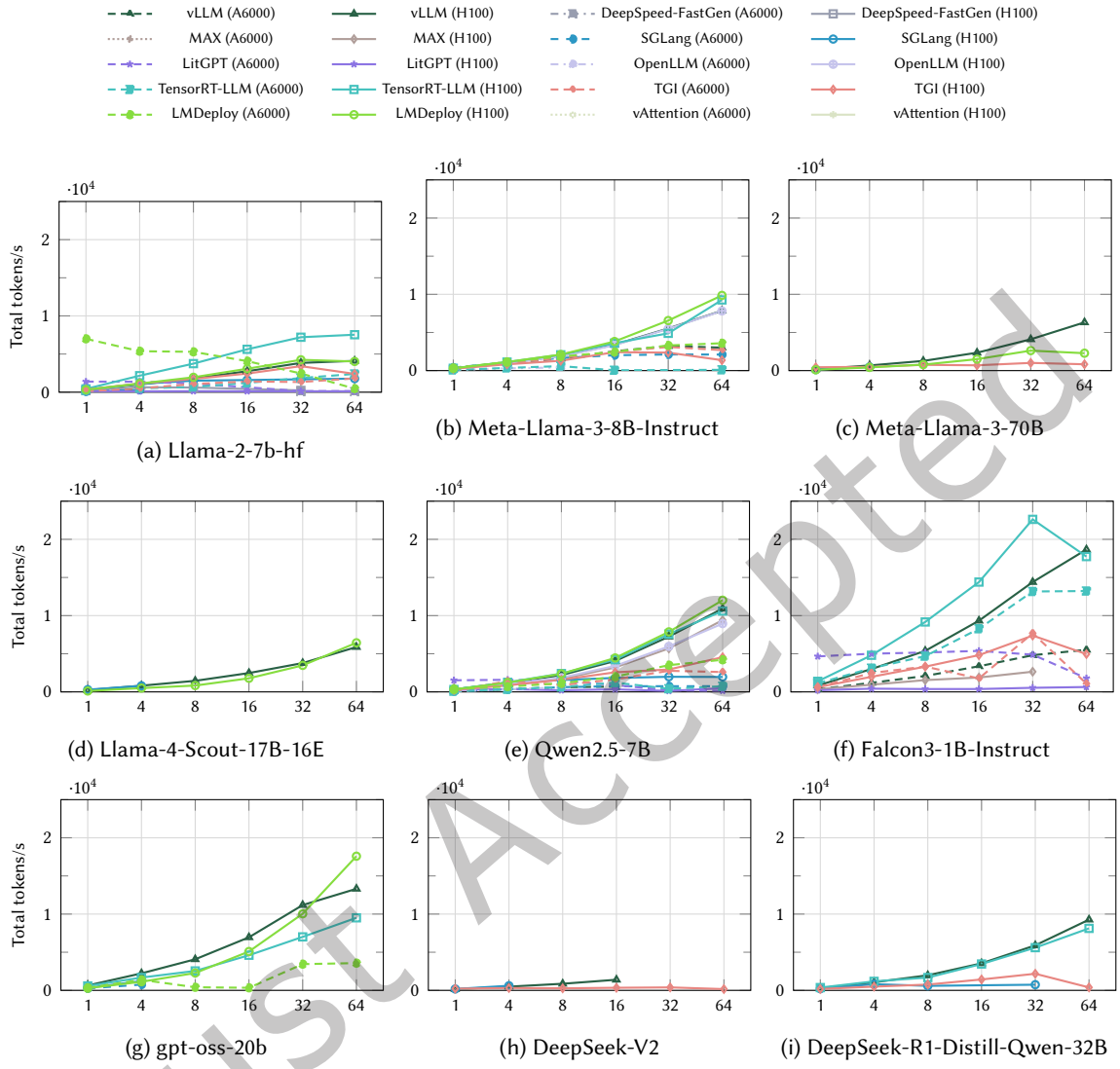


Fig. 37. Total tokens per second on server devices with varying concurrency (Prompt length = 1024, Output length = 1024)

Just Accepted

Key Takeaways

- **Top Server-Side Performance:** On H100 servers, TensorRT-LLM, vLLM, LMDeploy, and TGI consistently delivered the strongest performance among full-precision inference engines.
- **TensorRT-LLM Shows Peak Performance with Minor Stability Trade-Offs:** TensorRT-LLM achieved the highest performance across nearly all metrics—TTFT, TBT, Requests/s, Total tokens/s, and request latency. However, it occasionally showed higher failure or partial-response rates for certain models or at high concurrency.

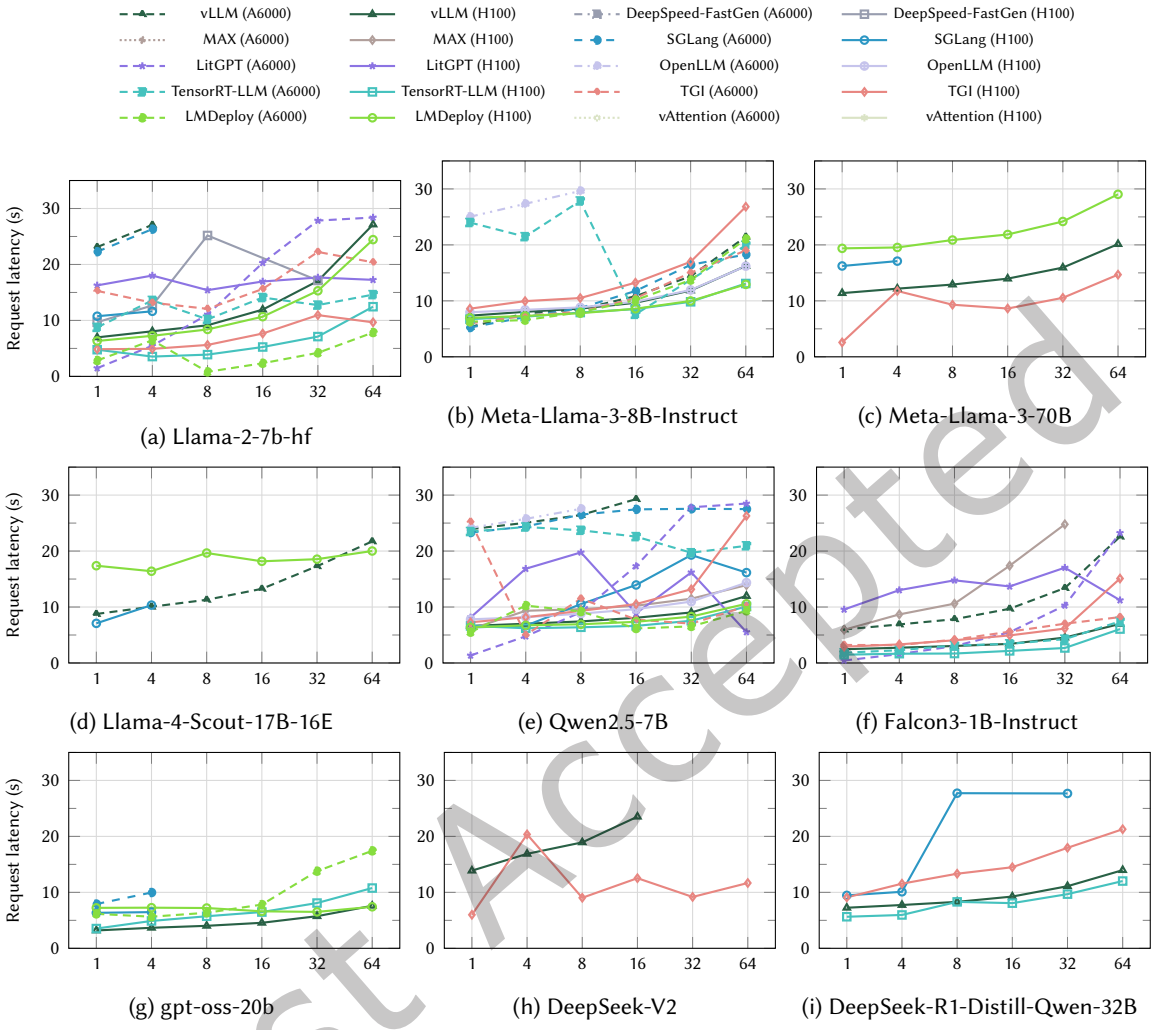
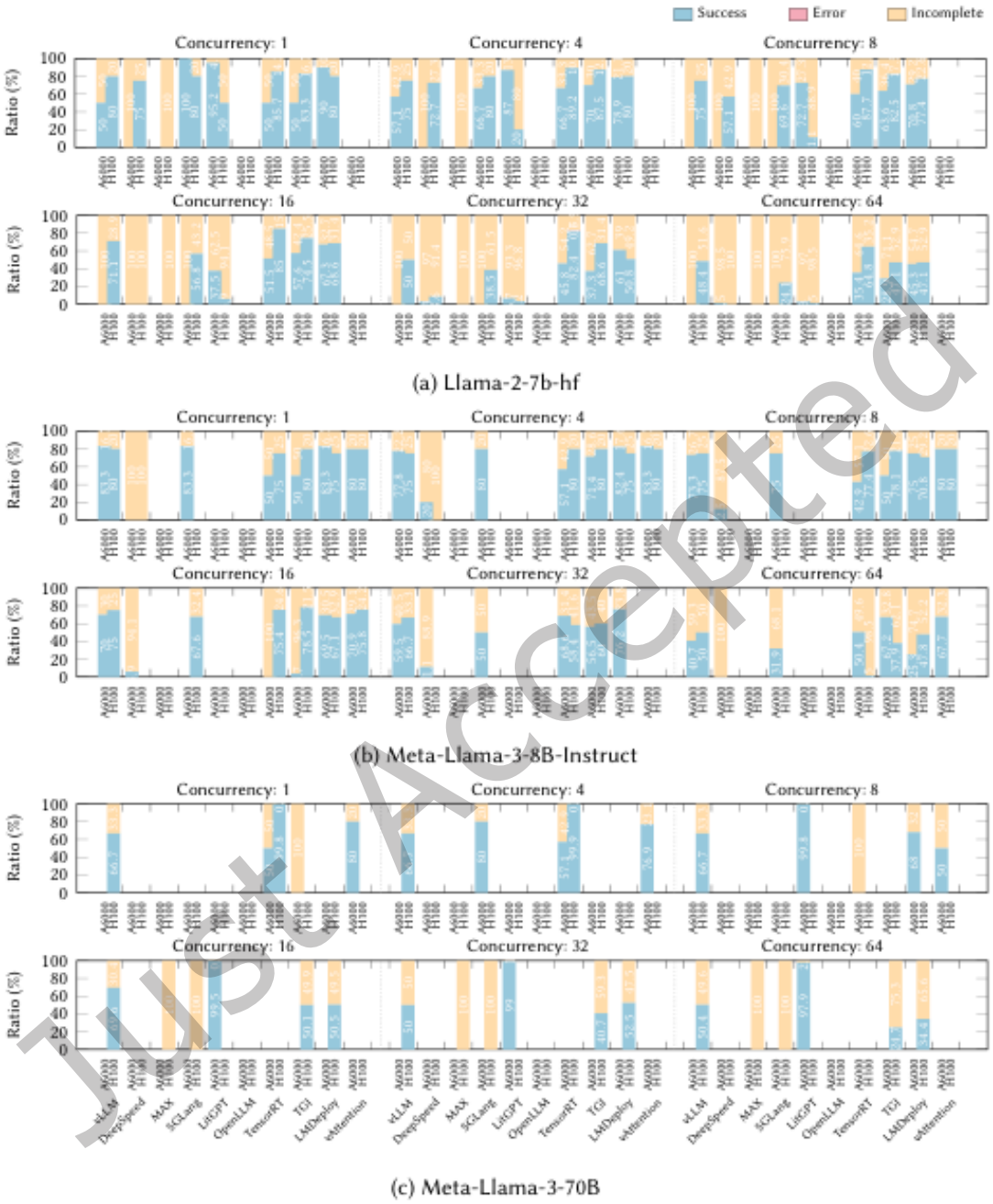


Fig. 38. Request Latency on server devices with varying concurrency (Prompt length = 1024, Output length = 1024)

Just Accepted

- **vLLM, LMDeploy, and TGI: Balanced and Stable Alternatives:** Although these engines did not reach TensorRT-LLM's peak performance, they exhibited smoother degradation and more stable behavior under load, making them strong general-purpose choices.
- **SGLang, LitGPT, and DeepSpeed-FastGen offer competitive performance but reduced stability:** These engines achieved competitive performance under specific hardware-model-concurrency combinations. Their stability, however, decreased noticeably as concurrency levels or model size increased, leading to reduced throughput and lower success rates.
- **Consistency Across Diverse Models:** The observed performance-stability patterns were consistent across a wide range of architectures, including Llama-2-7B-hf, Meta-Llama-3-8B/70B, Llama-4-Scout-17B, Qwen-2.5-7B, Falcon-3-1B, GPT-OSS-20B, DeepSeek-V2, and DeepSeek-R1-Distill-Qwen-32B.



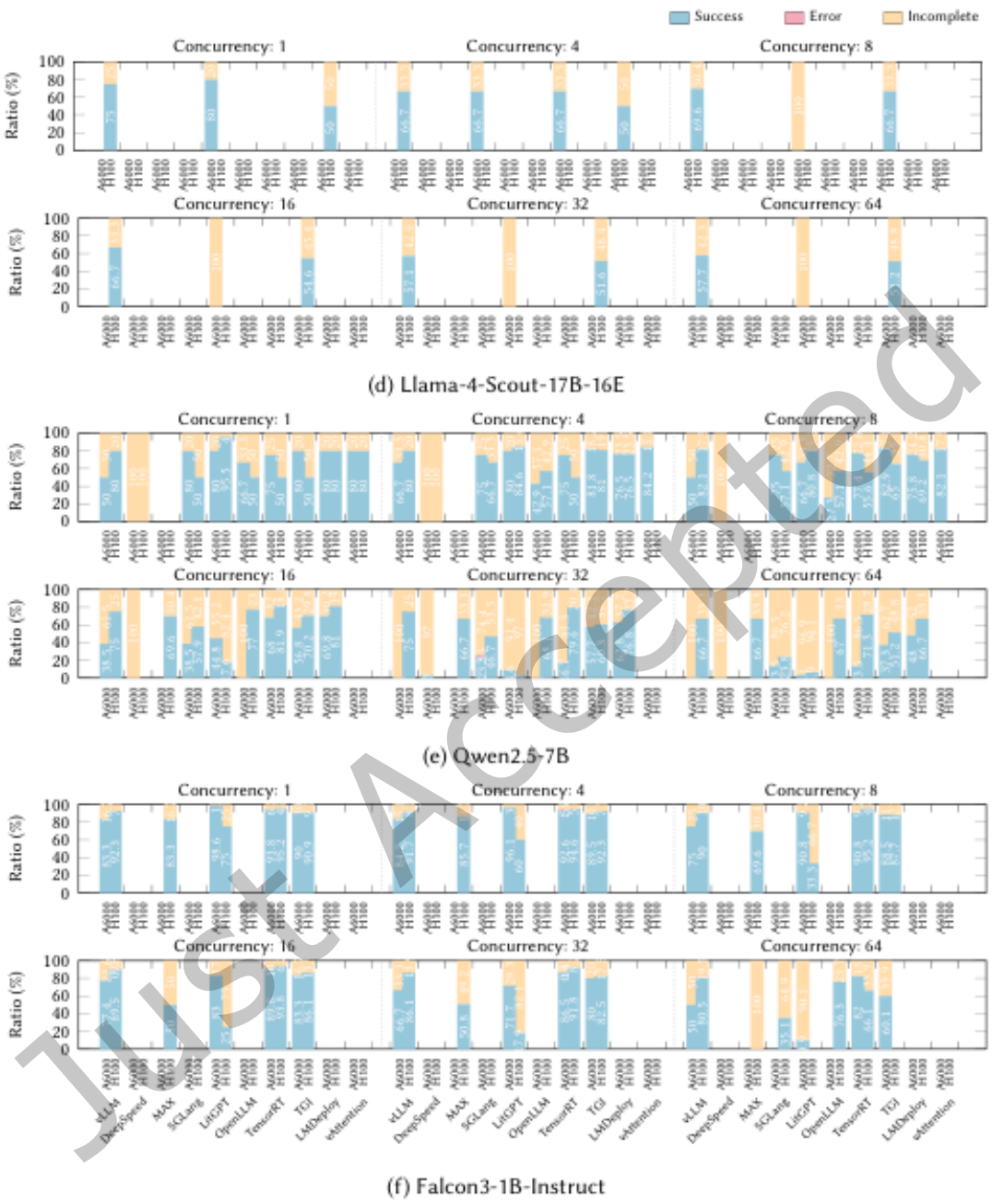
(a) Llama-2-7b-hf

(b) Meta-Llama-3-8B-Instruct

(c) Meta-Llama-3-70B

Just Accepted

Fig. 39. Impact of concurrency scaling on inference reliability on servers (Prompt length = 1024, Output length = 1024)



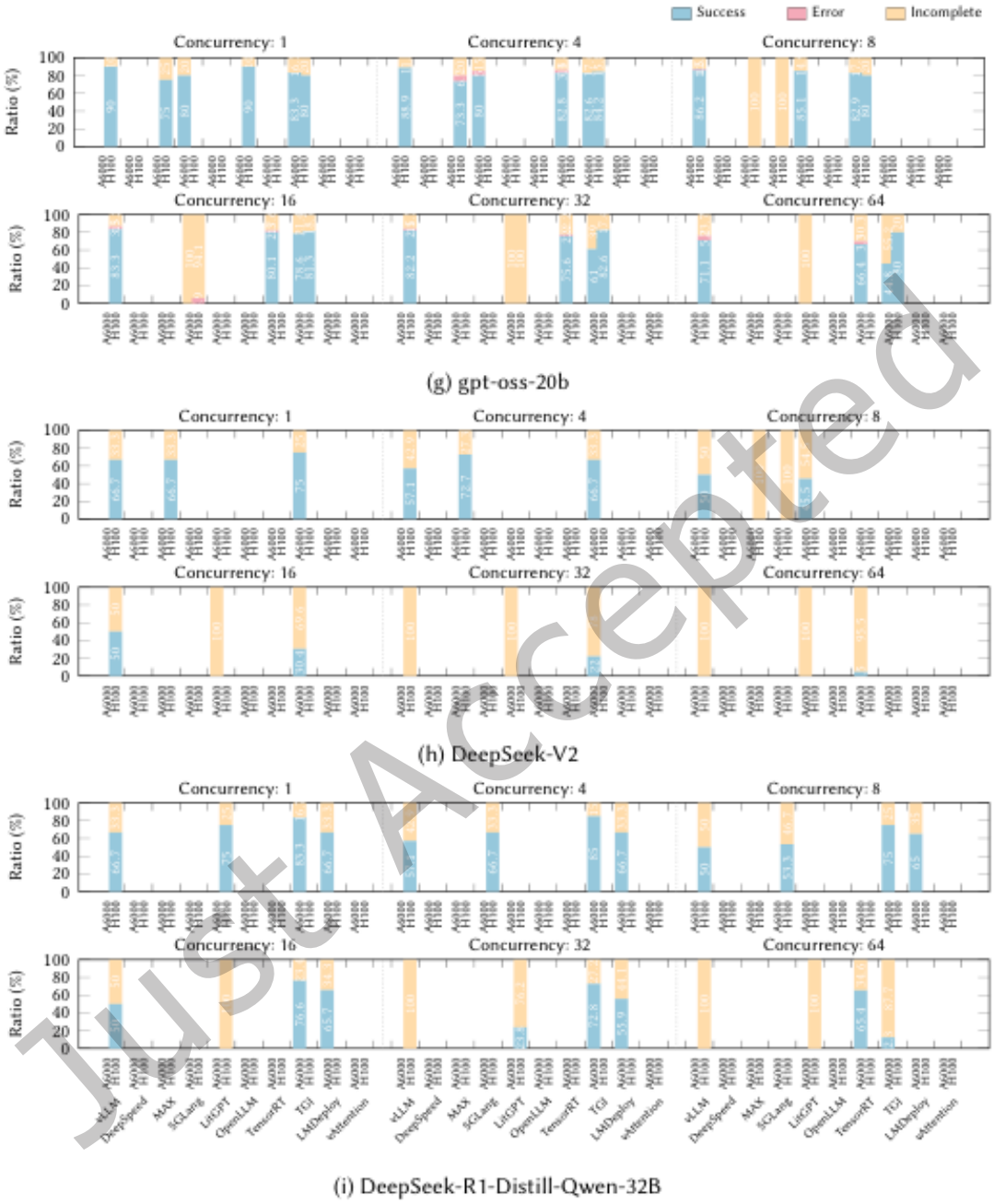
(d) Llama-4-Scout-17B-16E

(e) Qwen2.5-7B

(f) Falcon3-1B-Instruct

Fig. 39. (continued) Impact of concurrency scaling on inference reliability on servers (Prompt length = 1024, Output length = 1024)

Just Accepted



Just Accepted

Fig. 39. (continued) Impact of concurrency scaling on inference reliability on servers (Prompt length = 1024, Output length = 1024)

Table 18. Hands-on Experience with Inference Engines

Inference Engine	Feature Items
Ollama	- Easy installation and execution - Supports model hot-swap and multi-model serving during server operation - Allocates only the memory required for inference
LLaMA.cpp	- Utilizes multiple GPUs by default for optimized computation - Stability degrades when prompt or output length becomes large - Uses relatively low memory
MLC LLM	- Server performance is slow due to RESTful API wrapping into an OpenAI-compatible server - Allocates only necessary memory - Lacks model-checking when receiving client requests - Strong synergy for users proficient in compiler-level optimizations
vLLM	- Supported context length is strict and tied to the model's maximum window - Allocates GPU memory aggressively (1B models may reach 90 percent of total GPU memory)
DeepSpeed-FastGen	- Wraps a RESTful interface as an OpenAI API, resulting in slow serving performance - Unstable with large models or large input/output sequences - Limited support for long-context inference - Aggressive memory allocation (7B models may reach 90 percent of GPU memory)
MAX	- Supports only specific accelerators (H100, MI300, CPUs) - A6000 is not supported - Multi-GPU execution unavailable for many models
SGLang	- Requires an additional Python server implementation - Moderate memory usage (40 percent of GPU memory for 7B models)
LitGPT	- Not directly compatible with GuideLLM's OpenAI API interface - Requires a proxy server or code modifications - Moderate GPU memory usage (40 percent for 7B models)
TensorRT-LLM	- Compatible with OpenAI API (GuideLLM request format requires modification) - Supports only chat-oriented models in current evaluation - Utilizes full available GPU memory
TGI	- Provides optimizations such as prefix caching and FlashInfer - Consumes substantial GPU memory (90 percent for 8B models)
LMDeploy	- Based on the Turbomind backend - Tensor parallelism can be unstable - High GPU memory usage (90 percent for 8B models)
vAttention	- Uses Sarathi as backend - Serving via Ray Workers - Moderate GPU memory usage (50 percent for 8B models) - Tensor parallelism is unstable - Does not support H100 due to incompatible kernel versions

– **Implication for Engine Selection:** Choosing an inference engine for server-side deployment involves balancing peak speed against operational stability. In practice, TensorRT-LLM offers unmatched performance, while vLLM, LMDeploy, and TGI provide more stable and reliable behavior under sustained workloads.

6.2.4 Evaluation Results on Edge. Edge-device performance evaluation was conducted only for Ollama and LLaMA.cpp, the two engines that can run natively on the NVIDIA Jetson Orin AGX 32GB board. Due to hardware constraints, the evaluation was limited to quantized models, and OpenAI API-style requests were generated using GuideLLM, consistent with the server-side experiments. However, considering the limited computational resources and throughput of the edge device, the experimental configuration was adjusted by reducing concurrency levels, prompt lengths, and output lengths.

Just Accepted

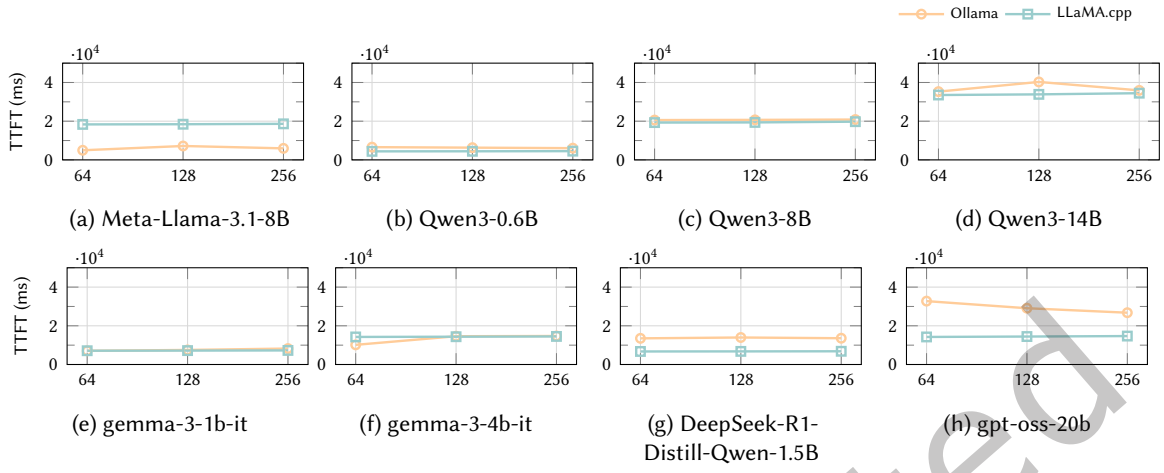


Fig. 40. TTFT variation with prompt length on edge devices (Concurrency=2, Output length=512)

TTFT variation with respect to prompt length. On the Jetson Orin AGX 32GB board, all models exhibited multi-second to multi-tens-of-seconds delays before returning the first token, with the magnitude of latency varying significantly across engine-model combinations. As shown in Fig. 40, for the Llama-3.1 8B model under concurrency 2 with prompt lengths of 64, 128, and 256 tokens, Ollama recorded TTFT values of approximately 4.97 s, 7.18 s, and 5.96 s, respectively. Under the same conditions, LLaMA.cpp measured 18.3 s, 18.4 s, and 18.6 s, indicating that Ollama provided initial response times roughly 2.5-3.5× faster for this model. In contrast, the trend reversed for the Qwen3 0.6B model. For prompt lengths of 64, 128, and 256 tokens, Ollama recorded TTFT values of 6.59 s, 6.33 s, and 6.06 s, while LLaMA.cpp achieved 4.41 s, 4.44 s, and 4.51 s, respectively, offering about 20-30% lower latency. For medium-to-large models such as Qwen3 8B and Qwen3 14B, TTFT increased sharply for both engines. On Qwen3 8B, Ollama reached 20.6-20.8 s and LLaMA.cpp 19.3-19.7 s. On Qwen3 14B, Ollama recorded 35.3-40.3 s, while LLaMA.cpp measured 33.5-34.5 s. Although LLaMA.cpp consistently provided 1-6 s lower TTFT in this range, the absolute delay exceeding 30 seconds indicates substantial initial response latency for practical use. Clear differences in initial response characteristics between the two engines were also observed for models such as the Gemma family, DeepSeek, and gpt-oss-20B. For Gemma3-1B, both Ollama and LLaMA.cpp recorded TTFT values in the 7.1-8.2 s range, showing no meaningful gap between the two engines. However, for Gemma3-4B, Ollama achieved 10.2 s (prompt 64), 14.6 s (128), and 14.7 s (256), whereas LLaMA.cpp recorded 14.2 s, 14.4 s, and 14.5 s, indicating that Ollama provided initial response times approximately 20-30% faster. The opposite trend appeared for DeepSeek-R1-Distill-Qwen-1.5B. While Ollama produced TTFT values around 13.5-14.0 s, LLaMA.cpp achieved 6.7-6.8 s, more than twice as fast. A similar pattern was observed for gpt-oss-20B, where Ollama recorded 26.8-32.8 s, whereas LLaMA.cpp measured 14.2-14.7 s, delivering roughly half the TTFT. In summary, on the Jetson Orin platform, Ollama provides significantly faster TTFT for certain models such as Llama-3.1-8B and Gemma3-4B, whereas LLaMA.cpp consistently outperforms Ollama for other model families including Qwen3-0.6B, DeepSeek-1.5B, and gpt-oss-20B. This indicates that the two engines differ in their optimization pathways, and their performance advantages vary depending on model architecture characteristics.

TBT variation with respect to output length. In the Jetson Orin environment, the measured TBT latency generally ranged from several milliseconds to several tens of milliseconds per token. For example, as shown in Fig. 41, for Llama-3.1-8B, even when the output length increased to 256, 512, and 1,024 tokens, Ollama maintained TBT values of 38.4 ms, 38.5 ms, and 38.4 ms, respectively, while LLaMA.cpp recorded 35.2 ms, 35.2 ms, and 35.3

Just Accepted

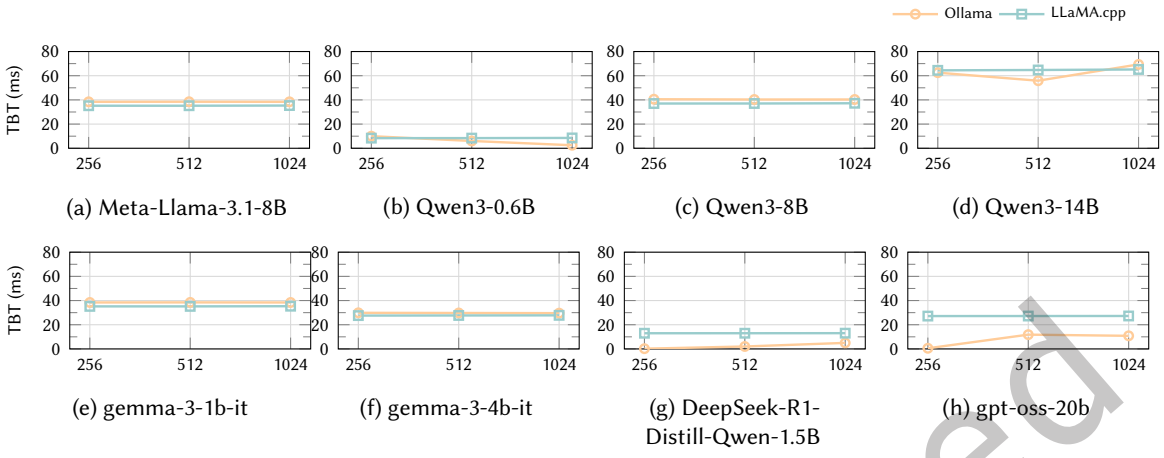


Fig. 41. TBT on edge devices with varying output length (Concurrency = 2, Prompt length = 128)

ms, showing an almost flat profile. For Qwen3-8B, Ollama remained around 40 ms, and LLaMA.cpp around 37 ms, resulting in a difference of only about 3 ms between the engines. For larger models such as Qwen3-14B, per-token latency increased to around 60 ms, but the performance gap between engines remained within 10 percent. Small models exhibited a different pattern. With Qwen3-0.6B, LLaMA.cpp remained nearly constant at 8.4-8.6 ms across all output lengths, but Ollama decreased from 10.0 ms (256 tokens) to 6.0 ms (512 tokens) and 2.5 ms (1,024 tokens), showing lower latency as output length increased. For Gemma-3 1B and 4B, LLaMA.cpp recorded 35-36 ms and 27-28 ms, respectively, which were 2-3 ms faster than Ollama. Conversely, for DeepSeek-1.5B and gpt-oss-20B, Ollama's TBT dropped to as low as 1 ms in some segments, whereas LLaMA.cpp maintained approximately 13 ms and 27 ms, respectively. These results indicate that per-token latency varies depending on the model family and scale, and no single engine dominates across all cases. However, since overall response time is more heavily influenced by TTFT, differences in TBT may have a relatively limited impact on perceived user performance.

Request throughput (Requests/s) with respect to concurrency. The request throughput on edge devices is one to two orders of magnitude lower than in server environments. Due to the limited GPU compute capability and memory bandwidth of the Jetson Orin platform, increasing concurrency does not meaningfully improve throughput; for some models, throughput remained nearly unchanged even as concurrency was raised. For example, as shown in Fig. 42, the throughput of Ollama on Llama-3 8B remained almost flat at 0.14, 0.14, and 0.15 req/s for concurrency levels 1, 2, and 4, respectively, indicating that 8B models cannot benefit from batch parallelism on Jetson-class hardware. Under the same settings, LLaMA.cpp achieved 0.05, 0.05, and 0.04 req/s, approximately one-third of Ollama's throughput. For smaller models such as Qwen3 0.6B, both engines achieved around 0.20 req/s, though at concurrency 4, Ollama showed a 5-10 percent lower throughput than LLaMA.cpp. This reflects the architectural characteristics of Jetson devices, where front-end and model execution threads become less of a bottleneck as model size decreases. For mid-size models (Qwen3 8B and 14B), throughput dropped sharply for both engines. In particular, Qwen3 14B reached only 0.07 req/s at concurrency 4, making real-time service effectively infeasible. Gemma-3 1B and 4B also remained within 0.15-0.25 req/s, even when concurrency increased. For the large-scale gpt-oss-20B model, Ollama achieved 0.06 req/s and LLaMA.cpp 0.08 req/s at concurrency 4, maintaining a 20-30 percent performance gap.

Token Processing Capability. On the Jetson Orin platform, the significantly lower memory bandwidth and compute capability compared to server-grade hardware lead to a pronounced decline or stagnation in throughput

Just Accepted

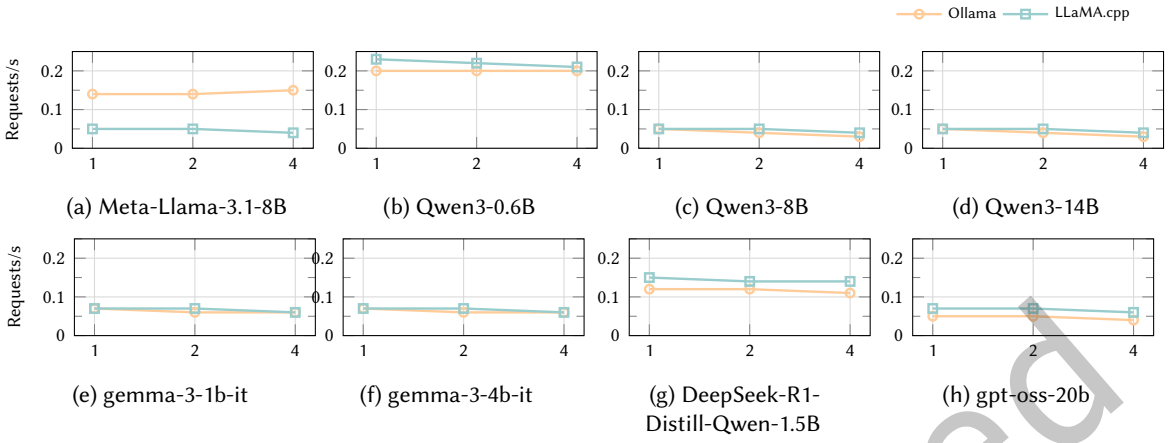


Fig. 42. Requests per second on edge devices with varying concurrency (Prompt length = 128, Output length = 512)

Just Accepted

as concurrency increases. This effect is particularly evident in Ollama, which shows stronger sensitivity to higher concurrency levels. As shown in Fig. 43, for Llama-3 8B, Ollama maintained an almost flat throughput of 42.9, 43.3, and 42.2 tokens/s at concurrency 1, 2, and 4, respectively. In contrast, LLaMA.cpp dropped from 35.0 to 32.0 and 25.6 tokens/s, showing notable degradation as concurrency increased. For this model, Ollama holds a clear advantage in token throughput. The trend reverses for the smaller Qwen3 0.6B model. LLaMA.cpp sustained 145.6, 143.3, and 137.2 tokens/s across concurrency levels 1, 2, and 4, demonstrating high stability. Meanwhile, Ollama decreased sharply from 117.6 to 63.3 and 31.8 tokens/s. At concurrency 4, the throughput gap widens beyond a factor of four, suggesting that Jetson's memory and cache hierarchy align more favorably with LLaMA.cpp for smaller models. For larger models such as Qwen3 8B and 14B, throughput dropped significantly for both engines. In Qwen3 14B, Ollama recorded 18.2, 12.0, and 7.3 tokens/s, whereas LLaMA.cpp achieved 19.1, 15.2, and 9.6 tokens/s, consistently maintaining a 20-30 percent advantage. A similar trend appeared in Gemma-3 1B, where LLaMA.cpp delivered 89.4, 87.0, and 81.0 tokens/s, while Ollama fluctuated between 71.6, 76.9, and 71.8 tokens/s. The same pattern persists in DeepSeek R1-1.5B and gpt-oss-20B. For DeepSeek-1.5B, LLaMA.cpp maintained 95.4, 92.5, and 86.6 tokens/s, whereas Ollama dropped sharply to 73.7, 39.3, and 19.7 tokens/s. In gpt-oss-20B, LLaMA.cpp achieved 45.1, 41.9, and 35.5 tokens/s, compared to Ollama's 37.2, 19.3, and 9.8 tokens/s, resulting in more than a threefold gap at concurrency 4.

End-to-End Latency. Request latency trends closely follow the TTFT and TBT characteristics described earlier. In Fig. 44 ,for Llama-3.1-8B, the average response latency of Ollama was 7.2 s, 13.7 s, and 24.8 s at concurrency levels 1, 2, and 4, respectively, whereas LLaMA.cpp recorded 18.3 s, 36.4 s, and 72.7 s, showing that latency nearly doubled whenever concurrency doubled. Smaller models such as Qwen3-0.6B exhibited notably shorter delays, with LLaMA.cpp measuring 4.4 s, 8.7 s, and 17.5 s under the same conditions. In contrast, mid- to large-scale models such as Qwen3-8B and Qwen3-14B experienced sharp latency increases, reaching 76-83 s and 134-142 s, respectively, at concurrency 4. For gpt-oss-20B, Ollama reported 18.9 s, 35.3 s, and 69.1 s, while LLaMA.cpp recorded 14.2 s, 28.4 s, and 56.7 s. These results indicate that serving large models under high concurrency on Jetson Orin is effectively infeasible. In practical edge deployments, concurrency must be limited to 1-2, and model size realistically restricted to the 1B-4B range to maintain acceptable latency.

Capacity under increasing concurrency. The request latency measured on the Jetson Orin closely reflects the previously observed TTFT and TBT characteristics. As shown in Fig. 45, for Llama-3 8B, Ollama recorded

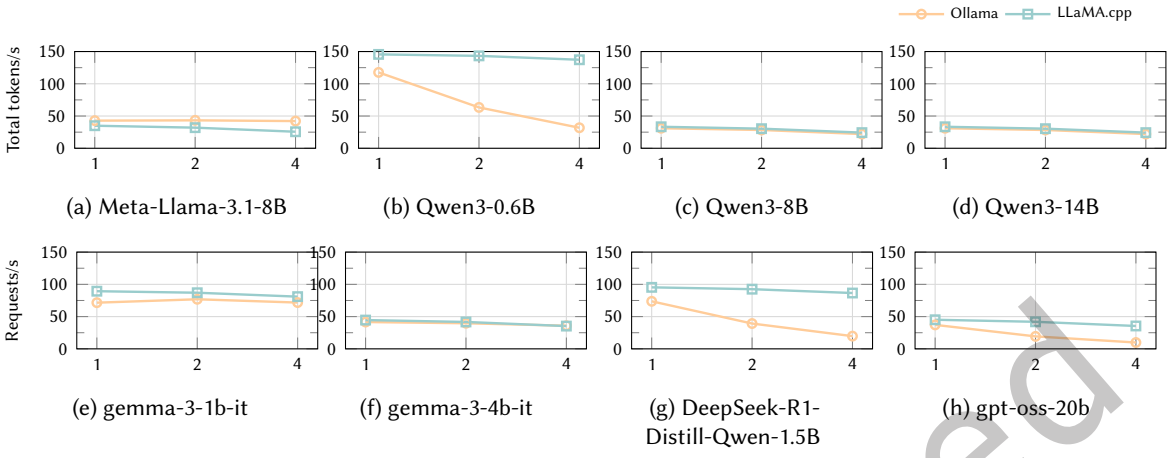


Fig. 43. Total tokens per second on edge devices with varying concurrency (Prompt length = 128, Output length = 512)

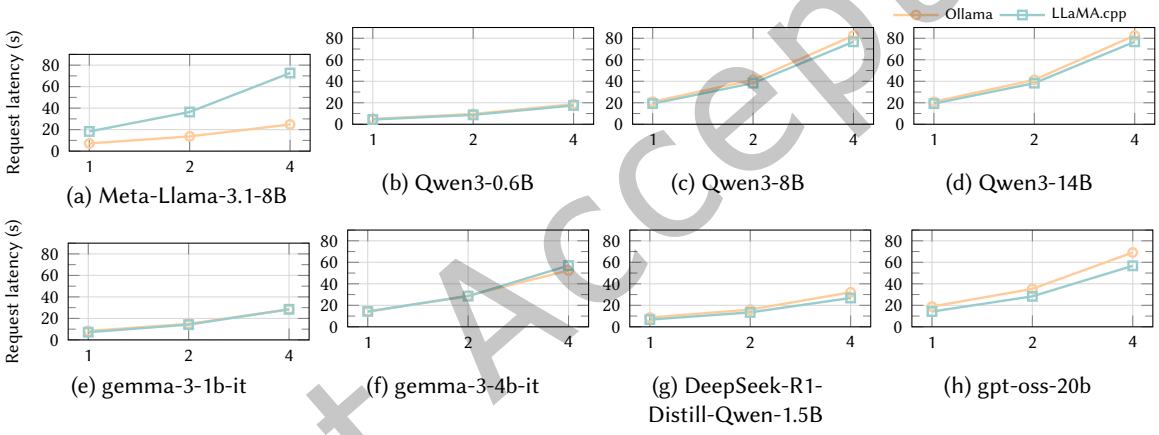


Fig. 44. Request Latency on edge devices with varying concurrency (Prompt length = 128, Output length = 512)

Just Accepted

average latencies of 7.2 s, 13.7 s, and 24.8 s at concurrency levels 1, 2, and 4, respectively. In contrast, LLaMA.cpp showed 18.3 s, 36.4 s, and 72.7 s, with latency nearly doubling each time concurrency doubled. For the small-scale Qwen3 0.6B, both engines generally exhibited short delays. LLaMA.cpp measured 4.4 s, 8.7 s, and 17.5 s at concurrency 1, 2, and 4, with Ollama showing similar values. However, as model size increases, latency grows sharply. At concurrency 4, Qwen3 8B and 14B reached average latencies of 76-83 s and 134-142 s, respectively. For gpt-oss-20B, Ollama recorded 18.9 s, 35.3 s, and 69.1 s, while LLaMA.cpp measured 14.2 s, 28.4 s, and 56.7 s across concurrency levels 1, 2, and 4-indicating that a single request can take from one to nearly two minutes to complete at concurrency 4. Overall, serving large models under high concurrency on the Jetson Orin is practically infeasible. The realistic operational range is limited to concurrency levels 1-2 with 1B-4B scale models, which offer manageable latency and consistent responsiveness.

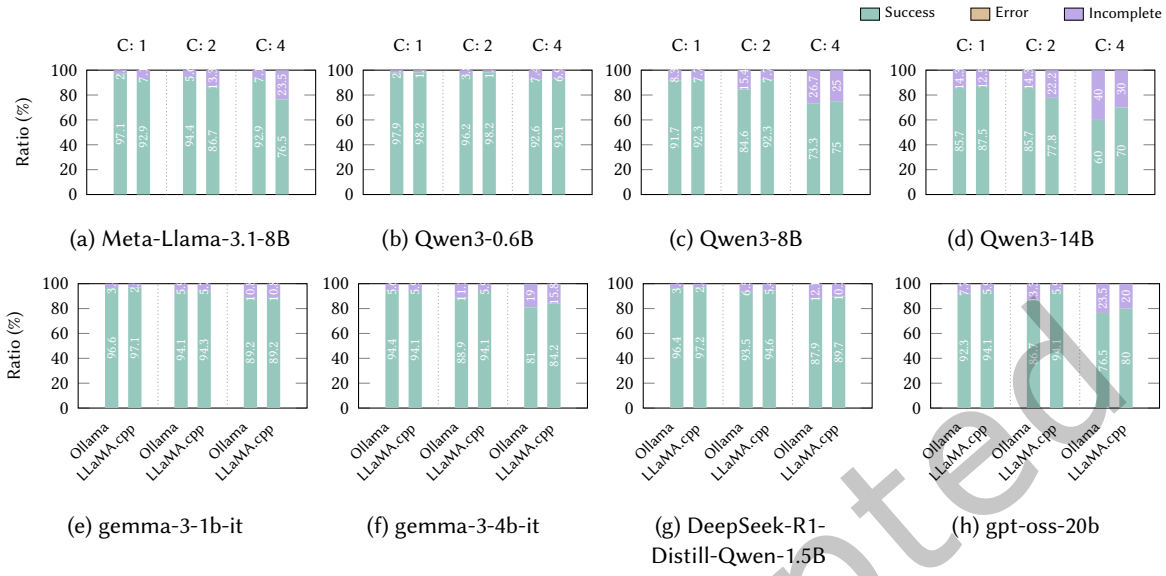


Fig. 45. Impact of concurrency scaling on inference reliability on edge device (4bit Quantized models, Prompt length = 128, Output length = 512)

Key Takeaways

- **Ollama excels for 4–8B models on edge devices:** On Jetson Orin AGX 32GB, Ollama consistently outperformed LLaMA.cpp for models such as Llama-3.1-8B and Gemma-3 4B, achieving lower TTFT, higher request/token throughput, and lower latency—making it well suited for interactive assistant-style workloads.
- **LLaMA.cpp performs better for sub-2B models:** For smaller models including Qwen-3 0.6B, Gemma-3 1B, DeepSeek R1 1.5B, and gpt-oss-20B, LLaMA.cpp delivered lower TTFT and higher token throughput. Its latency also scaled more smoothly up to concurrency 4, favoring batch-oriented, resource-efficient edge deployments.
- **Large models become impractical on Jetson hardware:** Once model size exceeds 7B, both engines exhibit TTFTs in the tens of seconds, while end-to-end latency exceeds one minute at concurrency 4. Success rates also decline to the 60–80% range, indicating severe performance constraints.
- **Implication for edge deployment:** Effective edge operation requires small-model-centric design and low concurrency. Large-scale models remain impractical in constrained edge environments due to latency, throughput, and stability limitations.

6.3 Limitations

In this chapter, we evaluated the performance of open-source LLM inference engines that support OpenAI API-compatible serving on both server-class hardware (NVIDIA H100 and A6000) and edge devices (NVIDIA Jetson AGX Orin). The key metrics considered were TTFT, TBT, Requests/s, and Total tokens/s. To emulate multi-user scenarios, we varied the concurrency level and output length, enabling a comparative analysis of engine-specific characteristics. Through these experiments, we assessed how well each engine sustains response rates and throughput under concurrent workloads.

However, this study primarily aims to provide initial performance indicators to inform inference engine selection, rather than a detailed system-level analysis. As such, hardware resource metrics such as CPU utilization, memory footprint, and network bandwidth usage, as well as in-depth analyses of the causes behind request

Just Accepted

success or failure rates, were not included. In particular, engines such as vAttention [268], which processed very few requests successfully, were not further investigated to identify root causes. In addition, as summarized in Table 5, we cataloged inference engines that support non-NVIDIA accelerators such as Google TPU [151] and AMD Instinct [300]. However, these hardware platforms were not used in the present experiments. This decision reflects the fact that NVIDIA GPUs are currently the most widely adopted accelerators in both server and edge deployment scenarios, and thus served as the primary basis for performance comparison in this study. These limitations will be addressed in future work.

7 Future Directions and Open Challenges

LLM inference engines continue to support new optimization methods and models, but to flexibly respond to the rapidly changing LLM ecosystem and diverse service requirements, the following additional considerations are necessary.

7.1 Extended Context Windows and Memory Management

There is a growing trend in LLMs toward handling extremely long context windows, ranging from tens of thousands to millions of tokens. For example, ChatGPT o1 [257] supports up to 128K tokens, Google Gemini 2.0 Flash [107] supports 1M tokens, and the recently introduced Llama 4 Scout [227] claims to handle up to 10M tokens.

This expansion leads to a dramatic increase in the **KV** cache size, posing significant challenges for memory management. To address this, techniques such as hierarchical cache management [397], partial offloading to CPU memory [171], and memory-efficient attention mechanisms [166] have been proposed. However, these methods are not yet sufficient to fully cope with the increasing context length, and further research is required.

To optimally handle long-context scenarios, multiple inference optimization techniques must be applied in combination. For example, PagedAttention [166] manages the **KV** cache to reduce internal fragmentation, while chunked prefill [373] divides long prompts into chunks to mitigate TTFT without sacrificing throughput. In addition, speculative decoding [45, 183] accelerates generation by verifying and accepting tokens proposed in advance by a draft model, thereby avoiding rollbacks. Efforts to compress the context itself are also actively explored. Selective-Context [181] reduces input length by removing redundancy or irrelevant information within the prompt, but this approach risks information loss and semantic distortion. To overcome these limitations, LLMLingua [150] employs a coarse-to-fine compression scheme. It incorporates a budget controller to adjust the target compression ratio, applies a token-level bidirectional compression algorithm to preserve interdependencies across compressed segments, and aligns semantic fidelity through instruction-tuning-based distribution matching. Experiments have demonstrated up to a 20× compression ratio without performance degradation.

In real-world services, multi-turn dialogues and streaming generation essentially require handling inputs of unbounded length [92]. However, tokens that exceed the context window not only degrade model quality but also cause the **KV** cache to grow explosively. Traditional sliding-window methods [35] retain only the most recent L tokens while recomputing the **KV** of earlier tokens, but this incurs substantial overhead. StreamingLLM [356] addresses this by fixing the **KV** of the initial tokens, maintaining only the most recent L tokens in a rolling cache, and discarding the rest. Combined with relative position encoding methods such as RoPE [307] or ALiBi [269], this design enables infinite-length inputs without additional training and achieves up to a 22.2× speedup compared to conventional sliding-window approaches. Nevertheless, since the context window size is not actually extended during training, its performance gains are limited for tasks requiring long-range dependencies, such as document summarization or long-term memory-based queries.

In the case of vLLM [166], when the input sequence length exceeds the maximum position embedding length supported by the model, the prompt is automatically split into multiple chunks. Each chunk is converted into an

Just Accepted

embedding vector using the model's original pooling strategy (either CLS token [161] or mean pooling). In the final step, chunk embeddings are aggregated into a single representation by computing their weighted average according to the number of tokens per chunk. This approach is simple to implement and computationally efficient, but its reliance on averaging limits the ability to capture cross-chunk contextual interactions.



Keywords Long-context processing, Memory-efficient attention, KV cache optimization, Context compression, Retrieval-based attention
Solutions
PagedAttention [166] and chunked prefill [373] optimize KV cache usage and TTFT, while speculative decoding [45, 183] speeds up generation. LLMLingua [150] compresses context effectively, and RetrievalAttention [199] or MoBA [214] adaptively balance sparsity and accuracy for long-context processing.

7.2 Complex Logical Reasoning

Modern LLMs are evolving beyond simple response generation toward performing complex logical reasoning. This includes guiding users step by step through problem solving processes, autonomously generating CoT reasoning [346], and interacting with external tools to complete tasks.

CoT prompting has become a key technique for improving accuracy and interpretability in complex mathematical, logic, and code problems, generalizing structured reasoning such as problem decomposition, proof generation, and verification to language agents [392]. In such scenarios, a large number of tokens may be consumed during intermediate reasoning steps, and multi-turn dialogues are often required to refine answers.

CoT induces long reasoning chains, which significantly increase the number of tokens in the decode phase, thereby causing quasi-linear growth in FLOPs and memory access. Since each token requires access to the **KV** cache, cache capacity and bandwidth bottlenecks become more severe.

To mitigate this, methods have been proposed to keep Keys in low-rank representations while offloading Values and dynamically reconstructing sparse **KV** caches as needed [308]. Additionally, long CoT requests can cause head-of-line blocking in the queue, worsening the TTFT for shorter interactive requests. This interference can be reduced by separating prefill and decode phases and batching them across heterogeneous devices [264].

An important thing to note is that overly verbose CoT outputs can lead to bloated responses rather than higher quality [241]. Therefore, it is necessary to optimize prompts using metrics such as correct-conciseness, and to design reward functions that encourage models to suppress unnecessary tokens autonomously.

Along with it, managing session continuity and context preservation becomes critical. In response, inference engines are being developed with support for streaming output [356] and multi-turn dialogue optimization [9, 92]. The ability to stably manage long token sequences and complex reasoning flows is becoming a key competency for LLM inference systems.



Keywords Chain-of-Thought (CoT), Logical Reasoning, KV Cache Optimization, Efficient Decoding, Reward Optimization
Solutions
Low-rank and sparse KV caching [308] and phase-splitting [264] reduce reasoning latency, while conciseness-aware reward modeling [241] improves CoT efficiency. Combined with streaming [356] and session management [9, 92], these enable stable long reasoning flows.

Just Accepted

7.3 Inference Engine Selection Based on Application Needs

The selection and design of LLM inference engines should be based on a balance between application requirements and system constraints. For applications like translation services or conversational agents where real-time interaction is critical, latency optimization is the top priority. On the other hand, server-side applications that must handle high-volume traffic will prioritize throughput maximization.

Looking ahead, it will be increasingly important to develop inference engines that support both hardware acceleration for multimodal data and general-purpose compatibility with diverse model architectures. Persistent challenges include optimizing memory for extended context windows, designing architectures that can flexibly handle complex reasoning, and developing strategies that strike the right balance between latency and throughput.

Meeting the accuracy and latency requirements demanded by applications and services cannot be achieved solely by enhancing the structure of inference engines, the model architecture itself must also be optimized. Compared to traditional CNNs or DNNs, LLMs exhibit a lower computational density relative to their parameter scale [235]. As a result, inefficiencies in computation and memory arise that cannot be fully resolved by quantization or pruning alone. Thus, low-rank decomposition has emerged as a key complementary technique [159].

Low-rank decomposition [146, 156, 381] reconstructs the weight matrices or tensors into lower-dimensional components, thereby reducing both memory usage and computational cost. A representative approach applies singular value decomposition (SVD) [341, 381] to retain only dominant singular values as an approximation, while another approach constrains rank during training via non-convex regularization [272]. In addition, tensor decomposition methods such as Tensor Train (TT) [273], Tensor Ring (TR) [125], and Tucker [162] represent high-order tensors as products of smaller cores. Within Transformer-based LLMs, applying these decompositions to attention weights and FFN weights can substantially reduce both computation and memory demands.

The low-rank decomposition can be applied in two primary stages. The first is during pre-training, where the layers are parameterized in a low-rank form so that the model maintains a low-rank structure throughout training [177, 222, 273]. The second is post-training decomposition applied to a pre-trained model, where SVD or tensor decomposition is used to adjust the rank of each layer and balance the trade-off between latency and accuracy. For example, CALDERA [282] progressively applies decomposition across multiple stages, yielding more stable results than one-shot compression [156].

The impact of low-rank decomposition on model performance depends on the model architecture, data, and chosen rank value. Studies on the accuracy-efficiency trade-off show that applying Tucker decomposition to the Llama 2 family reduces model size by approximately 9% while maintaining precision losses within 4.5 to 10% points, thereby lowering both latency and energy consumption [235].

To effectively apply low-rank decomposition in real-world inference services, the decomposition scheme and kernel implementation must be co-designed in accordance with the memory hierarchy and computational units of the target hardware. First, when determining the dimension along which matrices or tensors are decomposed, the rank value must be selected with consideration of micro-level resource constraints such as GPU/NPU warp size, memory bank organization, and shared memory capacity. Second, although the decomposed low-dimensional matrix multiplications are computationally lighter, the increased number of calls may lead to excessive kernel launch overhead and frequent accesses to global memory. To mitigate this, multiple multiplications should be fused within a single kernel or repacked into tensor-core-friendly block structures arranged within contiguous memory ranges. Finally, the execution scheduler should analyze the computation graph before and after decomposition and reorder operations so that parts with low data reuse are retained in shared memory or registers, thereby alleviating memory bandwidth bottlenecks. In summary, optimal low-rank decomposition yields maximum benefit only when the decomposition algorithm, kernel fusion strategy, and scheduling policy are integrated with hardware characteristics.

Just Accepted

Thus, a low-rank decomposition at the model level complements inference engine optimizations. For instance, if inference engines supporting post-training quantization, such as Unsloth [325], additionally provide low-rank decomposition modules, LLMs could be executed more efficiently even on personal devices or edge hardware.



Keywords Inference Engine Optimization, Application-driven Design, Low-rank Decomposition, Latency-Throughput Trade-off, Hardware-aware Kernel Co-design
Solutions
Low-rank decompositions such as SVD, TT, and Tucker [125, 162, 273, 341] reduce model cost with minimal accuracy loss. Progressive compression [282] and kernel fusion tuned to hardware improve throughput, while integration with quantization engines like Unsloth [325] enhances edge efficiency.

7.4 Increasing Importance of LLM Alignment

As LLMs spread across various domains and services, LLM alignment [147] has become as important as the accuracy of the model. Alignment guides the outputs toward responses that users prefer and that meet policy or style rules. Fine-tuning usually improves prediction accuracy on tasks with ground truth, while alignment pursues broader goals such as usefulness, policy compliance, and domain tone. Thus, fine-tuning helps answer oriented tasks, and alignment covers those tasks while adding higher-level constraints.

Several alignment techniques have been widely studied. First, after Supervised Fine-Tuning (SFT) [294], Reinforcement Learning with Human Feedback (RLHF) [61, 400] trains a reward model [333] using human-preference pairwise data and then optimizes the policy using Proximal Policy Optimization (PPO) [359]. Second, RL from AI Feedback (RLAIF) [170]/Constitutional AI [175, 347] that replaces human feedback with an LLM-based judge to generate preference data and perform alignment accordingly. Third, Direct Preference Optimization (DPO) [359], which directly optimizes the policy from preference pairs without a separate reward model or PPO, has recently gained significant attention.

Aligning LLMs, which often have billions or even trillions of parameters, requires a downstream phase after supervised or instruction fine-tuning. This alignment phase uses preference-based methods such as RLHF, RLAIF/Constitutional AI with AI feedback, or DPO to embed user or organizational policies. Because the process requires large amounts of hardware, several large-scale alignment frameworks have appeared.

Verl [289] combines PPO-based RLHF, DPO, and AI feedback in a single pipeline after fine-tuning and supports parallel training at scale. LlamaRL [351] is a light alignment framework tailored to LLaMA models that allows researchers to run quick tests on small GPU configurations. LlamaRL is less suitable for very large distributions, memory or communication optimization, and flexible pipelines. Other tools, including Transformer Reinforcement Learning (TRL) [140], OpenRLHF [134], and DeepSpeed-Chat [371], also support alignment training.

LLM Alignment remains crucial at inference time. An aligned model better matches user intent and policy, reducing post-processing and retries. It also produces outputs with more stable formats and lengths, easing batch processing and scheduling. Even so, alignment does not change the parameter count, so inference engines still need techniques such as quantization, caching, and careful batch scheduling to meet real-time service goals.



Keywords LLM Alignment, RLHF, RLAIF, DPO, Preference Optimization, Human/AI Feedback, Policy Compliance
Solutions
RLHF [61, 400], RLAIF [170], and DPO [359] align models with user preferences and policies. Frameworks like Verl [289], LlamaRL [351], and DeepSpeed-Chat [371] scale alignment efficiently, ensuring consistent, compliant outputs at inference time.

Just Accepted

7.5 Hardware-Aware Fusion and Mixed-Precision Kernels for Efficiency

In traditional AI workloads that primarily relied on convolution operations, optimization was often achieved through simple operator fusions, such as ReLU. However, in the era of generative AI based on Transformers and diffusion models, more sophisticated fusion strategies tailored to specific hardware architectures are required. A representative example is FlashAttention-3 [285], which is highly optimized for NVIDIA's H100 hardware. These complex fusion techniques involve advanced tiling strategies, carefully designed by expert engineers to align with the hardware's shared memory and cache size constraints.

Low-precision microscaling data types, such as MXFP4, can accelerate GEMM operations while simultaneously reducing both training and inference costs. The NVIDIA Blackwell architecture natively supports multiple low-precision formats, including FP4, MXFP4 [279], and NVFP4 [57]. Since the training process of LLMs also relies heavily on large-scale matrix multiplications, a recent study has proposed leveraging MXFP4 to perform training without accuracy degradation [322]. This study computes low-bias, low-variance gradient estimates, enabling more accurate parameter updates, and applies the Random Hadamard transform to mitigate the impact of rare outliers on the overall training process. As a result, when pretraining a 6.7B-parameter GPT model, they achieved an accuracy virtually identical to that obtained using BF16 mixed-precision.

OpenAI's open-source model gpt-oss [8] is an autoregressive MoE Transformer based on the GPT-2 [274] and GPT-3 [43] architectures, supporting both CoT reasoning and structured output generation. It employs RoPE for positional encoding and can process up to 128K tokens of context. This model reduces memory requirements substantially by quantizing MoE weights into the MXFP4 format through post-training quantization, storing them at approximately 4.25 bits per parameter.

With the growing adoption of MoE and the continued expansion of LLM scales, the use of microscaling data types is expected to become increasingly important. Currently, vLLM [166] provides limited support for MXFP4, and effectively deploying such ultra-low-precision formats in LLM services requires kernel-level optimizations within inference engines that are deeply aligned with hardware architectures.

Moreover, as shown in Table 11, generative AI models demand support for a wide range of data type precisions to reduce model size while preserving accuracy. Therefore, it is essential to develop operator kernels that can flexibly and efficiently handle mixed-precision computation.



Keywords Hardware-aware Fusion, Mixed-Precision Kernels, Microscaling Data Types, FP4/MXFP4/NVFP4, FlashAttention, Low-Precision Optimization
Solutions FlashAttention-3 [285] maximizes GPU memory efficiency, while MXFP4 and NVFP4 [57, 279] enable faster, smaller models. MXFP4 training [322] maintains BF16-level accuracy, and MoE quantization [8] further reduces cost. Hardware-aligned kernels in engines like vLLM [166] are key for FP4 deployment.

7.6 Support for On-Device Inference

Most LLM services perform inference using large-scale resources in cloud or data center environments, delivering the results to users. Although this approach enables fast computation, it is network-dependent and requires transmitting user data to servers, raising privacy concerns. As a result, demand for on-device (or on-premise) LLM inference on edge and mobile devices has been increasing.

Traditionally, LLM models were too large to run on a single device, but the emergence of small language models (SLMs) such as Llama 3.2 [109], Gemma [314], Phi-3 [2], and Pythia [39] has enabled LLM execution on embedded systems, mobile devices, and IoT systems, as well as single-GPU environments.

Since edge environments (e.g., embedded and mobile systems) have lower hardware specifications than servers, both model compression and hardware-specific parallelization and memory optimizations are essential. For

Just Accepted

example, mobile inference optimizations include tolerance-aware compression, I/O recomputation pipeline loading, and chunk life cycle management [377]. Research has also explored collaborative inference, where multiple edge devices share computational workloads [384]. For single GPU environments, 4-bit quantization and memory offloading techniques that distribute weights, activations, and **KV** caches across CPU, disk, and other memory resources [292] are being investigated. These advances help reduce server power consumption and enable personalized models and inference in network-limited environments. However, edge devices vary significantly by manufacturer and environment, making generalized optimization difficult. Additionally, developing compiler and hardware-dependent transformation tools incurs additional development costs.

In on-device inference environments, achieving efficiency requires not only optimization techniques and supporting inference engines but also lightweight yet high-performing models. Several models such as DistilBERT [284], the DeepSeek-R1-Distill family [120], and DistilQwen-2.5 [334] have been released, which use KD [364] to reduce the number of parameters while maintaining accuracy close to that of the original LLM.

KD is a representative model compression technique in which a large teacher model transfers its trained knowledge to a smaller student model. Through this process, smaller models can be trained to approximate the performance of larger models, thus achieving strong accuracy with a relatively compact size. This approach enables straightforward training of domain-specific models suited for on-device environments and helps narrow the performance gap between proprietary large-scale models and open-source models [360].

There are various ways to extract knowledge from teacher models: using teacher output labels directly as supervision (`Labeling`), generating additional input-output pairs from a few examples (`Expansion`), transferring probability distributions or intermediate representations (`Feature`), synthesizing data from external meta-information (`Data curation`), having the teacher provide feedback on student outputs (`Feedback`), or adopting self-knowledge methods where the student filters and refines its own outputs (`Self-Knowledge`) [360]. To transfer such knowledge into student models, fine-tuning can be employed with supervised or semi-supervised learning [7, 226], divergence-based loss for distribution alignment [114, 366], or reinforcement learning-based policy optimization [187, 215] can be employed.

When internal weights, logits, or attention values of the teacher model are accessible, white-box distillation [206, 298] can be applied to achieve fine-grained distribution matching and structural preservation. In this case, it is easier to bring student performance close to that of the teacher, though transfer efficiency may degrade when there is a large-scale gap between the teacher and student. In contrast, in black-box environments, such as commercial APIs where internal states are inaccessible, distillation is based solely on the final output [335, 367]. Sequence-level imitation learning and self-training methods fall into this category, but the absence of logits or intermediate representations often limits the performance gain.

KD can be applied either during the fine-tuning stage of a pre-trained model or throughout the entire pre-training process. For inference engines to support KD, they must be able to handle loss computation and backpropagation within the training loop. Alternatively, lightweight distillation techniques, such as zero-shot or few-shot prompt-based Self-Instruct distillation or prompt-supervised generation, rely only on the teacher's output. These methods enable the creation of student models without requiring additional computation, even in engines that do not support training functionalities.

Among existing LLM inference engines, llama.cpp [101], MLC LLM [234], and TensorRT-LLM [252] offer partial support for edge environments. llama.cpp [101], implemented in C/C++, is highly portable across different platforms. MLC LLM [234] uses the TVM [51] compiler to support GPU, mobile, and web environments, although its hardware compatibility is limited. TensorRT-LLM [252] supports only specific edge devices, such as NVIDIA Jetson series.

Just Accepted



Keywords On-device LLM Inference, Edge/Mobild device optimization, Model Compression
Solutions Small language models (SLMs) (Llama 3.2 [109], Gemma [314], Phi-3 [2]), tolerance-aware compression, I/O recomputation, and chunk life cycle management [377], collaborative inference [384], knowledge distillation [364]

7.7 Support Diverse Hardware for Inference Optimization

LLM inference, which was traditionally centered around NVIDIA GPUs, is now expanding to heterogeneous hardware with the emergence of TPU [151], Neural Processing Units (NPUs), and various LLM accelerators. In addition to widely used AWS Inferentia [17] and Google TPU [151], new accelerators such as AMD Instinct MI300X [300], Furiosa AI RNGD (Tensor Contraction Processor) [160], and Cerebras CS-2 (WSE-2) [189] are being developed. Furthermore, next-generation memory technologies such as Processing-in-Memory (PIM) [157, 259] are also under development.

To support heterogeneous hardware, engines must incorporate pipeline execution, batch optimization, and load balancing. However, differences in performance, synchronization, and communication overhead across hardware types can pose challenges. Research has explored LLM inference optimizations in heterogeneous GPU clusters using Phase-Aware Partitioning and Adaptive Quantization [393] and hardware-aware allocation of prefill and decode processes for optimized inference [264]. Furthermore, techniques such as sub-batch interleaving [128] have been proposed to optimize inference across systems with multiple NPUs and PIM devices.

To optimize LLM inference on accelerators beyond GPUs, studies [119, 152, 176] have also analyzed the characteristics of CPUs, GPUs, FPGAs, ASICs, and PIM or NDP platforms. Especially in [176], by synthesizing previous work, the reasearch summarized hardware-specific optimization strategies for both the prefill and decode phases and compared throughput (tokens per second) relative to power consumption. In particular, they examined in detail how optimizations such as quantization, sparsity, and fast decoding (e.g., speculative decoding) exhibit different performance behaviors across hardware depending on batch size. Based on the findings, the research paper argued that addressing recent inference trends such as longer input sequences and expanded prefill phase requires hardware-software-algorithm co-design to simultaneously achieve real-time responsiveness, high-throughput, and resource efficiency.

Moreover, most accelerators are accompanied by dedicated compilers and frameworks to generate hardware-friendly code. For example, Google TPU [151] leverages the XLA [281] compiler and JAX [88], while Groq LPU [4] provides a dedicated software stack consisting of GroqWare/GroqFlow [110], and the LPU Inference Engine. Although attempts have been made to support multiple accelerators beyond NVIDIA GPUs, such as AMD MI300X [300], Google TPU [151], and Huawei Ascend [188] by vLLM [166], many inference engines still provide official support only for a limited set of hardware. Integrating new accelerators requires coordination across engine-level optimizations, runtimes, compilers, and libraries, which often entails significant delays before official repository integration. Consequently, it has become increasingly common for hardware vendors to adapt existing engines and provide wrappers or bindings tailored to their accelerators. Realizing broad hardware support thus necessitates these preparatory steps.



Keywords Heterogeneous Hardware, LLM Accelerator, Hardware-Aware Optimization, Compiler and Runtime Integration
Solutions Phase-Aware Partitioning and Adaptive Quantization [393], hardware-aware allocation of prefill and decode processes [264], sub-batch interleaving for multi-NPU and PIM inference [128], hardware-specific optimization across CPUs, GPUs, FPGAs, ASICs, and PIM/NDP platforms [119, 152, 176]

Just Accepted

7.8 Requirements for Multimodal LLMs

Most current LLM inference engines are optimized for text-based models. However, relying solely on text has limitations when processing information. To achieve human-level intelligence, it is essential to support multiple data types such as images, audio, and video. In response to this, multimodal models like Qwen2-VL [338] and LLaVA-1.5 [200] have been developed. To support such models effectively, inference engines must be designed to handle multimodal data preprocessing and multi-stream parallel execution efficiently.

In this context, existing model compression techniques, such as quantization must be adapted to preserve modality-specific information while still reducing model size. Software-level methods like hybrid parallelization are not sufficient on their own. Therefore, new hardware-accelerated kernels and decoding methods, such as speculative decoding tailored for multimodal inputs, need to be considered.

A good example of adapting model architecture to hardware for multimodal tasks is Multimodal Rotary Position Embedding (M-RoPE), introduced in Qwen2-VL[338] and LLaVA-1.5 [200]. M-RoPE extends the traditional positional embedding used in Transformer models to more effectively capture the positional relationships in various multimodal inputs.



Keywords Multimodal LLM Inference, Multimodal Model Compression, Hardware-Accelerated Decoding
Solutions Multimodal models (Qwen2-VL [338] and LLaVA-1.5 [200]), multimodal data preprocessing and multi-stream parallel execution support in inference engines, modality-preserving quantization for model compression, hardware-accelerated kernels and speculative decoding for multimodal inputs, Multimodal Rotary Position Embedding (M-RoPE) [200, 338]

7.9 Alternative Architectures Beyond Transformers

While Transformer-based models still dominate LLM inference, new architectures tailored for multimodal LLM are emerging. Models like RetNet [312] and RWKV [265] propose alternatives to the standard Transformer design. Another notable direction is Mamba [113], a sequence modeling architecture developed to overcome the limitations of Transformers. Mamba uses a Selective State Space Model (SSM) to process long sequences more efficiently, achieving linear time complexity without relying on the standard attention mechanism.

Jamba [190] is a hybrid model that combines the Mamba [113] and Transformer architectures, aiming to take advantage of both. It also integrates a MoE strategy that increases model capacity while keeping the number of active parameters manageable during inference.

IBM Granite 4.0 [142] is a hybrid architecture that combines Mamba [113] and Transformer [327] models, achieving over 70% reduction in memory usage while maintaining performance comparable to conventional Transformer-based models. It alleviates the quadratic computational growth of traditional Transformers with respect to sequence length by leveraging Mamba's linear scaling, thereby enabling efficient processing of long contexts without compromising in-context learning capability. Because of its Mamba-based structure, positional embeddings are also unnecessary. Granite 4.0 is distributed under the Apache 2.0 open-source license and has obtained ISO 42001 certification. The model family includes Granite-4.0-H-Small with 32B parameters, Granite-4.0-H-Tiny, a 7B MoE hybrid model, Granite-4.0-H-Micro with a 3B hybrid configuration, and Granite-4.0-Micro, a pure 3B Transformer variant. It can operate across diverse hardware platforms, including AMD Instinct MI300X [300] and Qualcomm Hexagon NPU [221], supporting inference in both server and edge environments. Furthermore, Granite 4.0 is compatible with inference engines such as Ollama [255], vLLM [166], and llama.cpp [101].

These trends highlight the growing need for general-purpose inference engines that can support diverse architectures. Future inference systems must not only be optimized for internal operations of Transformer models but also be scalable and flexible enough to support new and evolving model structures.

Just Accepted



Keywords Transformer Alternatives, Hybrid Models, MoE
Solutions Selective State Space Models (SSM) (RetNet [312], RWKV [265], Mamba [113]), hybrid architectures (Jamba [190], IBM Granite 4.0 [142])

7.10 Security Support in Inference

During LLM inference, vulnerabilities such as prompt injection attacks, jailbreak attacks, and data leaks have emerged [370]. Prompt injection attacks occur when an attacker manipulates inputs to override the model's system prompts or objectives. In environments handling sensitive data, such as finance and healthcare, personal data exposure risks are significant. Additionally, if malicious attacks generate abnormal or harmful data, it can severely impact user experience and system stability.

To mitigate these risks, robustness techniques, such as adversarial training [203] can be applied during the model training phase. During inference, tools like OpenAI Moderation [223], instruction manipulation prevention, and input sanitization methods [239, 243] can be used to block harmful or malicious inputs.

From a service security perspective, role-based access control (RBAC) and multi-factor authentication (MFA) can be implemented to prevent unauthorized access. In addtion, access tokens can be set to expire after a certain period to enhance security policies.

Currently, most LLM inference engines focus primarily on performance and do not include dedicated security features. However, they aim to reduce risks through methods such as data filtering and strengthened ethical policies.



Keywords Prompt Injection, Jailbreak Attack, Data Leakage, Model Robustness, Security in LLM Inference
Solutions Adversarial Training [203], OpenAI Moderation [223], Instruction Manipulation Prevention, Input Sanitization [239, 243], Role-Based Access Control (RBAC), Multi-Factor Authentication (MFA), Access Token Expiration, Data Filtering, Ethical Policy Enforcement

7.11 Support for Cloud Orchestration and Multi-node Serving Platforms

Cloud orchestration and serving strategies are critical for LLM inference services. When deploying large-scale inference services in the cloud, orchestration platforms such as Kubernetes [44] enable autoscaling, hardware resource monitoring (Prometheus [324], Grafana [47]), and failover recovery. To facilitate this, inference engines should provide containerized environments, multi-node deployment, and load balancing tools that allow easy configuration based on service requirements and SLOs.

Most LLM inference engines offer built-in serving functionalities, but large-scale serving systems require additional workload distribution, scheduling, and autoscaling optimizations. vLLM [166], TensorRT-LLM [252], DistServe [402], and Sarathi-Serve [402] utilize Ray [240] to support distributed runtime and serving. Additionally, TensorRT-LLM [252] integrates with NVIDIA Triton Inference Server [248] and NVIDIA Dynamo [253] for model deployment and execution, while TGI [139] enables model deployment via Hugging Face Spaces [138].

With the growing adoption of MoE models [46] and multi-agent [179] inference environments, serving platforms are rapidly expanding beyond single devices or nodes to multi-device and multi-node architectures. MegaScale-Infer [406], proposed for efficient large-scale MoE model serving, addresses the problem that as model size increases, the number of experts grows and sparsity intensifies, resulting in fewer tokens being assigned to each expert within a batch and consequently lower GPU utilization of the FFN modules. This framework disaggregates the attention and FFN modules and employs a ping-pong pipeline parallelism strategy that overlaps the two computations to hide communication latency. As a result, it achieves up to 1.9× higher decoding throughput, 4.2× higher communication throughput compared to NCCL, and 68% lower latency on an 8-node cluster of NVIDIA

Just Accepted

A100 GPUs connected via NVLink, as well as on a heterogeneous cluster consisting of NVIDIA H20 and L40S GPUs.

In complex multi-agent systems, **KV** cache sharing between models often becomes a major bottleneck. To alleviate this, KVCOMM [372] defines offsets that allow reuse of overlapping **KV** cache segments across different prefix regions without retraining, thereby reducing prefill latency. Another research, Cache-to-Cache [90], projections, fuses **KV** caches between source and target models, significantly reducing inter-model data transfer overhead.

In multi-node environments scaled across tens to thousands of GPUs, low-latency inter-node communication is essential for efficiently handling user requests. To address this requirement, the collective communication framework NCCLX [299] has been proposed as an enhancement to NCCL. NCCLX introduces a host-based customizable transport layer (CTran) that enables zero-copy, SM-free transmission, and fault-tolerant All-Reduce operations. As a result, NCCLX achieves up to 2.7× higher throughput compared to NCCL's copy-based communication and reduces latency by up to 1.57× in tensor parallel workloads, which are critical for inference. Furthermore, in multi-node inference settings, NCCLX introduces the AllToAllvDynamic operation as a replacement for traditional AlltoAll computation, yielding up to 43% performance improvement.

As LLM inference environments continue to evolve toward multi-node and heterogeneous device architectures, existing inference engines must incorporate capabilities such as distributed expert placement, cache-sharing optimization, and scalable communication layers to meet the demands of real-world service deployment.



Keywords Cloud Orchestration, Multi-node/-agent LLM Serving, Autoscaling, Load Balancing, Distributed Inference, Containerization
Solutions Kubernetes [44], Prometheus [324], Grafana [47], Ray [240], NVIDIA Triton Inference Server [248], NVIDIA Dynamo [253], Hugging Face Spaces [138], KVCOMM [372], Cache-to-Cache [90], NCCLX [299]

8 Conclusion

This paper systematically analyzed the optimization methods and hardware adaptation strategies of LLM inference engines. First, we identified the memory and computation bottlenecks of decoder-only transformers and summarized mitigation methods such as batching, parallelism, caching, and compression. Second, we classified 25 open-source and commercial inference engines along two axes, single-node versus multi-node, and homogeneous versus heterogeneous device support, and compared their architectural goals and supported hardware. In particular, we analyzed the inference engine with a focus on ease-of-use, ease-of-deployment, general-purpose support, scalability, throughput-aware, and latency-aware. Our analysis showed that selecting an inference engine required balancing multiple factors, including latency-throughput trade-offs, hardware diversity, inference engine-level optimization support, and SLO. Additionally, we outlined future directions that included multi-agent, multimodal inference support, alternative transformer architectures, longer context windows, improved logical reasoning, application-specific design, stronger security, on-device execution, heterogeneous acceleration, and cloud orchestration. In general, this study provided a practical foundation for designing and operating next-generation inference infrastructure.

Just Accepted

Acknowledgments

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No. RS-2024-00402898, Simulation-based High-speed/High-Accuracy Data Center Workload/System Analysis Platform). This paper was supported by research funds for newly appointed professors of Jeonbuk National University in 2026.

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 265–283.

[2] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219* (2024).

[3] Dennis Abts, Garrin Kimmell, Andrew Ling, John Kim, Matt Boyd, Andrew Bitar, Sahil Parmar, Ibrahim Ahmed, Roberto DiCecco, David Han, et al. 2022. A software-defined tensor streaming multiprocessor for large-scale machine learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 567–580.

[4] Dennis Abts, Jonathan Ross, Jonathan Sparling, Mark Wong-VanHaren, Max Baker, Tom Hawkins, Andrew Bell, John Thompson, Temesghen Kahsai, Garrin Kimmell, et al. 2020. Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 145–158.

[5] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[6] Abien Fred Agarap. 2018. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375* (2018).

[7] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *The twelfth international conference on learning representations*.

[8] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).

[9] Saurabh Agarwal, Anyong Mao, Aditya Akella, and Shivaram Venkataraman. 2024. SYMPHONY: Improving Memory Management for LLM Inference Workloads. *arXiv preprint arXiv:2412.16434* (2024).

[10] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.

[11] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).

[12] AICPA & CIMA. 2022. SOC 2® Reporting on an Examination of Controls at a Service Organization Relevant to Security, Availability, Processing Integrity, Confidentiality, or Privacy. E-book.

[13] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 4895–4901.

[14] Yash Akhauri, Ahmed F AbouElhamayed, Jordan Dotzel, Zhiru Zhang, Alexander M Rush, Safeen Huda, and Mohamed S Abdelfattah. 2024. Shadowllm: Predictor-based contextual sparsity for large language models. *arXiv preprint arXiv:2406.16635* (2024).

[15] Ahsan Ali, Riccardo Pinciroli, Feng Yan, and Evgenia Smirni. 2020. Batch: Machine learning inference serving on serverless platforms with adaptive batching. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.

[16] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, et al. 2023. The falcon series of open language models. *arXiv preprint arXiv:2311.16867* (2023).

[17] Amazon Web Services. 2018. AWS Inferentia. https://aws.amazon.com/ai/machine-learning/inferentia/

[18] Amazon Web Services. 2024. Amazon SageMaker AI. https://aws.amazon.com/sagemaker-ai/

[19] Amazon Web Services. 2024. SwiftChat - A Cross-platform AI Chat App. https://github.com/aws-samples/swift-chat

[20] AMD. 2025. AMD Quark Model Optimizer. https://github.com/amd/Quark

[21] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.

[22] Anaconda, Inc. 2012. Conda: OS-agnostic, system-level binary package and environment manager. https://anaconda.org/anaconda/conda

[23] Andrej Karpathy. 2022. nanoGPT. https://github.com/karpathy/nanoGPT

[24] Andrej Karpathy. 2023. llama2.c: Inference Llama 2 in one file of pure C. https://github.com/karpathy/llama2.c

[25] AI Anthropic. 2024. The claude 3 model family: Opus, sonnet, haiku. *Claude-3 Model Card* 1 (2024), 1.

[26] Anthropic PBC. 2023. Claude. https://claude.ai/ March 12, 2025.

Just Accepted

[27] Apple Inc. 2017. Core ML. https://developer.apple.com/documentation/coreml/
[28] Artificial Analysis. 2024. Artificial Analysis: AI Model & API Providers Analysis. https://artificialanalysis.ai/ March 12, 2025.
[29] Astral. 2024. uv - An extremely fast Python package and project manager, written in Rust. https://github.com/astral-sh/uv
[30] AutoGPT. 2023. AutoGPT: Build, Deploy, and Run AI Agents. https://agpt.co/
[31] Axolotl. 2023. Axolotl: Open Source LLM Post-Training. https://github.com/axolotl-ai-cloud/axolotl
[32] Guangji Bai, Zheng Chai, Chen Ling, Shiyu Wang, Jiaying Lu, Nan Zhang, Tingwei Shi, Ziyang Yu, Mengdan Zhu, Yifei Zhang, et al. 2024. Beyond efficiency: A systematic survey of resource-efficient large language models. *arXiv preprint arXiv:2401.00625* (2024).
[33] Oana Balmau, Anne-Marie Kermarrec, Rafael Pires, André Loureiro Espírito Santo, Martijn de Vos, and Milos Vujasinovic. 2025. Accelerating MoE Model Inference with Expert Sharding. In *Proceedings of the 5th Workshop on Machine Learning and Systems*. 192–199.
[34] Shraddha Barke, Emmanuel Anaya Gonzalez, Saketh Ram Kasibatla, Taylor Berg-Kirkpatrick, and Nadia Polikarpova. 2024. Hysynth: Context-free llm approximation for guiding program synthesis. *Advances in Neural Information Processing Systems* 37 (2024), 15612–15645.
[35] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020).
[36] BentoML. 2019. BentoML: Unified Inference Platform for any model, on any cloud. https://www.bentoml.com/
[37] BentoML. 2023. OpenLLM: Self-Hosting LLMs Made Easy. https://github.com/bentoml/OpenLLM
[38] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954* (2024).
[39] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*. PMLR, 2397–2430.
[40] bitsandbytes-foundation. 2021. bitsandbytes. https://github.com/bitsandbytes-foundation/bitsandbytes
[41] Urban Borštnik, Joost VandeVondele, Valéry Weber, and Jürg Hutter. 2014. Sparse matrix multiplication: The distributed block-compressed sparse row library. *Parallel Comput.* 40, 5-6 (2014), 47–58.
[42] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. 2023. Striped attention: Faster ring attention for causal transformers. *arXiv preprint arXiv:2311.09431* (2023).
[43] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
[44] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. Borg, omega, and kubernetes. *Commun. ACM* 59, 5 (2016), 50–57.
[45] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774* (2024).
[46] Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. 2024. A survey on mixture of experts. *arXiv preprint arXiv:2407.06204* (2024).
[47] Mainak Chakraborty and Ajit Pratap Kundan. 2021. Grafana. In *Monitoring cloud-native applications: Lead agile operations confidently using open source software*. Springer, 187–240.
[48] Karam Chatha. 2021. Qualcomm® Cloud AI 100: 12TOPS/W scalable, high performance and low latency deep learning inference accelerator. In *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 1–19.
[49] Hailin Chen, Fangkai Jiao, Mathieu Ravaut, Nawshad Farruque, Xuan Phi Nguyen, Chengwei Qin, Manan Dey, Bosheng Ding, Caiming Xiong, Shafiq Joty, et al. 2024. StructTest: Benchmarking LLMs' Reasoning through Compositional Structured Outputs. *arXiv preprint arXiv:2412.18011* (2024).
[50] Mengzhao Chen, Wenqi Shao, Peng Xu, Jiahao Wang, Peng Gao, Kaipeng Zhang, and Ping Luo. 2024. Efficientqat: Efficient quantization-aware training for large language models. *arXiv preprint arXiv:2407.11062* (2024).
[51] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
[52] Tianlong Chen, Zhenyu Zhang, Ajay Jaiswal, Shiwei Liu, and Zhangyang Wang. 2023. Sparse moe as the new dropout: Scaling dense and self-slimmable transformers. *arXiv preprint arXiv:2303.01610* (2023).
[53] Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. 2023. Universal self-consistency for large language model generation. *arXiv preprint arXiv:2311.17311* (2023).
[54] Yanxi Chen, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. 2023. Ee-llm: Large-scale training and inference of early-exit large language models with 3d parallelism. *arXiv preprint arXiv:2312.04916* (2023).
[55] Yunfei Cheng, Aonan Zhang, Xuanyu Zhang, Chong Wang, and Yi Wang. 2024. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919* (2024).

Just Accepted

[56] Krishna Teja Chitty-Venkata, Sparsh Mittal, Murali Emani, Venkatram Vishwanath, and Arun K Somani. 2023. A survey of techniques for optimizing transformer inference. *Journal of Systems Architecture* 144 (2023), 102990.
[57] Brian Chmiel, Maxim Fishman, Ron Banner, and Daniel Soudry. 2025. FP4 All the Way: Fully Quantized Training of LLMs. *arXiv preprint arXiv:2505.19115* (2025).
[58] Conainers. 2018. Podman: A tool for managing OCI containers and pods. https://github.com/containers/podman
[59] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2017. Clipper: A {Low-Latency} online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 613–627.
[60] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066* (2024).
[61] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. 2023. Safe rlhf: Safe reinforcement learning from human feedback. *arXiv preprint arXiv:2310.12773* (2023).
[62] Penglin Dai, Biao Han, Ke Li, Xincao Xu, Huanlai Xing, and Kai Liu. 2024. Joint optimization of device placement and model partitioning for cooperative DNN inference in heterogeneous edge computing. *IEEE Transactions on Mobile Computing* (2024).
[63] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
[64] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
[65] Sajal Dash, Isaac R Lyngaas, Junqi Yin, Xiao Wang, Romain Egele, J Austin Ellis, Matthias Maiterth, Guojing Cong, Feiyi Wang, and Prasanna Balaprakash. 2024. Optimizing distributed training on frontier for large language models. In *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*. Prometeus GmbH, 1–11.
[66] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems* 35 (2022), 30318–30332.
[67] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.
[68] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence* 5, 3 (2023), 220–235.
[69] Docker Inc. 2013. Docker. https://www.docker.com/
[70] Harry Dong, Beidi Chen, and Yuejie Chi. 2023. Towards structured sparsity in transformers for efficient inference. In *Workshop on Efficient Systems for Foundation Models@ ICML2023*. 1–12.
[71] Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. 2024. Flex Attention: A Programming Model for Generating Optimized Attention Kernels. *arXiv preprint arXiv:2412.05496* (2024).
[72] Yixin Dong, Charlie F Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. Xgrammar: Flexible and efficient structured generation engine for large language models. *arXiv preprint arXiv:2411.15100* (2024).
[73] dottxt-ai. 2023. outlines: Make LLMs speak the language of every application. https://github.com/dottxt-ai/outlines
[74] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning*. PMLR, 5547–5569.
[75] Dustin Franklin. 2024. CUDA Containers for Edge AI & Robotics. https://github.com/dusty-nv/jetson-containers
[76] Kazuki Egashira, Mark Vero, Robin Staab, Jingxuan He, and Martin Vechev. 2025. Exploiting llm quantization. *Advances in Neural Information Processing Systems* 37 (2025), 41709–41732.
[77] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme compression of large language models via additive quantization. In *Proceedings of the 41st International Conference on Machine Learning*. 12284–12303.
[78] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. 2018. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks* 107 (2018), 3–11.
[79] Falcon-LLM Team. 2024. The Falcon 3 Family of Open Models. https://huggingface.co/blog/falcon3
[80] Ruibo Fan, Xiangrui Yu, Peijie Dong, Zeyu Li, Gu Gong, Qiang Wang, Wei Wang, and Xiaowen Chu. 2025. SpInfer: Leveraging Low-Level Sparsity for Efficient Large Language Model Inference on GPUs. In *Proceedings of the Twentieth European Conference on Computer Systems*. 243–260.
[81] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
[82] Jiří Filipovič, Matúš Madzin, Jan Fousek, and Luděk Matyska. 2015. Optimizing CUDA code by kernel fusion: application on BLAS. *The Journal of Supercomputing* 71, 10 (2015), 3934–3957.
[83] Fireworks AI, Inc. 2023. Fireworks AI. https://fireworks.ai/
[84] Elias Frantar and Dan Alistarh. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*. PMLR, 10323–10337.

Just Accepted

[85] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
[86] Elias Frantar, Roberto L Castro, Jiale Chen, Torsten Hoefler, and Dan Alistarh. 2025. Marlin: Mixed-precision auto-regressive parallel inference on large language models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 239–251.
[87] FriendliAI Inc. 2023. Friendli Inference: The fastest LLM inference engine on the market. https://friendli.ai/solutions/inference
[88] Roy Frostig, Matthew James Johnson, and Chris Leary. 2019. Compiling machine learning programs via high-level tracing. In *SysML conference 2018*.
[89] Qichen Fu, Minsik Cho, Thomas Merth, Sachin Mehta, Mohammad Rastegari, and Mahyar Najibi. 2024. Lazyllm: Dynamic token pruning for efficient long context llm inference. *arXiv preprint arXiv:2407.14057* (2024).
[90] Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. 2025. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. *arXiv preprint arXiv:2510.03215* (2025).
[91] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the sequential dependency of LLM inference using LOOKAHEAD DECODING. In *Proceedings of the 41st International Conference on Machine Learning*. 14060–14079.
[92] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.
[93] Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, et al. 2024. Seerattention: Learning intrinsic sparse attention in your llms. *arXiv preprint arXiv:2410.13276* (2024).
[94] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801* (2023).
[95] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. 2025. Generating Structured Outputs from Language Models: Benchmark and Studies. *arXiv preprint arXiv:2501.10868* (2025).
[96] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 10932–10952.
[97] Evangelos Georganas, Dhiraj Kalamkar, Alexander Kozlov, and Alexander Heinecke. 2025. ML-SpecQD: Multi-Level Speculative Decoding with Quantized Drafts. *arXiv preprint arXiv:2503.13565* (2025).
[98] Georgi Gerganov. 2022. GGML: Tensor library for machine learning. https://github.com/ggml-org/ggml/blob/master/docs/gguf.md
[99] Georgi Gerganov. 2023. GGUF. https://github.com/ggml-org/ggml/blob/master/docs/gguf.md
[100] ggml-org. 2023. GBNF: Format for defining formal grammars to constrain model outputs in llama.cpp. https://github.com/ggml-org/llama.cpp/blob/master/grammars/README.md
[101] ggml.ai. 2023. llama.cpp. https://github.com/ggml-org/llama.cpp
[102] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems* 6 (2024), 325–338.
[103] GitHub, Inc. 2021. GitHub Copilot: The AI editor for everyone. https://github.com/features/copilot
[104] GNU. 1976. GNU Make. https://www.gnu.org/software/make/
[105] Google. 2009. Go: Build simple, secure, scalable systems with Go. https://go.dev/
[106] Google. 2010. OR-Tools - Google Optimization Tools. https://github.com/google/or-tools
[107] Google. 2023. Gemini. https://gemini.google.com/app
[108] Google. 2024. Google AI Edge SDK. https://developer.android.com/ai/gemini-nano
[109] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
[110] Groq. 2023. GroqFlow. https://github.com/groq/groqflow
[111] Groq, Inc. 2024. GroqCloud: Easy Access to Fast AI Inference. https://groq.com/groqcloud/
[112] gRPC. 2016. gRPC: A high performance, open source universal RPC framework. https://grpc.io/
[113] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).
[114] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2023. Minillm: Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543* (2023).
[115] Yanchu Guan, Dong Wang, Zhixuan Chu, Shiyu Wang, Feiyue Ni, Ruihua Song, and Chenyi Zhuang. 2024. Intelligent agents with llm-based process automation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5018–5027.
[116] guidance-ai. 2024. Low-level Guidance (llguidance). https://github.com/guidance-ai/llguidance
[117] guidance-ai. 2025. Derivative based regex matcher. https://github.com/guidance-ai/derivre
[118] Ozgur Guldogan, Jackson Kunde, Kangwook Lee, and Ramtin Pedarsani. 2024. Multi-bin batching for increasing LLM inference throughput. *arXiv preprint arXiv:2412.04504* (2024).

Just Accepted

[119] Cong Guo, Feng Cheng, Zhixu Du, James Kiessling, Jonathan Ku, Shiyu Li, Ziru Li, Mingyuan Ma, Tergel Molom-Ochir, Benjamin Morris, et al. 2025. A survey: Collaborative hardware and software design in the era of large language models. *IEEE Circuits and Systems Magazine* 25, 1 (2025), 35–57.
[120] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
[121] Gurobi Optimization, LLC. 2008. Gurobi Optimizer - The World's Fastest Solver. https://www.gurobi.com/solutions/gurobi-optimizer/
[122] Jan Hansen-Palmus, Michael Truong Le, Oliver Hausdörfer, and Alok Verma. 2024. Communication Compression for Tensor Parallel LLM Inference. *arXiv preprint arXiv:2411.09510* (2024).
[123] Shibo Hao, Yi Gu, Haotian Luo, Tianyang Liu, Xiyan Shao, Xinyuan Wang, Shuhua Xie, Haodi Ma, Adithya Samavedhi, Qiyue Gao, et al. 2024. Llm reasoners: New evaluation, library, and analysis of step-by-step reasoning with large language models. *arXiv preprint arXiv:2404.05221* (2024).
[124] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.
[125] Yicong He and George K Atia. 2023. Scalable and robust tensor ring decomposition for large-scale data. In *Uncertainty in Artificial Intelligence*. PMLR, 860–869.
[126] Yongjun He, Yao Lu, and Gustavo Alonso. 2024. Deferred continuous batching in resource-efficient large language model serving. In *Proceedings of the 4th Workshop on Machine Learning and Systems*. 98–106.
[127] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415* (2016).
[128] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 722–737.
[129] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, et al. 2024. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671* (2024).
[130] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Yuhan Dong, Yu Wang, et al. 2024. Flashdecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics. *Proceedings of Machine Learning and Systems* 6 (2024), 148–161.
[131] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37 (2024), 1270–1303.
[132] CunChen Hu, HeYang Huang, LiangLiang Xu, XuSheng Chen, Chenxi Wang, Jiang Xu, Shuang Chen, Hao Feng, Sa Wang, Yungang Bao, et al. 2025. ShuffleInfer: Disaggregate LLM Inference for Mixed Downstream Workloads. *ACM Transactions on Architecture and Code Optimization* (2025).
[133] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
[134] Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, et al. 2024. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143* (2024).
[135] Yang Hu, Connor Imes, Xuanang Zhao, Souvik Kundu, Peter A Beerel, Stephen P Crago, and John Paul N Walters. 2021. Pipeline parallelism for inference on heterogeneous edge computing. *arXiv preprint arXiv:2110.14895* (2021).
[136] Zhiqiang Hu, Lei Wang, Yihuai Lan, Wanyu Xu, Ee-Peng Lim, Lidong Bing, Xing Xu, Soujanya Poria, and Roy Ka-Wei Lee. 2023. Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models. *arXiv preprint arXiv:2304.01933* (2023).
[137] Hugging Face. 2022. Safetensors: ML Safer For All. https://huggingface.co/docs/safetensors/index
[138] Hugging Face. 2022. Spaces: The AI App Directory. https://huggingface.co/spaces
[139] Hugging Face. 2023. Text Generation Inference. https://huggingface.co/docs/text-generation-inference/index
[140] Hugging Face. 2023. TRL - Transformer Reinforcement Learning. https://github.com/huggingface/trl
[141] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
[142] IBM. 2025. IBM Granite 4.0. https://huggingface.co/ibm-granite
[143] Intel. 2020. Intel Neural Compressor - An open-source Python library supporting popular model compression techniques on all mainstream deep learning frameworks. https://github.com/intel/neural-compressor
[144] Intel technologies. 2023. Intel Data Center GPU Max Series. https://www.intel.com/content/www/us/en/products/details/discrete-gpus/data-center-gpu/max-series.html
[145] Chandra Irugalbandara. 2024. Meaning Typed Prompting: A Technique for Efficient, Reliable Structured Output Generation. *arXiv preprint arXiv:2410.18146* (2024).

Just Accepted

[146] Ajay Jaiswal, Lu Yin, Zhenyu Zhang, Shiwei Liu, Jiawei Zhao, Yuandong Tian, and Zhangyang Wang. 2024. From galore to welore: How low-rank weights non-uniformly emerge from low-rank gradients. *arXiv preprint arXiv:2407.11239* (2024).
[147] Jiaming Ji, Tianyi Qiu, Boyuan Chen, Borong Zhang, Hantao Lou, Kaile Wang, Yawen Duan, Zhonghao He, Jiayi Zhou, Zhaowei Zhang, et al. 2023. Ai alignment: A comprehensive survey. *arXiv preprint arXiv:2310.19852* (2023).
[148] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *CoRR* abs/2310.06825 (2023). doi:10.48550/ARXIV.2310.06825 arXiv:2310.06825
[149] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
[150] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 13358–13376.
[151] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.
[152] Christoforos Kachris. 2025. A survey on hardware accelerators for large language models. *Applied Sciences* 15, 2 (2025), 586.
[153] Andreas Kosmas Kakolyris, Dimosthenis Masouros, Sotirios Xydis, and Dimitrios Soudris. 2024. Slo-aware gpu dvfs for energy-efficient llm inference serving. *IEEE Computer Architecture Letters* 23, 2 (2024), 150–153.
[154] Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2024. Pod-attention: Unlocking full prefill-decode overlap for faster llm inference. *arXiv preprint arXiv:2410.18038* (2024).
[155] Roman Kaplan. 2024. Intel gaudi 3 ai accelerator: Architected for gen ai training and inference. In *2024 IEEE Hot Chips 36 Symposium (HCS)*. IEEE, 1–16.
[156] Ayush Kaushal, Tejas Vaidhya, and Irina Rish. 2023. Lord: Low rank decomposition of monolingual code llms for one-shot compression. *arXiv preprint arXiv:2309.14021* (2023).
[157] Byeongho Kim, Sanghoon Cha, Sangsoo Park, Jieun Lee, Sukhan Lee, Shin-haeng Kang, Jinin So, Kyungsoo Kim, Jin Jung, Jong-Geon Lee, et al. 2024. The breakthrough memory solutions for improved performance on llm inference. *IEEE Micro* (2024).
[158] Gun Il Kim, Sunga Hwang, and Beakcheol Jang. 2024. Efficient Compressing and Tuning Methods for Large Language Models: A Systematic Literature Review. *Comput. Surveys* (2024).
[159] Gun Il Kim, Sunga Hwang, and Beakcheol Jang. 2025. Efficient compressing and tuning methods for large language models: A systematic literature review. *Comput. Surveys* 57, 10 (2025), 1–39.
[160] Hanjoon Kim, Younggeun Choi, Junyoung Park, Byeongwook Bae, Hyunmin Jeong, Sang Min Lee, Jeseung Yeon, Minho Kim, Changjae Park, Boncheol Gu, et al. 2024. TCP: A Tensor Contraction Processor for AI Workloads Industrial Product. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 890–902.
[161] Taeuk Kim, Kang Min Yoo, and Sang-goo Lee. 2021. Self-Guided Contrastive Learning for BERT Sentence Representations. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 2528–2540.
[162] Toshiaki Koike-Akino, Xiangyu Chen, Jing Liu, Ye Wang, Matthew Brand, et al. 2025. LatentLLM: Attention-Aware Joint Tensor Compression. *arXiv preprint arXiv:2505.18413* (2025).
[163] Oleksii Kuchaiev, Jason Li, Huyen Nguyen, Oleksii Hrinchuk, Ryan Leary, Boris Ginsburg, Samuel Kriman, Stanislav Beliaev, Vitaly Lavrukhin, Jack Cook, et al. 2019. Nemo: a toolkit for building ai applications using neural modules. *arXiv preprint arXiv:1909.09577* (2019).
[164] Taku Kudo and John Richardson. 2018. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *arXiv preprint arXiv:1808.06226* (2018).
[165] Komal Kumar, Tajamul Ashraf, Omkar Thawakar, Rao Muhammad Anwer, Hisham Cholakkal, Mubarak Shah, Ming-Hsuan Yang, Phillip HS Torr, Salman Khan, and Fahad Shahbaz Khan. 2025. Llm post-training: A deep dive into reasoning large language models. *arXiv preprint arXiv:2502.21321* (2025).
[166] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
[167] LangChain. 2022. LangChain: The largest community building the future of LLM apps. https://www.langchain.com/langchain
[168] Lark-Parser. 2018. Lark - a parsing toolkit for Python. https://github.com/lark-parser/lark
[169] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2–14.

Just Accepted

[170] Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Ren Lu, Thomas Mesnard, Johan Ferret, Colton Bishop, Ethan Hall, Victor Carbune, and Abhinav Rastogi. 2023. Rlaif: Scaling reinforcement learning from human feedback with ai feedback. (2023).
[171] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. {InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.
[172] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691* (2021).
[173] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.
[174] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
[175] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. Llms-as-judges: a comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579* (2024).
[176] Jinhao Li, Jiaming Xu, Shan Huang, Yonghua Chen, Wen Li, Jun Liu, Yaoxiu Lian, Jiayi Pan, Li Ding, Hao Zhou, et al. 2024. Large language model inference acceleration: A comprehensive hardware perspective. *arXiv preprint arXiv:2410.04466* (2024).
[177] Jiaxi Li, Lu Yin, Li Shen, Jinjin Xu, Liwu Xu, Tianjin Huang, Wenwu Wang, Shiwei Liu, and Xilu Wang. 2025. LOST: Low-rank and Sparse Pre-training for Large Language Models. *arXiv preprint arXiv:2508.02668* (2025).
[178] Qingyuan Li, Yifan Zhang, Liang Li, Peng Yao, Bo Zhang, Xiangxiang Chu, Yerui Sun, Li Du, and Yuchen Xie. 2023. Fptq: Fine-grained post-training quantization for large language models. *arXiv preprint arXiv:2308.15987* (2023).
[179] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth* 1, 1 (2024), 9.
[180] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463* (2023).
[181] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing Context to Enhance Inference Efficiency of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 6342–6353.
[182] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858* (2024).
[183] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077* (2024).
[184] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. *arXiv preprint arXiv:2503.01840* (2025).
[185] Zongbiao Li, Xiezhao Li, Yinghao Cui, Yijun Chen, Zhixuan Gu, Yuxuan Liu, Wenbo Zhu, Fei Jia, Ke Liu, Qifeng Li, et al. 2024. Automatically Planning Optimal Parallel Strategy for Large Language Models. *arXiv preprint arXiv:2501.00254* (2024).
[186] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, et al. 2024. TorchTitan: One-stop PyTorch native solution for production ready LLM pre-training. *arXiv preprint arXiv:2410.06511* (2024).
[187] Zi Liang, Qingqing Ye, Yanyun Wang, Sen Zhang, Yaxin Xiao, Ronghua Li, Jianliang Xu, and Haibo Hu. 2024. " Yes, My LoRD." Guiding Language Model Extraction with Locality Reinforced Distillation. *arXiv preprint arXiv:2409.02718* (2024).
[188] Heng Liao, Jiajin Tu, Jing Xia, Hu Liu, Xiping Zhou, Honghui Yuan, and Yuxing Hu. 2021. Ascend: a scalable and unified architecture for ubiquitous deep neural network computing: Industry track paper. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 789–801.
[189] Sean Lie. 2024. Inside the cerebras wafer-scale cluster. *IEEE Micro* (2024).
[190] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, et al. 2024. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887* (2024).
[191] Lightllm Team. 2023. LightLLM: A Light and Fast Inference Service for LLM. https://github.com/ModelTC/lightllm
[192] Lightning AI. 2023. Lightning Fabric. https://lightning.ai/docs/fabric/stable/
[193] Lightning AI. 2023. Lit-LLaMA. https://github.com/Lightning-AI/lit-llama
[194] Lightning AI. 2023. LitGPT: Use, finetune, pretrain, and deploy LLMs Lightning fast. https://github.com/Lightning-AI/litgpt
[195] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.
[196] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).

Just Accepted

[197] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[198] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Reza Haffari, and Bohan Zhuang. 2024. Minicache: Kv cache compression in depth dimension for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 139997–140031.

[199] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, et al. 2024. Retrievalattention: Accelerating long-context llm inference via vector retrieval. *arXiv preprint arXiv:2409.10516* (2024).

[200] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. 2024. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 26296–26306.

[201] Michael Xieyang Liu, Frederick Liu, Alexander J Fiannaca, Terry Koo, Lucas Dixon, Michael Terry, and Carrie J Cai. 2024. " We Need Structured Output": Towards User-centered Constraints on Large Language Model Output. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–9.

[202] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E Gonzalez, Ion Stoica, and Matei Zaharia. 2024. Optimizing llm queries in relational workloads. *arXiv preprint arXiv:2403.05821* (2024).

[203] Xiaodong Liu, Hao Cheng, Pengcheng He, Weizhu Chen, Yu Wang, Hoifung Poon, and Jianfeng Gao. 2020. Adversarial training for large neural language models. *arXiv preprint arXiv:2004.08994* (2020).

[204] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. 2024. Online speculative decoding. In *Proceedings of the 41st International Conference on Machine Learning*. 31131–31146.

[205] Xiaoran Liu, Hang Yan, Shuo Zhang, Chenxin An, Xipeng Qiu, and Dahua Lin. 2023. Scaling laws of rope-based extrapolation. *arXiv preprint arXiv:2310.05209* (2023).

[206] Xiaoyu Liu, Yun Zhang, Wei Li, Simiao Li, Xudong Huang, Hanting Chen, Yehui Tang, Jie Hu, Zhiwei Xiong, and Yunhe Wang. 2024. Multi-Granularity Semantic Revision for Large Language Model Distillation. *arXiv preprint arXiv:2407.10068* (2024).

[207] Yu Liu, Duantengchuan Li, Kaili Wang, Zhuoran Xiong, Fobo Shi, Jian Wang, Bing Li, and Bo Hang. 2024. Are LLMs good at structured outputs? A benchmark for evaluating structured output capabilities in LLMs. *Information Processing & Management* 61, 5 (2024), 103809.

[208] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.

[209] Yijiang Liu, Huanrui Yang, Youxin Chen, Rongyu Zhang, Miao Wang, Yuan Du, and Li Du. 2025. PAT: Pruning-Aware Tuning for Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24686–24695.

[210] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. 2023. Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888* (2023).

[211] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, et al. 2023. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning*. PMLR, 22137–22176.

[212] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750* (2024).

[213] LMDeploy Contributors. 2023. LMDeploy: A Toolkit for Compressing, Deploying, and Serving LLM. https://github.com/InternLM/lmdeploy

[214] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, et al. 2025. Moba: Mixture of block attention for long-context llms. *arXiv preprint arXiv:2502.13189* (2025).

[215] Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583* (2023).

[216] Kai Lv, Yuqing Yang, Tengxiao Liu, Qinghui Gao, Qipeng Guo, and Xipeng Qiu. 2023. Full parameter fine-tuning for large language models with limited resources. *arXiv preprint arXiv:2306.09782* (2023).

[217] Ruilong Ma, Xiang Yang, Jingyu Wang, Qi Qi, Haifeng Sun, Jing Wang, Zirui Zhuang, and Jianxin Liao. 2024. HPipe: Large Language Model Pipeline Parallelism for Long Context on Heterogeneous Cost-effective Devices. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*. 1–9.

[218] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Lifeng Dong, Ruiping Wang, Jilong Xue, and Furu Wei. 2024. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764* 1 (2024).

[219] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems* 36 (2023), 21702–21720.

[220] Yuexiao Ma, Huixia Li, Xiawu Zheng, Feng Ling, Xuefeng Xiao, Rui Wang, Shilei Wen, Fei Chao, and Rongrong Ji. 2024. Affinequant: Affine transformation quantization for large language models. *arXiv preprint arXiv:2403.12544* (2024).

Just Accepted

[221] Eric Mahurin. 2023. Qualocmm® Hexagon™ NPU.. In *HCS*. 1–19.
[222] Mehdi Makni, Kayhan Behdin, Zheng Xu, Natalia Ponomareva, and Rahul Mazumder. 2025. A unified framework for Sparse plus Low-Rank Matrix Decomposition for LLMs. In *The Second Conference on Parsimony and Learning (Proceedings Track)*. https://openreview.net/forum?id=hyN75SAJTI
[223] Todor Markov, Chong Zhang, Sandhini Agarwal, Florentine Eloundou Nekoul, Theodore Lee, Steven Adler, Angela Jiang, and Lilian Weng. 2023. A holistic approach to undesired content detection in the real world. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 15009–15018.
[224] Jeffrey Marron. 2024. Implementing the Health Insurance Portability and Accountability Act (HIPAA) Security Rule A Cybersecurity Resource Guide.
[225] Max Howell. 2016. Homebrew: The Missing Package Manager for macOS (or Linux). https://brew.sh/
[226] Xiandong Meng, Yan Wu, Yexin Tian, Xin Hu, Tianze Kang, and Junliang Du. 2025. Collaborative distillation strategies for parameter-efficient language model deployment. *arXiv preprint arXiv:2507.15198* (2025).
[227] meta. 2025. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/
[228] Meta Platforms, Inc. 2022. xFormers: A modular and hackable Transformer modelling library. https://github.com/facebookresearch/xformers
[229] Meta-PyTorch. 2024. torchtune: PyTorch's finetuning library. https//github.com/pytorch/torchtune
[230] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. 2023. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234* (2023).
[231] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 932–949.
[232] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient transformer training over multiple gpus using automatic parallelism. *arXiv preprint arXiv:2211.13878* (2022).
[233] Microsoft. 2022. DeepSpeed Model Implementations for Inference (MII). https://github.com/deepspeedai/DeepSpeed-MII
[234] MLC-AI. 2023. MLC LLM: Universal LLM Deployment Engine With ML Compilation. https://llm.mlc.ai/
[235] Chakshu Moar, Faraz Tahmasebi, Michael Pellauer, and Hyoukjun Kwon. 2024. Characterizing the Accuracy-Efficiency Trade-off of Low-rank Decomposition in Language Models. In *2024 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 194–209.
[236] Modular Inc. 2023. Mojo: Powerful CPU+GPU Programming. https://www.modular.com/mojo
[237] Modular Inc. 2024. MAX. https://www.modular.com/max
[238] Ramaswami Mohandoss. 2024. Context-based semantic caching for llm applications. In *2024 IEEE Conference on Artificial Intelligence (CAI)*. IEEE, 371–376.
[239] Raha Moraffah, Shubh Khandelwal, Amrita Bhattacharjee, and Huan Liu. 2024. Adversarial text purification: A large language model approach for defense. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 65–77.
[240] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 561–577.
[241] Sania Nayab, Giulio Rossolini, Marco Simoni, Andrea Saracino, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. 2024. Concise thoughts: Impact of output length on llm reasoning and cost. *arXiv preprint arXiv:2407.19825* (2024).
[242] NetEase-FuXi. 2024. EETQ: Easy & Efficient Quantization for Transformers. https://github.com/NetEase-FuXi/EETQ
[243] Liang-bo Ning, Shijie Wang, Wenqi Fan, Qing Li, Xin Xu, Hao Chen, and Feiran Huang. 2024. Cheatagent: Attacking llm-empowered recommender systems via llm agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2284–2295.
[244] Ninja-build. 2012. Ninja. https://github.com/ninja-build/ninja
[245] Noam Gat. 2024. lm-format-enforce: Enforce the output format (JSON Schema, Regex etc) of a language model. https://github.com/noamgat/lm-format-enforcer
[246] NVIDIA. 2007. Parallel Thread Execution ISA. https://docs.nvidia.com/cuda/parallel-thread-execution/
[247] NVIDIA. 2016. NVIDIA Collective Communications Library (NCCL). https://developer.nvidia.com/nccl
[248] NVIDIA. 2018. NVIDIA Triton Inference Server. https://developer.nvidia.com/triton-inference-server
[249] NVIDIA. 2019. FasterTransformer. https://github.com/NVIDIA/FasterTransformer
[250] NVIDIA. 2019. NVIDIA TensorRT. https://github.com/NVIDIA/TensorRT
[251] NVIDIA. 2022. cuSPARSE: GPU library APIs for sparse computation. https://developer.nvidia.com/cusparse

Just Accepted

[252] NVIDIA. 2023. NVIDIA TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference. https://github.com/NVIDIA/TensorRT-LLM
[253] NVIDIA. 2025. NVIDIA Dynamo. https://github.com/ai-dynamo/dynamo
[254] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. 2024. Exegpt: Constraint-aware resource scheduling for llm inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 369–384.
[255] Ollama. 2023. Ollama. https://ollama.com/
[256] Open WebUI. 2024. Open WebUI. https://openwebui.com/
[257] OpenAI. 2022. ChatGPT. https://chatgpt.com/ March 12, 2025.
[258] OpenAI. 2024. Structured Outputs. https://platform.openai.com/docs/guides/structured-outputs
[259] Cristobal Ortega, Yann Falevoz, and Renaud Ayrignac. 2024. PIM-AI: A Novel Architecture for High-Efficiency LLM Inference. *arXiv preprint arXiv:2411.17309* (2024).
[260] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D Owens. 2023. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 429–431.
[261] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. 2024. Marconi: Prefix caching for the era of hybrid llms. *arXiv preprint arXiv:2411.19379* (2024).
[262] Seungcheol Park, Jaehyeon Choi, Sojin Lee, and U Kang. 2024. A comprehensive survey of compression algorithms for language models. *arXiv preprint arXiv:2401.15347* (2024).
[263] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
[264] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
[265] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. 2023. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048* (2023).
[266] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5 (2023), 606–624.
[267] Rohan Baskar Prabhakar, Hengrui Zhang, and David Wentzlaff. 2024. Kraken: Inherently Parallel Transformers For Efficient Multi-Device Inference. *Advances in Neural Information Processing Systems* 37 (2024), 7957–7980.
[268] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1133–1150.
[269] Ofir Press, Noah A Smith, and Mike Lewis. 2021. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409* (2021).
[270] Python Packaging Authority. 2008. pip - The Python Package Installer. https://github.com/pypa/pip
[271] PyTorch. 2020. PyTorch/XLA. https://github.com/pytorch/xla
[272] Wenjin Qin, Hailin Wang, Feng Zhang, Weijun Ma, Jianjun Wang, and Tingwen Huang. 2024. Nonconvex robust high-order tensor completion using randomized low-rank approximation. *IEEE Transactions on Image Processing* 33 (2024), 2835–2850.
[273] Zhen Qin and Zhihui Zhu. 2025. Computational and statistical guarantees for tensor-on-tensor regression with tensor train decomposition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
[274] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
[275] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
[276] Nived Rajaraman, Jiantao Jiao, and Kannan Ramchandran. 2024. Toward a theory of tokenization in llms. *arXiv preprint arXiv:2404.08335* (2024).
[277] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.
[278] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

Just Accepted

[279] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. 2023. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537* (2023).
[280] Charlie F Ruan, Yucheng Qin, Xun Zhou, Ruihang Lai, Hongyi Jin, Yixin Dong, Bohan Hou, Meng-Shiun Yu, Yiyan Zhai, Sudeep Agarwal, et al. 2024. WebLLM: A High-Performance In-Browser LLM Inference Engine. *arXiv preprint arXiv:2412.15803* (2024).
[281] Amit Sabne. 2020. Xla: Compiling machine learning for peak performance.
[282] Rajarshi Saha, Naomi Sagan, Varun Srivastava, Andrea Goldsmith, and Mert Pilanci. 2024. Compressing large language models using low rank and low precision decomposition. *Advances in Neural Information Processing Systems* 37 (2024), 88981–89018.
[283] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. 2023. From words to watts: Benchmarking the energy costs of large language model inference. In *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–9.
[284] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).
[285] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.
[286] Hang Shao, Bei Liu, and Yanmin Qian. 2024. One-shot sensitivity-aware mixed sparsity pruning for large language models. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 11296–11300.
[287] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. 2018. Self-Attention with Relative Position Representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. 464–468.
[288] Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150* (2019).
[289] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1279–1297.
[290] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. 2023. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. *CoRR* (2023).
[291] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. 2024. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 965–988.
[292] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
[293] Chao Wang Fan Yin Shuai Shi Yubo Wang Yi Zhang Yingyi Huang Haoshuai Zheng Yineng Zhang Shenggui Li, Yikai Zhu. 2025. SpecForge: Train speculative decoding models effortlessly. https://github.com/sgl-project/specforge
[294] Zhang Shengyu, Dong Linfeng, Li Xiaoya, Zhang Sen, Sun Xiaofei, Wang Shuhe, Li Jiwei, Runyi Hu, Zhang Tianwei, Fei Wu, et al. 2023. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792* (2023).
[295] Shengyu Liu. 2024. SwiftTransformer. https://github.com/LLMServe/SwiftTransformer
[296] Jiho Shin, Hoeseok Yang, and Youngmin Yi. 2024. SparseInfer: Training-free Prediction of Activation Sparsity for Fast LLM Inference. *arXiv preprint arXiv:2411.12692* (2024).
[297] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
[298] KaShun Shum, Minrui Xu, Jianshu Zhang, Zixin Chen, Shizhe Diao, Hanze Dong, Jipeng Zhang, and Muhammad Omer Raza. 2024. First: Teach a reliable large language model through efficient trustworthy distillation. *arXiv preprint arXiv:2408.12168* (2024).
[299] Min Si, Pavan Balaji, Yongzhou Chen, Ching-Hsiang Chu, Adi Gangidi, Saif Hasan, Subodh Iyengar, Dan Johnson, Bingzhe Liu, Jingliang Ren, et al. 2025. Collective Communication for 100k+ GPUs. *arXiv preprint arXiv:2510.20171* (2025).
[300] Alan Smith, Gabriel H Loh, John Wuu, Samuel Naffziger, Tyrone Huang, Hugh McIntyre, Ramon Mangaser, Wonjun Jung, and Raja Swaminathan. 2024. AMD Instinct™ MI300X Accelerator: Packaging and Architecture Co-Optimization. In *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 1–2.
[301] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).
[302] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 590–606.
[303] Benjamin Spector and Chris Re. 2023. Accelerating llm inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623* (2023).
[304] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. 2022. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615* (2022).

Just Accepted

[305] Jovan Stojkovic, Esha Choukse, Chaojie Zhang, Inigo Goiri, and Josep Torrellas. 2024. Towards greener llms: Bringing energy-efficiency to the forefront of llm inference. *arXiv preprint arXiv:2403.20306* (2024).
[306] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *Proceedings of the 41st International Conference on Machine Learning*. 46745–46771.
[307] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
[308] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. 2024. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference. *arXiv preprint arXiv:2410.21465* (2024).
[309] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. 2023. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695* (2023).
[310] Wei Sun, Ang Li, Sander Stuijk, and Henk Corporaal. 2024. How much can we gain from Tensor Kernel Fusion on GPUs? *IEEE Access* 12 (2024), 126135–126144.
[311] Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2025. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967* (2025).
[312] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. 2023. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621* (2023).
[313] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. QUEST: query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*. 47901–47911.
[314] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* (2024).
[315] Qwen Team. 2024. Qwq: Reflect deeply on the boundaries of the unknown. *Hugging Face* (2024).
[316] The ZeroMQ authors. 2023. ZeroMQ: An open-source universal messaging library. https://zeromq.org/
[317] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 10–19.
[318] Together AI. 2023. Together Inference. https://www.together.ai/products#inference
[319] torchao. 2024. TorchAO: PyTorch-Native Training-to-Serving Model Optimization. https://github.com/pytorch/ao
[320] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
[321] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
[322] Albert Tseng, Tao Yu, and Youngsuk Park. 2025. Training llms with mxfp4. *arXiv preprint arXiv:2502.20586* (2025).
[323] Turboderp. 2023. ExLlamaV2: Inference library for running local LLMs on modern consumer GPUs. https://github.com/turboderp-org/exllamav2
[324] James Turnbull. 2018. *Monitoring with Prometheus*. Turnbull Press.
[325] unsloth. 2023. unsloth. https://unsloth.ai/
[326] Tim Valicenti, Justice Vidal, and Ritik Patnaik. 2023. Mini-gpts: Efficient large language models through contextual pruning. *arXiv preprint arXiv:2312.12682* (2023).
[327] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[328] vLLM. 2024. GuideLLM: Scalable Inference and Optimization for Large Language Models. https://github.com/vllm-project/guidellm
[329] vLLM Team. 2024. LLM Compressor: An easy-to-use library for optimizing models for deployment with vllm. https://github.com/vllm-project/llm-compressor
[330] David Vos, Till Döhmen, and Sebastian Schelter. 2022. Towards parameter-efficient automation of data wrangling tasks with prefix-tuning. In *NeurIPS 2022 First Table Representation Workshop*. 1–9.
[331] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. 2024. Efficient Large Language Models: A Survey. *Transactions on Machine Learning Research* (2024). https://openreview.net/forum?id=bsCCJHbO8A Survey Certification.
[332] Bailin Wang, Zi Wang, Xuezhi Wang, Yuan Cao, Rif A Saurous, and Yoon Kim. 2023. Grammar prompting for domain-specific language generation with large language models. *Advances in Neural Information Processing Systems* 36 (2023), 65030–65055.
[333] Binghai Wang, Rui Zheng, Lu Chen, Yan Liu, Shihan Dou, Caishuang Huang, Wei Shen, Senjie Jin, Enyu Zhou, Chenyu Shi, et al. 2024. Secrets of rlhf in large language models part ii: Reward modeling. *arXiv preprint arXiv:2401.06080* (2024).

Just Accepted

[334] Chengyu Wang, Junbing Yan, Yuanhao Yue, and Jun Huang. 2025. DistilQwen2. 5: Industrial Practices of Training Distilled Open Lightweight Language Models. *arXiv preprint arXiv:2504.15027* (2025).
[335] Jun Wang, Eleftheria Briakou, Hamid Dadkhahi, Rishabh Agarwal, Colin Cherry, and Trevor Cohn. 2024. Don't Throw Away Data: Better Sequence Knowledge Distillation. *arXiv preprint arXiv:2407.10456* (2024).
[336] Jinheng Wang, Hansong Zhou, Ting Song, Shaoguang Mao, Shuming Ma, Hongyu Wang, Yan Xia, and Furu Wei. 2024. 1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1. 58 Inference on CPUs. *arXiv preprint arXiv:2410.16144* (2024).
[337] Luping Wang, Sheng Chen, Linnan Jiang, Shu Pan, Runze Cai, Sen Yang, and Fei Yang. 2025. Parameter-efficient fine-tuning in large language models: a survey of methodologies. *Artificial Intelligence Review* 58, 8 (2025), 227.
[338] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. 2024. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191* (2024).
[339] Thomas Wang, Adam Roberts, Daniel Hesslow, Teven Le Scao, Hyung Won Chung, Iz Beltagy, Julien Launay, and Colin Raffel. 2022. What language model architecture and pretraining objective works best for zero-shot generalization?. In *International Conference on Machine Learning*. PMLR, 22964–22984.
[340] Wenxiao Wang, Wei Chen, Yicong Luo, Yongliu Long, Zhengkai Lin, Liye Zhang, Binbin Lin, Deng Cai, and Xiaofei He. 2024. Model compression and efficient inference for large language models: A survey. *arXiv preprint arXiv:2402.09748* (2024).
[341] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. 2024. Svd-llm: Truncation-aware singular value decomposition for large language model compression. *arXiv preprint arXiv:2403.07378* (2024).
[342] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. 2024. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
[343] Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. 2024. Model tells you where to merge: Adaptive kv cache merging for llms on long-context tasks. *arXiv preprint arXiv:2407.08454* (2024).
[344] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2025. T-mac: Cpu renaissance via table lookup for low-bit llm deployment on edge. In *Proceedings of the Twentieth European Conference on Computer Systems*. 278–292.
[345] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* (2022).
[346] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
[347] Martin Weyssow, Aton Kamanda, Xin Zhou, and Houari Sahraoui. 2024. Codeultrafeedback: An llm-as-a-judge dataset for aligning large language models to coding preferences. *arXiv preprint arXiv:2403.09032* (2024).
[348] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. 2025. Exploring parameter-efficient fine-tuning techniques for code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 34, 7 (2025), 1–25.
[349] Brandon T Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702* (2023).
[350] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. 2022. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100* (2022).
[351] Bo Wu, Sid Wang, Yunhao Tang, Jia Ding, Eryk Helenowski, Liang Tan, Tengyu Xu, Tushar Gowda, Zhengxing Chen, Chen Zhu, et al. 2025. Llamarl: A distributed asynchronous reinforcement learning framework for efficient large-scale llm trainin. *arXiv preprint arXiv:2505.24034* (2025).
[352] Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. 2024. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. In *Findings of the Association for Computational Linguistics ACL 2024*. 7655–7671.
[353] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. 2023. Flash-llm: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity. *arXiv preprint arXiv:2309.10285* (2023).
[354] Lizhi Xiang, Omid Asudeh, Gerald Sabin, Aravind Sukumaran-Rajam, and P Sadayappan. 2025. cuTeSpMM: Accelerating Sparse-Dense Matrix Multiplication using GPU Tensor Cores. *arXiv preprint arXiv:2504.06443* (2025).
[355] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
[356] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453* (2023).
[357] Mengwei Xu, Dongqi Cai, Wangsong Yin, Shangguang Wang, Xin Jin, and Xuanzhe Liu. 2025. Resource-efficient algorithms and systems of foundation models: A survey. *Comput. Surveys* 57, 5 (2025), 1–39.
[358] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, Daliang Xu, Qipeng Wang, Bingyang Wu, Yihao Zhao, Chen Yang, Shihe Wang, et al. 2024. A survey of resource-efficient llm and multimodal foundation models. *arXiv preprint arXiv:2401.08092* (2024).

Just Accepted

[359] Shusheng Xu, Wei Fu, Jiaxuan Gao, Wenjie Ye, Weilin Liu, Zhiyu Mei, Guangju Wang, Chao Yu, and Yi Wu. 2024. Is dpo superior to ppo for llm alignment? a comprehensive study. *arXiv preprint arXiv:2404.10719* (2024).
[360] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. 2024. A survey on knowledge distillation of large language models. *arXiv preprint arXiv:2402.13116* (2024).
[361] ZHAO XUANLEI, Bin Jia, Haotian Zhou, Ziming Liu, Shenggan Cheng, and Yang You. 2024. Hetegen: Efficient heterogeneous parallel inference for large language models on resource-constrained devices. *Proceedings of Machine Learning and Systems* 6 (2024), 162–172.
[362] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. 2024. Powerinfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282* (2024).
[363] Minghao Yan, Saurabh Agarwal, and Shivaram Venkataraman. 2025. Decoding speculative decoding. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 6460–6473.
[364] Chuanpeng Yang, Yao Zhu, Wang Lu, Yidong Wang, Qian Chen, Chenlong Gao, Bingjie Yan, and Yiqiang Chen. 2024. Survey on knowledge distillation for large language models: methods, evaluation, and application. *ACM Transactions on Intelligent Systems and Technology* (2024).
[365] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. 2025. Kvlink: Accelerating large language models via efficient kv cache reuse. *arXiv preprint arXiv:2502.16002* (2025).
[366] Junjie Yang, Junhao Song, Xudong Han, Ziqian Bi, Tianyang Wang, Chia Xin Liang, Xinyuan Song, Yichao Zhang, Qian Niu, Benji Peng, et al. 2025. Feature alignment and representation transfer in knowledge distillation for large language models. *arXiv preprint arXiv:2504.13825* (2025).
[367] Mingke Yang, Yuqi Chen, Yi Liu, and Ling Shi. 2024. Distillseq: A framework for safety alignment testing in large language models using knowledge distillation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 578–589.
[368] Penghui Yang, Cunxiao Du, Fengzhuo Zhang, Haonan Wang, Tianyu Pang, Chao Du, and Bo An. [n. d.]. LongSpec: Long-Context Lossless Speculative Decoding with Efficient Drafting and Verification. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*.
[369] Shuo Yang, Ying Sheng, Joseph E Gonzalez, Ion Stoica, and Lianmin Zheng. 2024. Post-training sparse attention with double sparsity. *arXiv preprint arXiv:2408.07092* (2024).
[370] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing* (2024), 100211.
[371] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, et al. 2023. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales. *arXiv preprint arXiv:2308.01320* (2023).
[372] Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, et al. 2025. KVCOMM: Online Cross-context KV-cache Communication for Efficient LLM-based Multi-agent Systems. *arXiv preprint arXiv:2510.12872* (2025).
[373] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. *arXiv preprint arXiv:2402.15220* (2024).
[374] Qinyuan Ye, Maxamed Axmed, Reid Pryzant, and Fereshte Khani. 2023. Prompt engineering a prompt engineer. *arXiv preprint arXiv:2311.05661* (2023).
[375] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005* (2025).
[376] Zihao Ye, Ruihang Lai, Bo-Ru Lu, Chien-Yu Lin, Size Zheng, Lequn Chen, Tianqi Chen, and Luis Ceze. 2024. Cascade Inference: Memory Bandwidth Efficient Shared Prefix Batch Decoding. https://flashinfer.ai/2024/02/02/cascade-inference.html
[377] Wangsong Yin, Mengwei Xu, Yuanchun Li, and Xuanzhe Liu. 2024. Llm as a system service on mobile devices. *arXiv preprint arXiv:2403.11805* (2024).
[378] Yiorgis Gozadinos. 2023. oterm: the text-based terminal client for Ollama. https://github.com/ggozad/oterm
[379] Chengye Yu, Tianyu Wang, Zili Shao, Linjie Zhu, Xu Zhou, and Song Jiang. 2024. Twinpilots: A new computing paradigm for gpu-cpu parallel llm inference. In *Proceedings of the 17th ACM International Systems and Storage Conference*. 91–103.
[380] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
[381] Zhihang Yuan, Yuzhang Shang, Yue Song, Qiang Wu, Yan Yan, and Guangyu Sun. 2023. Asvd: Activation-aware singular value decomposition for compressing large language models. *arXiv preprint arXiv:2312.05821* (2023).

Just Accepted

[382] Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, et al. 2024. Llm inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363* (2024).
[383] Biao Zhang, Ivan Titov, and Rico Sennrich. 2019. Improving deep transformer with depth-scaled initialization and merged attention. *arXiv preprint arXiv:1908.11365* (2019).
[384] Mingjin Zhang, Xiaoming Shen, Jiannong Cao, Zeyang Cui, and Shan Jiang. 2024. Edgeshard: Efficient llm inference via collaborative edge computing. *IEEE Internet of Things Journal* (2024).
[385] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. 2023. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792* (2023).
[386] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
[387] Xuan Zhang, Navid Rajabi, Kevin Duh, and Philipp Koehn. 2023. Machine translation with large language models: Prompting, few-shot learning, and fine-tuning with QLoRA. In *Proceedings of the Eighth Conference on Machine Translation*. 468–481.
[388] Xiangyu Zhang, Yu Zhou, Guang Yang, Harald C Gall, and Taolue Chen. 2025. Anchor attention, small cache: Code generation with large language models. *IEEE Transactions on Software Engineering* (2025).
[389] Yuxin Zhang, Mingbao Lin, Zhihang Lin, Yiting Luo, Ke Li, Fei Chao, Yongjian Wu, and Rongrong Ji. 2022. Learning best combination for efficient n: M sparsity. *Advances in Neural Information Processing Systems* 35 (2022), 941–953.
[390] Yuxin Zhang, Lirui Zhao, Mingbao Lin, Yunyun Sun, Yiwu Yao, Xingjia Han, Jared Tanner, Shiwei Liu, and Rongrong Ji. 2023. Dynamic sparse no training: Training-free fine-tuning for sparse llms. *arXiv preprint arXiv:2310.08915* (2023).
[391] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems* 36 (2023), 34661–34710.
[392] Zhuosheng Zhang, Yao Yao, Aston Zhang, Xiangru Tang, Xinbei Ma, Zhiwei He, Yiming Wang, Mark Gerstein, Rui Wang, Gongshen Liu, et al. 2025. Igniting language intelligence: The hitchhiker's guide from chain-of-thought reasoning to language agents. *Comput. Surveys* 57, 8 (2025), 1–39.
[393] Juntao Zhao, Borui Wan, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. Llm-pq: Serving llm on heterogeneous clusters with phase-aware partition and adaptive quantization. *arXiv preprint arXiv:2403.01136* (2024).
[394] Pu Zhao, Fei Sun, Xuan Shen, Pinrui Yu, Zhenglun Kong, Yanzhi Wang, and Xue Lin. 2024. Pruning Foundation Models for High Accuracy without Retraining. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 9681–9694.
[395] Wenqian Zhao, Lancheng Zou, Zixiao Wang, Xufeng Yao, and Bei Yu. 2025. HAPE: Hardware-Aware LLM Pruning For Efficient On-Device Inference Optimization. *ACM Transactions on Design Automation of Electronic Systems* (2025).
[396] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).
[397] Youpeng Zhao, Di Wu, and Jun Wang. 2024. Alisa: Accelerating large language model inference via sparsity-aware kv caching. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1005–1017.
[398] Haizhong Zheng, Xiaoyan Bai, Xueshen Liu, Zhuoqing Morley Mao, Beidi Chen, Fan Lai, and Atul Prakash. 2024. Learn to be efficient: Build structured sparsity in large language models. *Advances in Neural Information Processing Systems* 37 (2024), 101969–101991.
[399] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems* 37 (2024), 62557–62583.
[400] Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Qin Liu, Yuhao Zhou, et al. 2023. Secrets of rlhf in large language models part i: Ppo. *arXiv preprint arXiv:2307.04964* (2023).
[401] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen. 2024. A review on edge large language models: Design, execution, and applications. *Comput. Surveys* (2024).
[402] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
[403] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. 2024. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294* (2024).
[404] Hanlin Zhu, Banghua Zhu, and Jiantao Jiao. 2024. Efficient prompt caching via embedding similarity. *arXiv preprint arXiv:2402.01173* (2024).
[405] Kan Zhu, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gao, Qinyu Xu, Tian Tang, Zihao Ye, et al. 2024. Nanoflow: Towards optimal large language model serving throughput. *arXiv preprint arXiv:2408.12757* (2024).
[406] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, et al. 2025. MegaScale-Infer: Efficient Mixture-of-Experts Model Serving with Disaggregated Expert Parallelism. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 592–608.

Just Accepted

[407] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2024. A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics* 12 (2024), 1556–1577.
[408] Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Tim Vieira, Mrinmaya Sachan, and Ryan Cotterell. 2023. A formal perspective on byte-pair encoding. *arXiv preprint arXiv:2306.16837* (2023).

Received 8 May 2025; revised 22 January 2026; accepted 8 March 2026

Just Accepted